

Chapter 5

Neural Networks and Deep Learning Algorithms

Overview

5.1 Neural Networks: Structures, Activation Functions and Universal Approximation Theorem

5.2 Perceptron, Multilayer Perceptron and Backpropagation

5.3 Introduction to Deep Learning

5.4 Concepts on Recurrent and Generative Neural Networks

5.1 Neural Networks: Structures, Activation Functions and Universal Approximation Theorem

- Neural Networks
- A neuron is a cell in brain whose principal function is the collection, processing, and dissemination of electrical signals.
- Brains Information processing capacity comes from networks of such neurons.
- Due to this reason some earliest AI work aimed to create such artificial networks/ connectionism / Parallel distributed processing/ Neural computing.

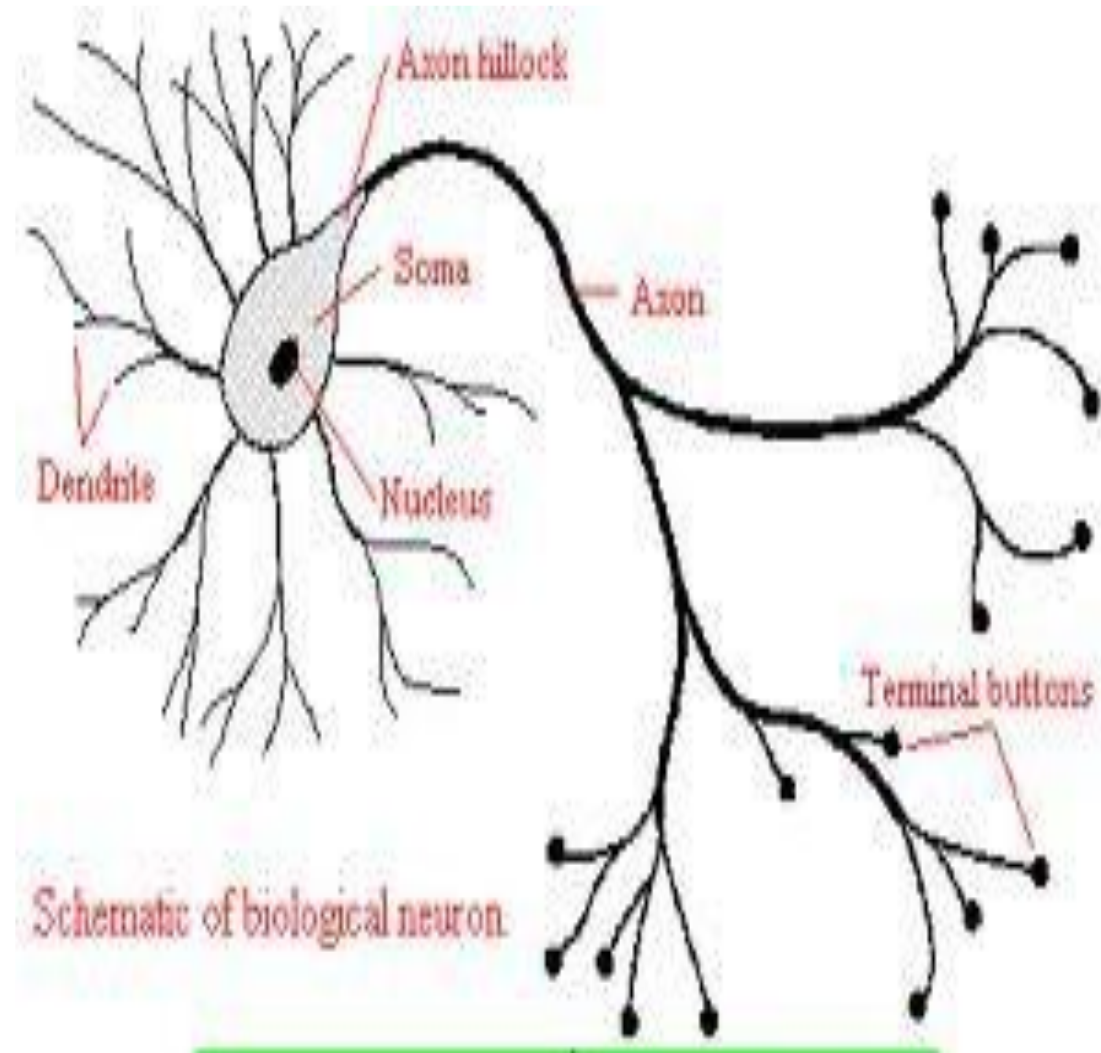
Neural Network

- An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the way biological nervous systems, such as the brain, process information.
- It is composed of a large number of highly interconnected processing elements (neurons) working to solve specific problems.
- ANNs, like people, learn by example.
- An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process.
- Artificial neurons are crude approximations of the neurons found in real brains. They may be physical devices, or purely mathematical constructs.

Neural Network

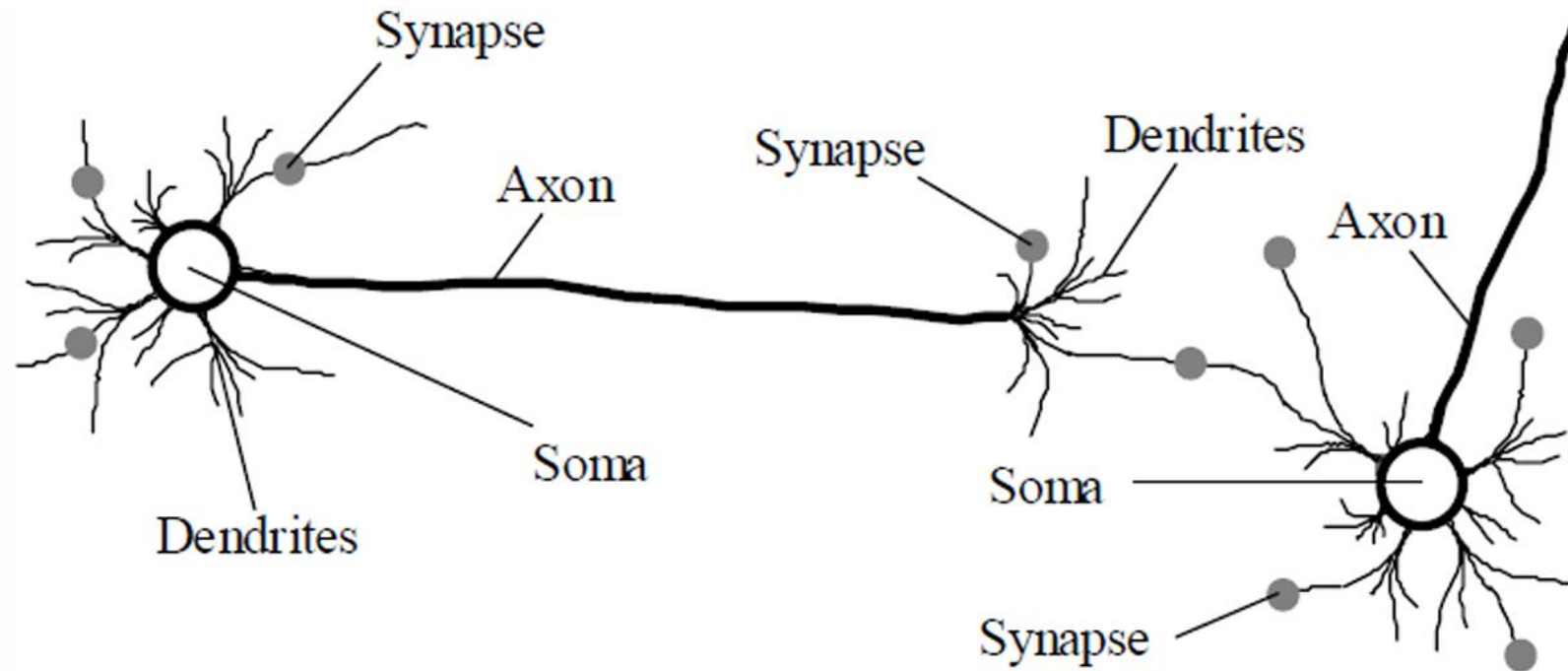
- A neural network is a system of hardware and/or software patterned after the operation of neurons in the human brain.
- A neural network usually involves a large number of processors operating in parallel and arranged in tiers.
- The first tier receives the raw input information analogous to optic nerves in human visual processing.
- Each successive tier receives the output from the tier preceding it, rather than from the raw input in the same way neurons further from the optic nerve receive signals from those closer to it.
- The last tier produces the output of the system.

Basic Components of Biological Neurons



- The majority of neurons encode their activation or outputs as a series of brief electrical pulses (i.e. spikes or action potentials).
- The neuron's cell body (soma) processes the incoming activations and converts them into output activations.
- The neuron's nucleus contains the genetic material (DNA).
- Dendrites are fibers which emanate from the cell body and provide the receptive zone that receive activation from other neurons.
- Axons are fibers acting as transmission lines that send action potentials to other neurons.
- The junctions that allow signal transmission between the axons and the dendrites are called synapses.
- The process of transmission is by diffusion of chemicals called neurotransmitters across the synaptic cleft.

Biological NN vs Artificial NN



Biological Neural Network	Artificial Neural Network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

Biological NN vs Artificial NN

- Size
 - Our brain contains about 86 billion neurons. The number of “neurons” in artificial networks is much less than that.
- Signal transport and processing
 - The human brain works asynchronously, and ANNs work synchronously.
- Speed
 - Single biological neurons are slow, while standard neurons in ANNs are fast.
 - Certain biological neurons can fire around 200 times a second on average. Signals travel at different speeds depending on the type of nerve impulse, ranging from 0.61 m/s up to 119 m/s and also vary from person to person depending on their sex, age, height, temperature, medical condition, lack of sleep etc.
 - Information in artificial neurons is carried over by the continuous, floating-point number values of synaptic weights. artificial neurons do not experience “fatigue”: they are functions that can be calculated as many times and as fast as the computer architecture would allow.

Biological NN vs Artificial NN

- Topology
 - Biological neural networks have complicated topologies, while ANNs are often in a tree structure.
- Fault-tolerance
 - biological neuron networks due to their topology are fault-tolerant. ANNs are not modelled for fault tolerance or self-regeneration, though recovery is possible by saving the current state (weight values) of the model and continuing the training from that saved state.
- Power consumption
 - The brain consumes about 20% of all the human body's energy: despite its large cut, an adult brain operates on about 20 watts (barely enough to dimly light a bulb) being extremely efficient.
 - For benchmark: a single Nvidia GeForce Titan X GPU runs on 250 watts alone and requires a power supply. Our machines are way less efficient than biological systems. Computers also generate a lot of heat.

Biological NN vs Artificial NN

- Learning

- We still do not understand how brains learn, or how redundant connections store and recall information. By learning, we are building on information that is already stored in the brain. Our knowledge is deepened by repetition and during sleep, and tasks that once required a focus can be executed automatically once mastered.
- Artificial neural networks, on the other hand, have a predefined model, where no further neurons or connections can be added or removed. Only the weights of the connections (and biases representing thresholds) can change during training. The networks start with random weight values and will slowly try to reach a point where further changes in the weights would no longer improve performance.
- Biological networks usually don't stop/start learning.
- ANNs have different fitting (train) and prediction (evaluate) phases.

- Field of application

- ANNs are specialized. They can perform one task. They might be perfect at playing chess, but they fail at playing go (or vice versa)
- Whereas biological neural networks can learn completely new tasks.

Units of Neural Network

- **Nodes (units)**

- Nodes represent a cell of neural network.

- **Links**

- Links are directed arrows that show propagation of information from one node to another node.

- **Activation**

- Activations are inputs to or outputs from a unit.

- **Weight**

- Each link has weight associated with it which determines strength and sign of the connection.

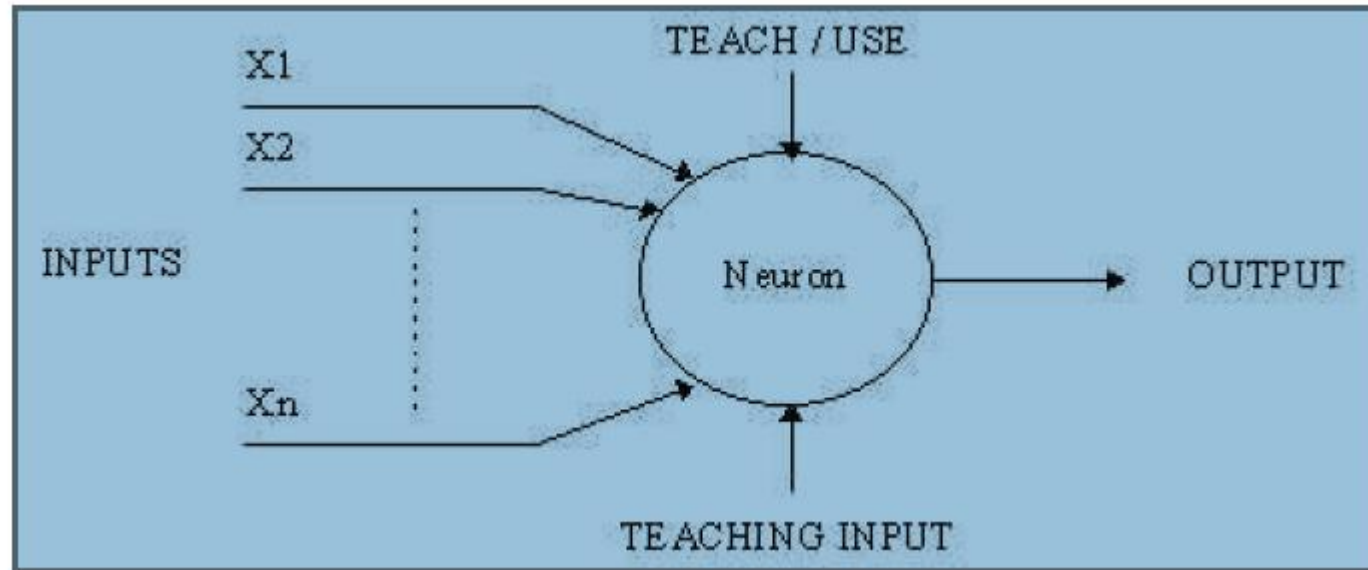
- **Activation function:**

- A function which is used to derive output activation from the input activations to a given node is called activation function.

- **Bias:**

- Bias is a feature of a statistical technique or of its results whereby the expected value of the results differs from the true underlying quantitative parameter being estimated.

A simple neuron



An artificial neuron is a device with many inputs and one output. The neuron has two modes of operation; the training mode and the using mode.

In the training mode, the neuron can be trained to fire (or not), for particular input patterns.

In the using mode, when a taught input pattern is detected at the input, its associated output becomes the current output.

Artificial Neuron

- An artificial neuron is designed to mimic the first-order characteristics of a biological neuron.
- A set of inputs, x_i .
- Each input is multiplied by the corresponding weight, w_i .
- All of the weighted inputs are summed to determine the activation level.
- An activation function is applied to determine the neuron output.



Mathematical Structure

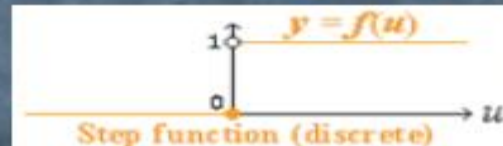
- A neuron is a simple sum and compare device.

$$u = x_1w_1 + x_2w_2 + \dots + x_nw_n - T$$

$$y = f(u)$$

threshold

One possible function for y is a step function:



If the weighted sum of the inputs is greater than the threshold, then the neuron “fires”

Right Brain & Left Brain in Human

- Human brain consists of two hemisphere: Left and Right.
- Generally it is believed that the left brain is "logical" and the right brain is "creative".
- However, modern neuroscience shows that both hemispheres work together continuously through millions of nerve fibers (the corpus callosum).
- While some functions show relative specialization (for example, language is often more left-lateralized and spatial processing more right-lateralized), nearly every complex task involves both sides of the brain.
- Both hemispheres contain approximately 86 billion neurons in total throughout the brain.
- Each neuron communicates with thousands of other neurons.

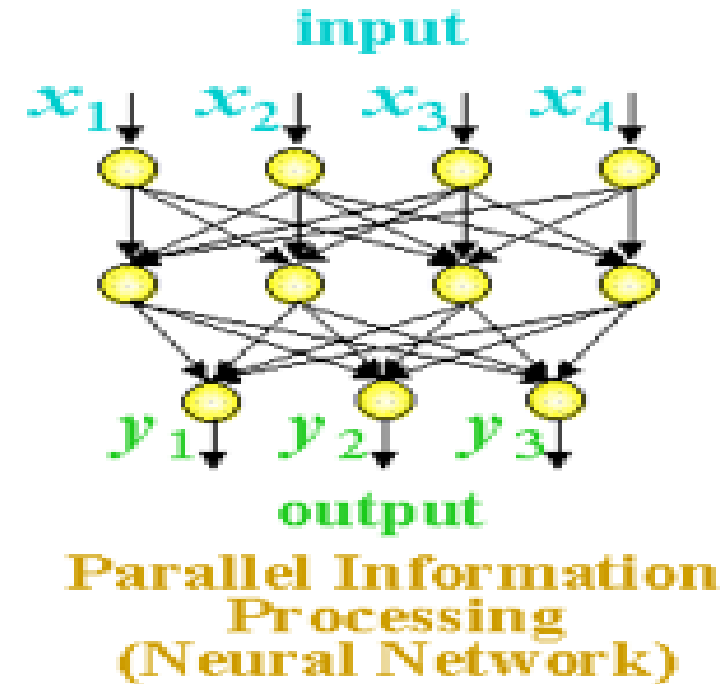
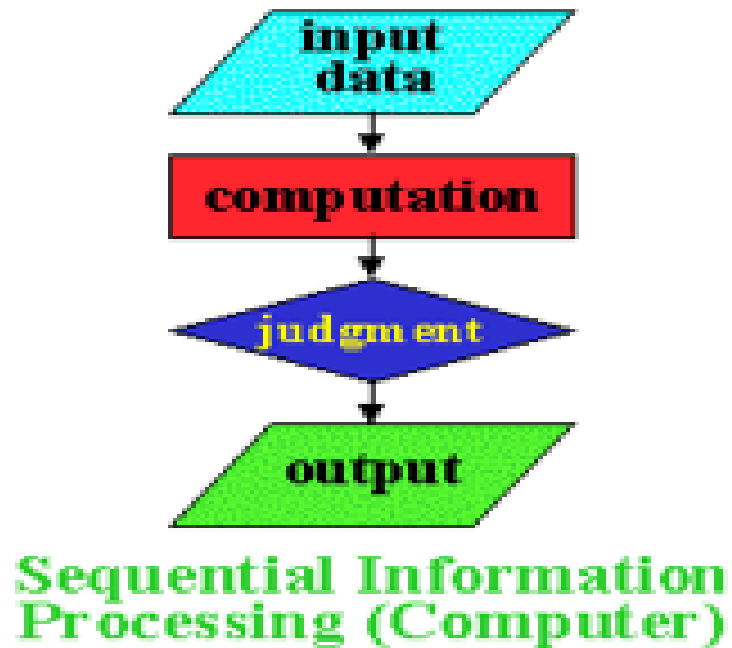
Right Brain & Left Brain in Human

- Example if a student in the class is asked to “add 3 and 4” then right brain recognizes word add while left brain performs addition operation. Hence, the answer is produced with the cooperation of both sides.
- Artificial Neural Network works in the similar fashion. Suppose we have an AI that identifies cats, then different groups of neurons specialize in different features.
- No single neuron says "I recognize cats." Instead, thousands of neurons cooperate.
- Our biological neuron receives signals, processes them, sends the result to other neurons. In similar way in ANN networks also a neuron in first layer receives the input, processes them and sends the result to neuron of other layers.

Features of Neural Network

1. Massive Parallel Processing

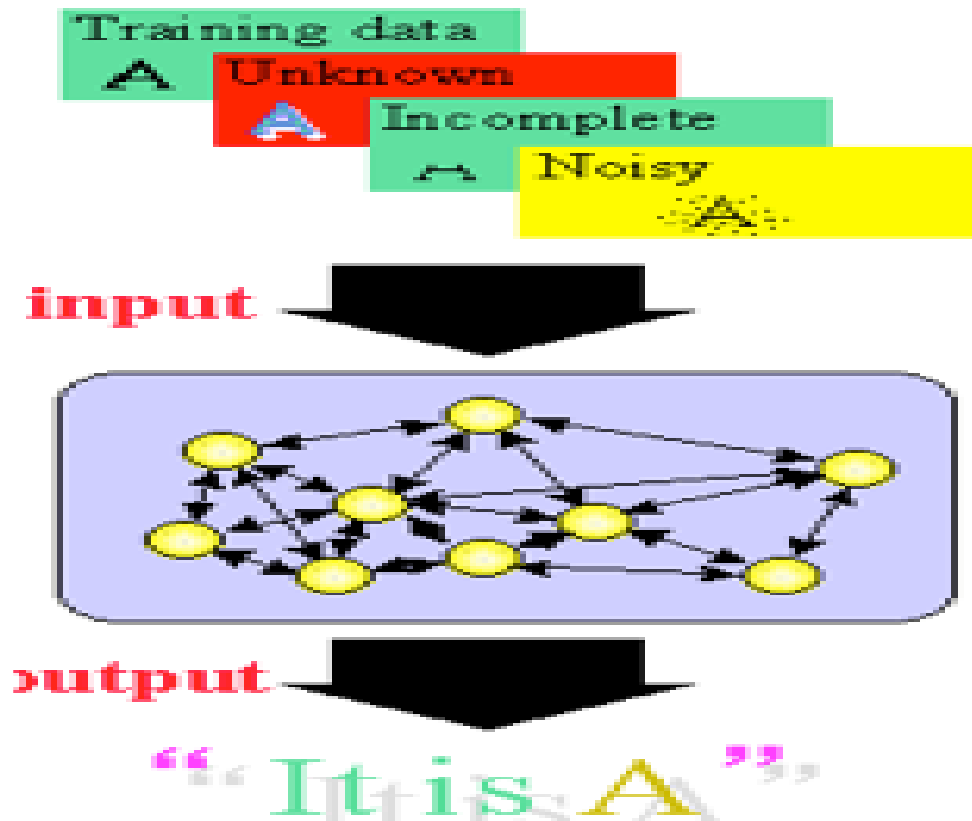
- Computers calculate in sequence whereas Neural Nets update in parallel.



Features of Neural Network

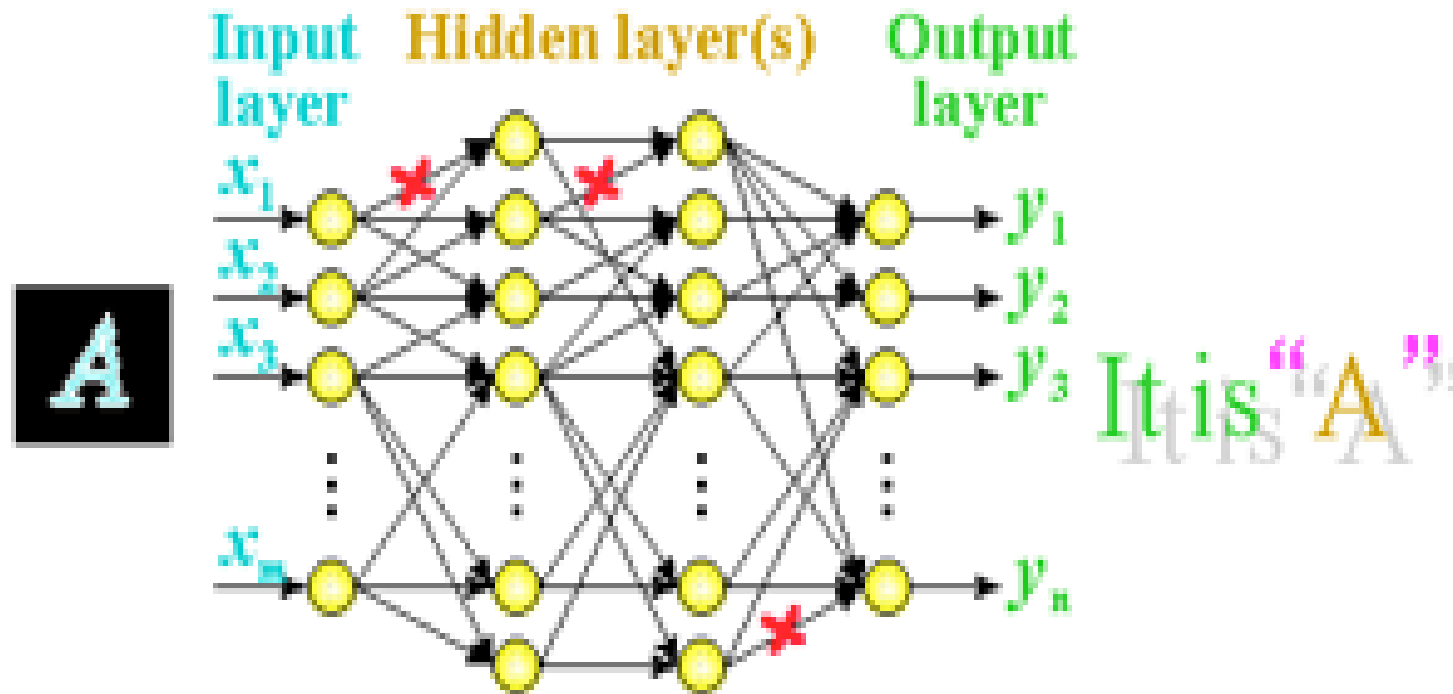
2. Neural nets can learn.

- Once a neural net is trained, it can handle unknown, incomplete or noisy data.



Features of Neural Network

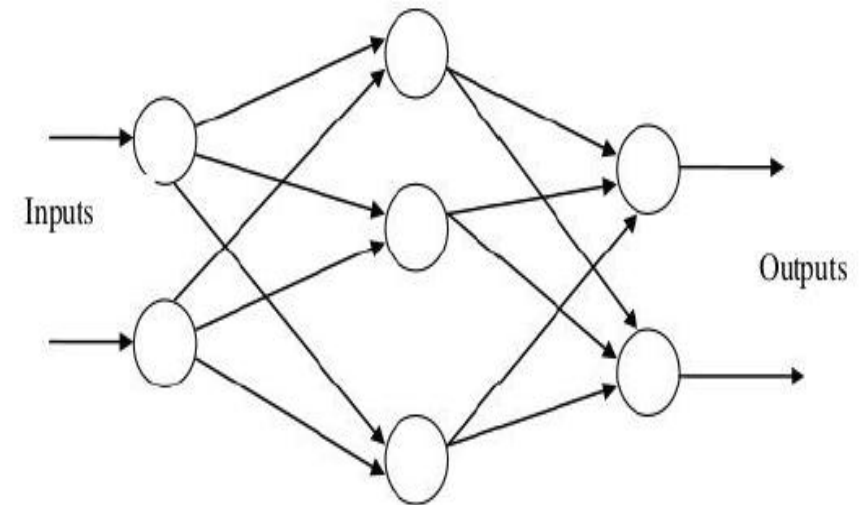
3. Unlike computer memory, information is distributed over the entire network, so a failure in one part of the network does not prevent the system for producing the correct output.



Types of Neural Network

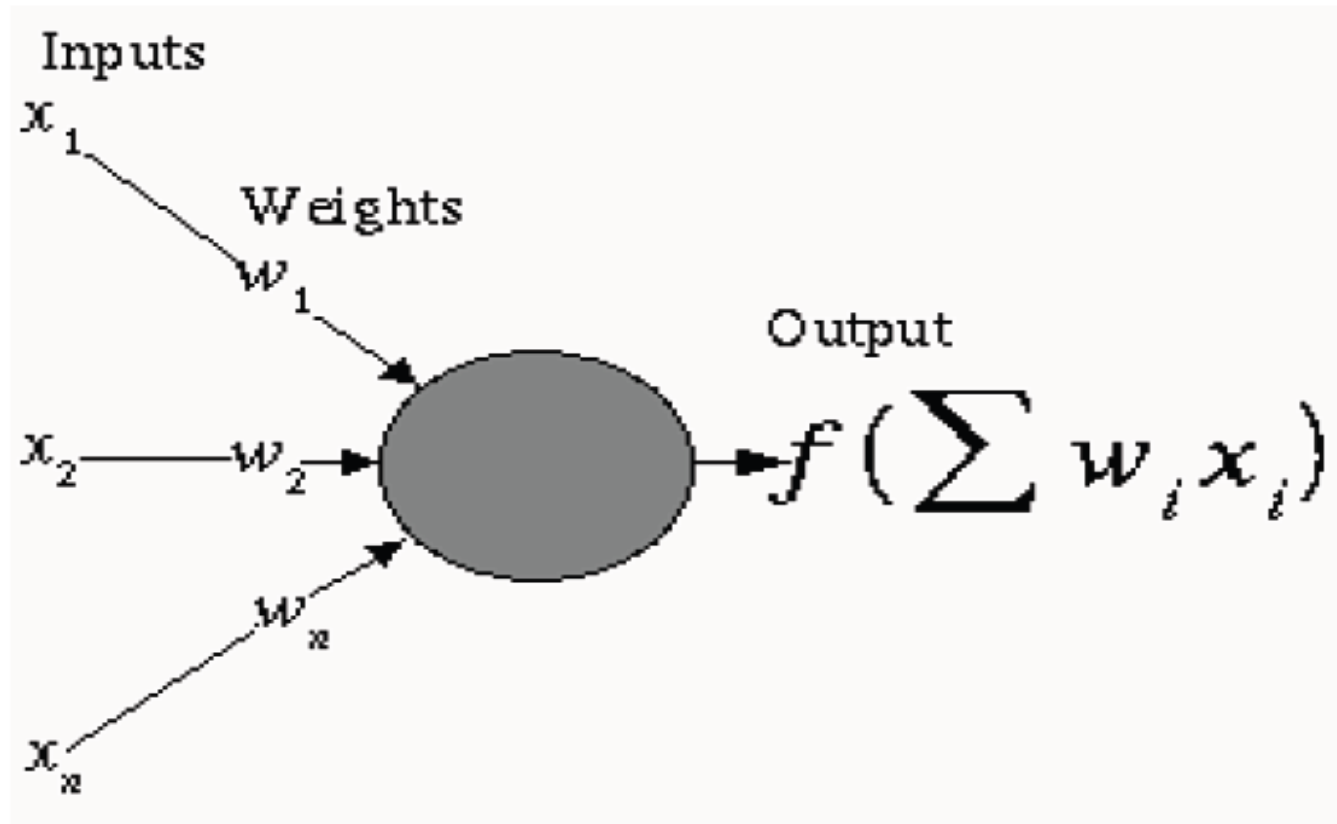
1. Feed-forward ANN

- Feed-forward ANNs allow signals to travel one way only; from input to output.
- There is no feedback (loops) i.e. the output of any layer does not affect that same layer.
- Feed forward ANNs tend to be straight forward networks that associate inputs with outputs.
- They are extensively used in pattern recognition.
- This type of organization is also referred to as bottom-up or top-down.



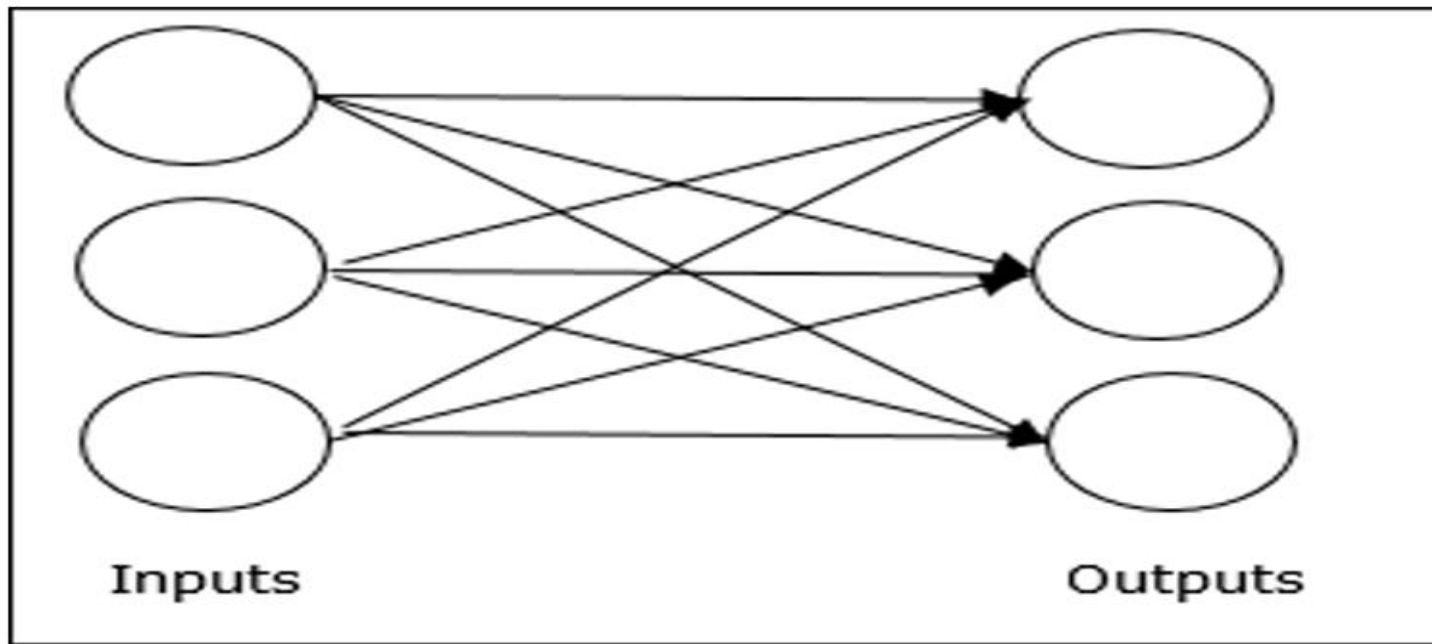
Types of Neural Network

1. Feed-forward ANN



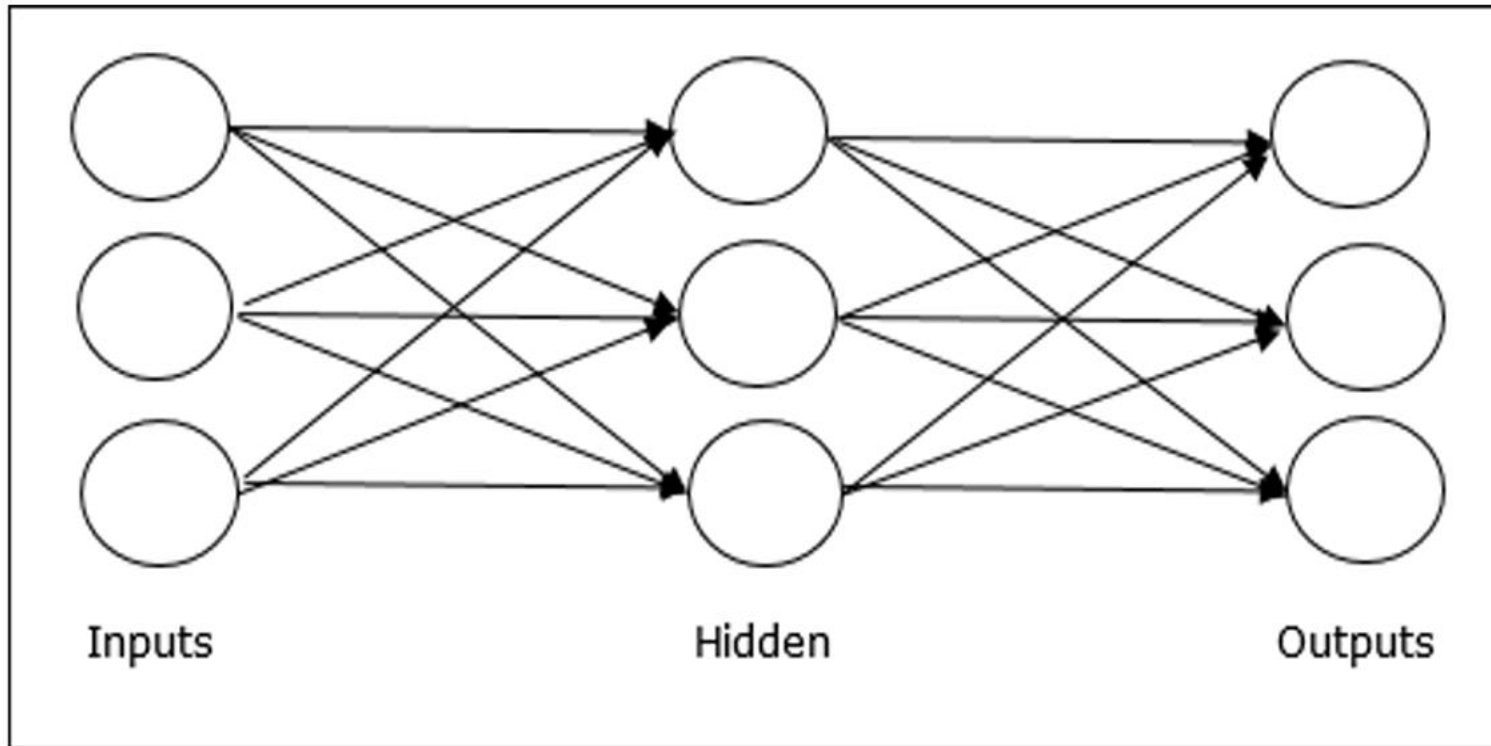
Types of Neural Network

a. Single Layer Feed Forward ANN



Types of Neural Network

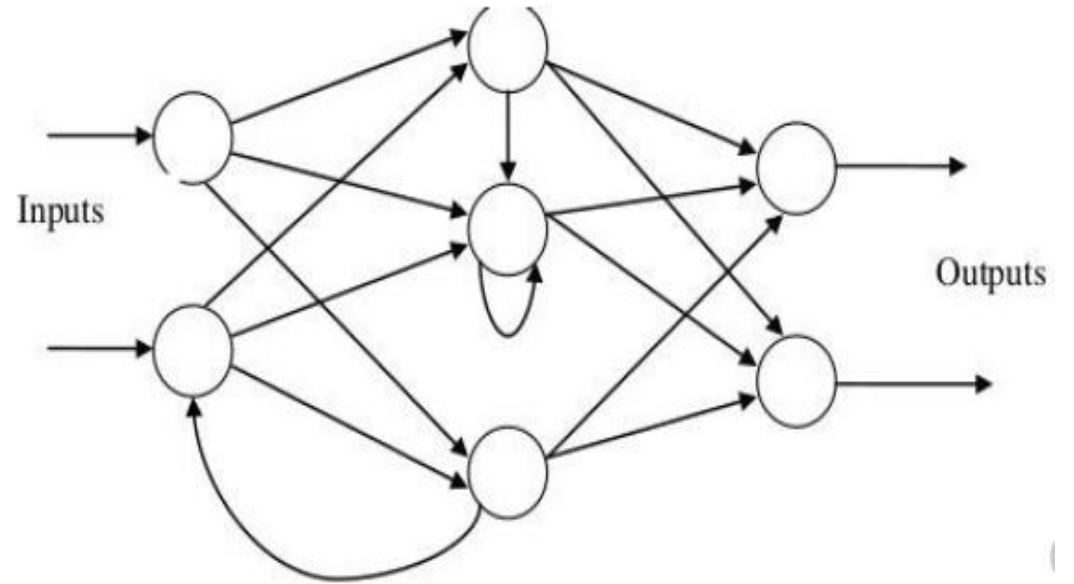
b. Multi Layer Feed Forward ANN



Types of Neural Network

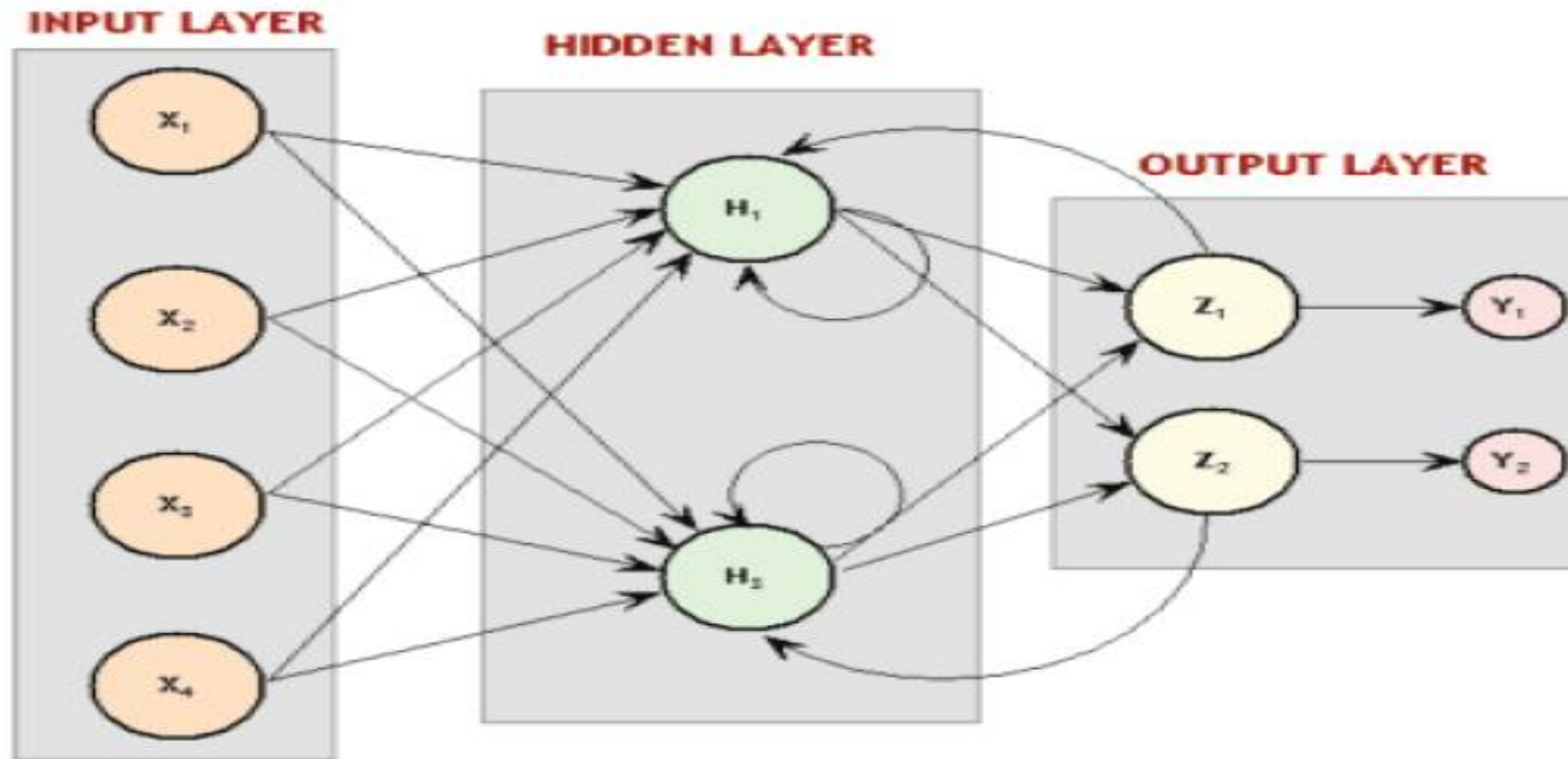
2. Feedback (Recurrent) ANN

- Feedback networks can have signals traveling in both directions by introducing loops in the network.
- Feedback networks are very powerful and can get extremely complicated.



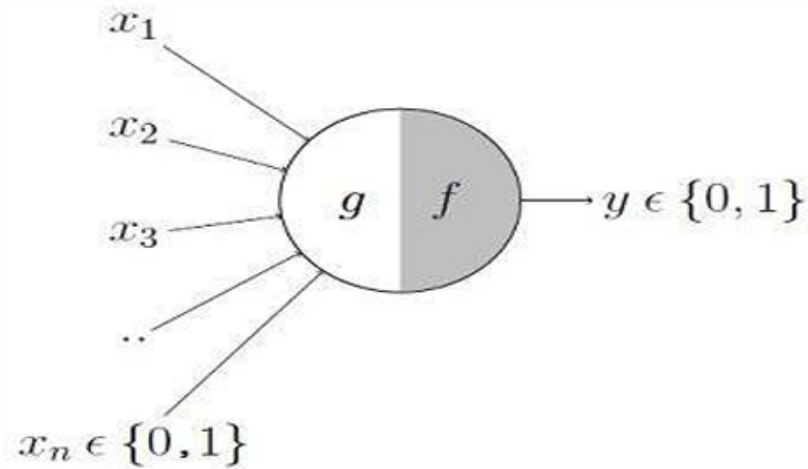
Types of Neural Network

2. Feedback (Recurrent) ANN



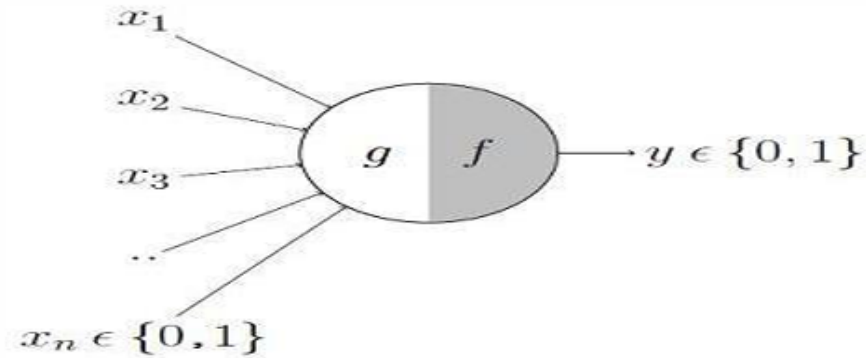
McCulloch/Pitts Neuron

- This is the first computational model of a neuron that was proposed by Warren McCulloch (neuroscientist) and Walter Pitts (logician) in 1943.



- It may be divided into 2 parts. The first part, g takes input and performs an aggregation.
- Based on the aggregated value, the second part, f makes a decision.
- Let us suppose that I want to predict my own decision, whether to watch a random football game or not on TV. The inputs are all Boolean i.e., $\{0, 1\}$ and my output variable is also Boolean $\{0: Will\ watch\ it, 1: Won't\ watch\ it\}$.
- Each input is weighted with weights in the range: -1 to + 1

McCulloch/Pitts Neuron



So

- x_1 could be *isPremierLeagueOn* (I like Premier League more)
- x_2 could be *isItAFriendlyGame* (I tend to care less about the friendlies)
- x_3 could be *isNotHome* (Can't watch it when I'm not home. Can I?)
- x_4 could be *isManUnitedPlaying* (I am a big Man United fan.) and so on.

These inputs can either be *excitatory* or *inhibitory*.

- *Inhibitory* inputs are those that have maximum effect on the decision-making irrespective of other inputs i.e., if x_3 is 1 (not home) then my output will always be 0 i.e., the neuron will never fire, so x_3 is an inhibitory input.
- *Excitatory inputs* are NOT the ones that will make the neuron fire on their own but they might fire it when combined.

McCulloch/Pitts Neuron

Formally, this is what is going on:

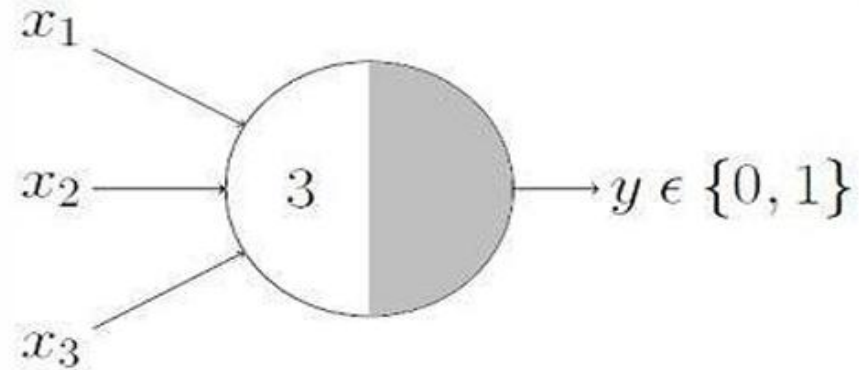
$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$\begin{aligned} y = f(g(\mathbf{x})) &= 1 && \text{if } g(\mathbf{x}) \geq \theta \\ &= 0 && \text{if } g(\mathbf{x}) < \theta \end{aligned}$$

- We can see that $g(\mathbf{x})$ is just doing a sum of the inputs — a simple aggregation. And θ here is called the thresholding parameter.
- For example, if I always watch the game when the sum turns out to be 2 or more, the θ is 2 here. This is called the Thresholding Logic.

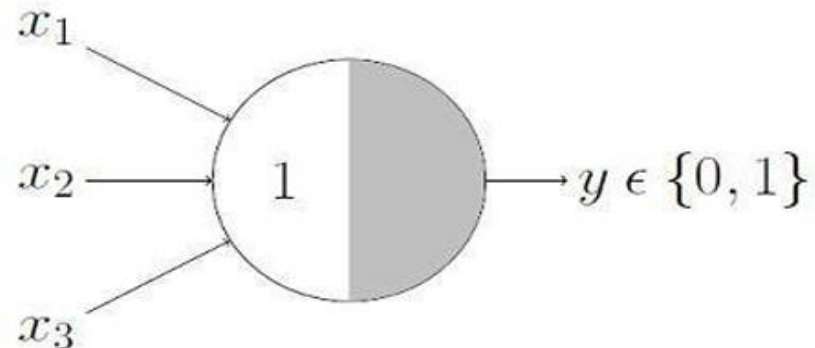
Boolean Functions Using McCulloch/Pitts Neuron

- **AND Function**



An AND function neuron would only fire when ALL the inputs are ON i.e., $g(x) \geq 3$ here.

- **OR Function**



For an OR function neuron would fire if ANY of the inputs is ON i.e., $g(x) \geq 1$ here.

McCulloch/Pitts Neuron

Example:

Construct an MCP (McCulloch/Pitts) neuron that will implement the function of the AND gate.

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{Y}$
0	0	0
0	1	0
1	0	0
1	1	1

Activation function is defined as:

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

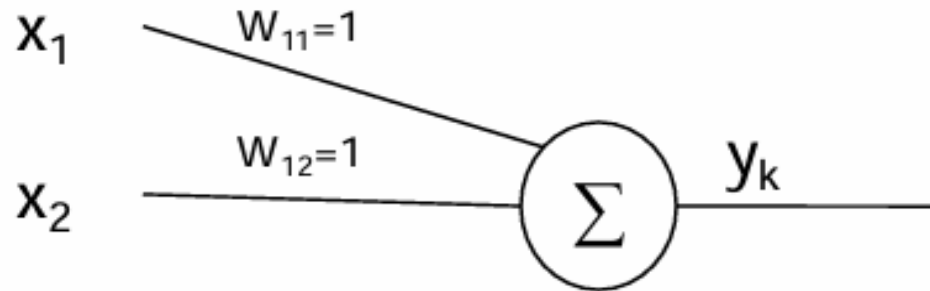
The value of theta should satisfy the following conditions:

$$\theta \geq \mathbf{n} * \mathbf{w} - \mathbf{p}$$

n= number of neurons in input layer, **w**= positive weight and **p**= negative weight

McCulloch/Pitts Neuron

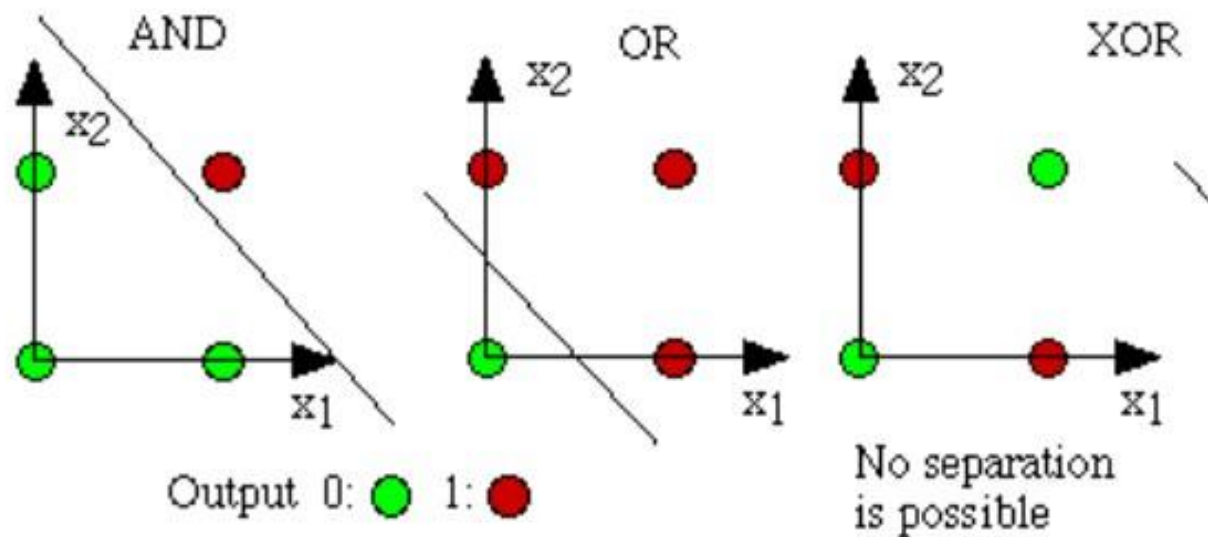
Let's assume the weight is $w_1=1$ and $w_2=2$



- Calculate threshold as $\Theta = n * w - p = 2 * 1 * 1 - 0 = 2$
- Calculate output value (y) as
 - $y(0,0) = x_1 * w_1 + x_2 * w_2 = 0 * 1 + 0 * 1 = 0 < \Theta$ ✓
 - $y(1,0) = x_1 * w_1 + x_2 * w_2 = 1 * 1 + 0 * 1 = 1 < \Theta$ ✓
 - $y(0,1) = x_1 * w_1 + x_2 * w_2 = 0 * 1 + 1 * 1 = 1 < \Theta$ ✓
 - $y(1,1) = x_1 * w_1 + x_2 * w_2 = 1 * 1 + 1 * 1 = 2 \geq \Theta$ ✓

McCulloch/Pitts Neuron

- Example 2: Construct a McCulloch/Pitts Neuron to implement XOR gate.

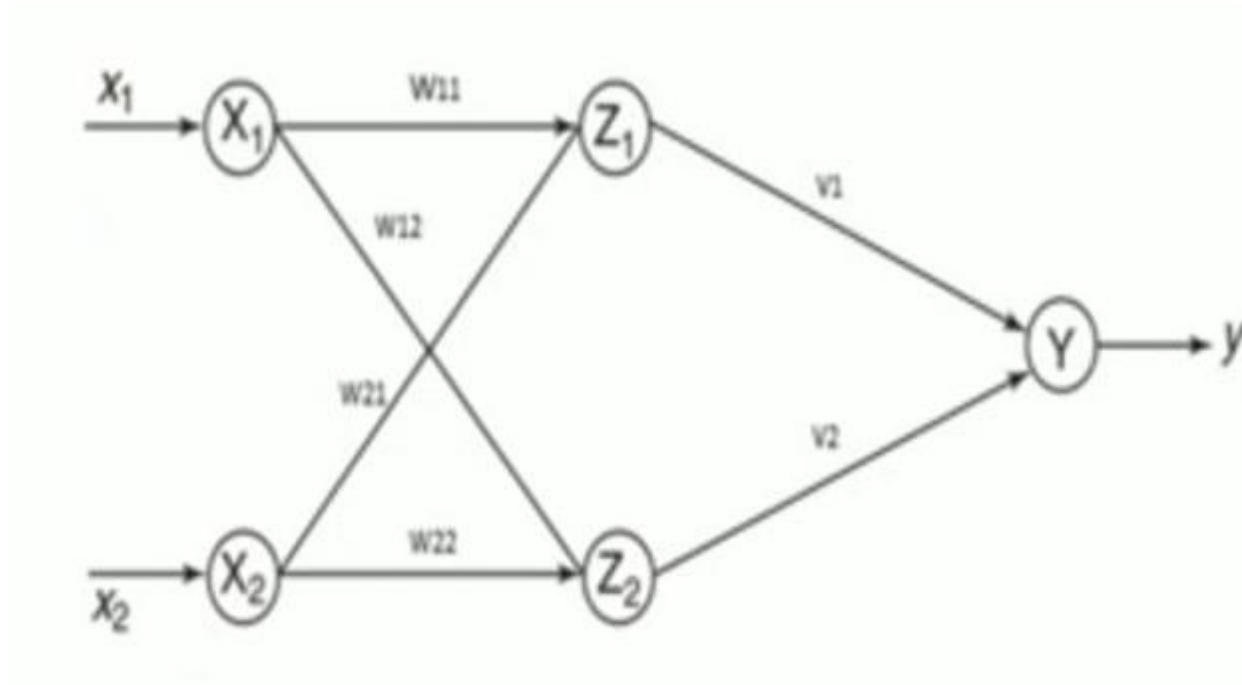


x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Since XOR gate is not a linearly separable problem a single layer neuron structure cannot implement it.

McCulloch/Pitts Neuron

- We need to add a hidden layer to solve the XOR gate as:



$$y = x_1 \overline{x_2} + \overline{x_1} x_2$$

$$y = z_1 + z_2$$

where

$$z_1 = x_1 \overline{x_2} \quad \text{(function 1)}$$

$$z_2 = \overline{x_1} x_2 \quad \text{(function 2)}$$

$$y = z_1 \text{ (OR) } z_2 \quad \text{(function 3)}$$

McCulloch/Pitts Neuron

Soln

Let solve the first function as:

$$Z_1 = x_1 \overline{x_2}$$

x_1	x_2	z_1
0	0	0
0	1	0
1	0	1
1	1	0

The truth table for function z_1 is:

Let us assume the initial weights as:
 $w_{11} = w_{21} = 1$

$$\underline{w_{11} = w_{21} = 1}$$

Now,

Now, ② becomes $\Rightarrow 1 \times 1 - 1 = 2 \times 1 \times 1 - 0 = 2$

Calculate the net inputs as:

$(0,0) = 0 \times 1 + 1 \times 0 = 0 < 2$, doesn't fix

$$(0, 1) = 0 \times 1 + 1 \times 1 = 1 < 2, \quad " \quad "$$

$$(1, 0) = 1 \times 1 + 0 \times 1 = 1 \leq 2$$

$(4, 1) = 1 \times 1 + 1 \times 1 = 2 = 2$ neuron fires

$\therefore$ It is not possible to obtain function z_1 using these weights.

Now, let us assume weights as:

$$w_{11} = 1, w_{21} = -1$$

$$\therefore Q \text{ becomes} = n \times w - p = 2 \times 1 - 1 = 1$$

Calculate the net inputs as:

$(0, 0) = 0 \times 1 + 0 \times -1 = 0$, doesn't fire

$$(0, 1) = 0 \times 1 + 1 \times -1 = -1, \quad "$$

$$(1, 0) = 1 \times 1 + 0 \times -1 = 1, \text{ neuron fires}$$

$(1, 1) = 1 \times 1 + 1 \times -1 = 0$, doesn't fit

$$\therefore \omega_{11} = 1 \text{ \& } \omega_{21} = -1$$

Now, let us solve 2nd function as:

$$Z_2 = \overline{x_1} x_2$$

∴ The truth table for function Z_2 is:

x_1	x_2	z_2
0	0	0
0	1	1
1	0	0
1	1	0

Let the weights be initialized as:

$$\underline{w_{12} = w_{22} = 1}$$

$\therefore \textcircled{1}$ becomes :- $1 \times W - P = 2 \times 1 \times 1 - 0 = 2$

McCulloch/Pitts Neuron

Calculate net i/ps as:

$$(0,0) = 0 \times 1 + 0 \times 1 = 0, \text{ doesn't fire}$$

$$(0,1) = 0 \times 1 + 1 \times 1 = 1, \text{ " "}$$

$$(1,0) = 1 \times 1 + 0 \times 1 = 1, \text{ " "}$$

$$(1,1) = 1 \times 1 + 1 \times 1 = 2, \text{ fires}$$

$\therefore$ Not possible to obtain function Z_2 with these weights.

Again, let us assume the weights as:
 $W_{12} = -1$ & $W_{22} = 1$

$$\therefore \Theta \text{ becomes: } 2 \times 1 - 1 = 1$$

Calculate net i/ps as:

$$(0,0) = 0 \times -1 + 0 \times 1 = 0, \text{ doesn't fire}$$

$$(0,1) = 0 \times -1 + 1 \times 1 = 1, \text{ fires}$$

$$(1,0) = 1 \times -1 + 0 \times 1 = -1, \text{ doesn't fire}$$

$$(1,1) = 1 \times -1 + 1 \times 1 = 0, \text{ " "}$$

$$\therefore W_{12} = -1 \text{ & } W_{21} = 1$$

Now, implement 3rd function:

$$y = Z_1 \text{ (OR) } Z_2$$

$\therefore$ The truth table becomes:

x_1	x_2	y	Z_1	Z_2
0	0	0	0	0
0	1	1	0	1
1	0	1	1	0
1	1	0	0	0

Now,

$$y_{in} = Z_1 V_1 + Z_2 V_2$$

Let initial weights be: $V_1 = V_2 = 1$

$$\therefore \Theta \text{ becomes: } -1 \times 1 - 0 = -1$$

Calculate the net i/ps as:

$$(0,0) = 0 \times 1 + 0 \times 1 = 0 < 1, \text{ doesn't fire}$$

$$(0,1) = 0 \times 1 + 1 \times 1 = 1 = 1, \text{ fires}$$

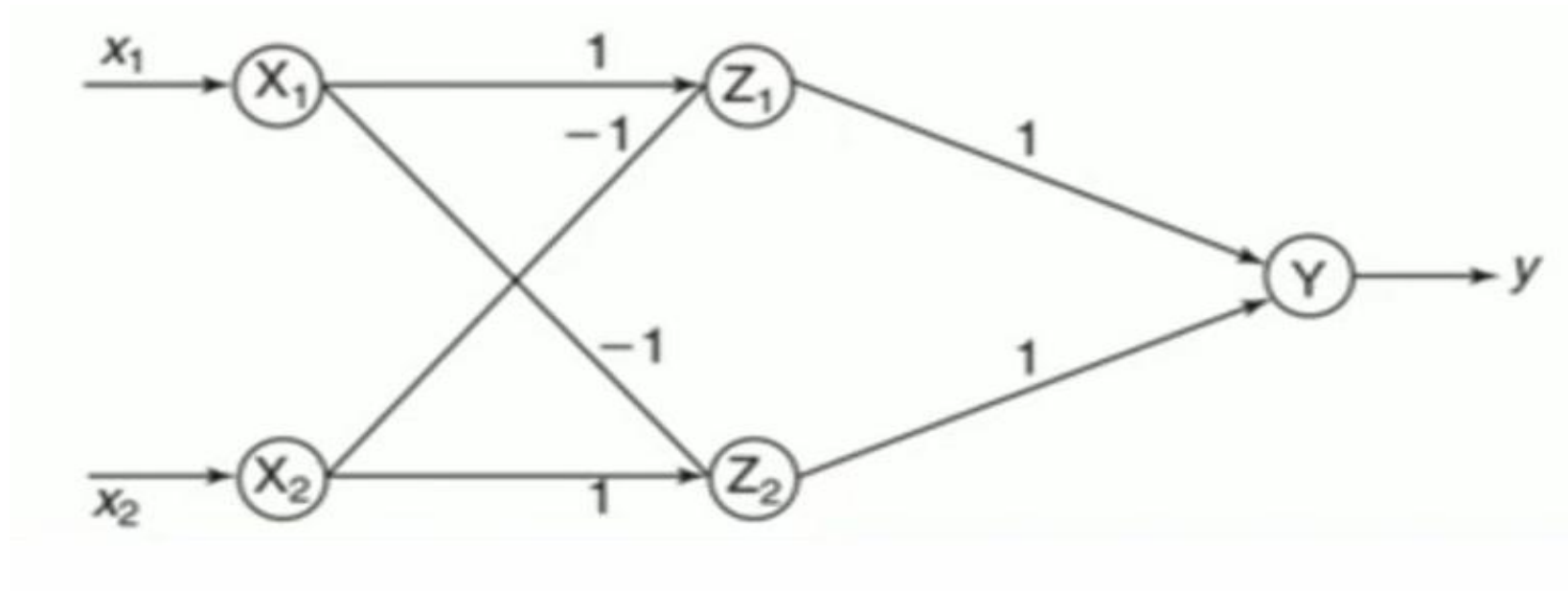
$$(1,0) = 1 \times 1 + 0 \times 1 = 1 = 1, \text{ "}$$

$$(1,1) = 0 \times 1 + 0 \times 1 = 0 < 1, \text{ doesn't fire}$$

$$\therefore V_1 = V_2 = 1$$

McCulloch/Pitts Neuron

- Therefore, the final network becomes:



Learning in Neural Network

Learning

- One of the powerful features of neural networks is learning.
- Learning in neural networks is carried out by adjusting the connection weights among neurons.
- Learning may be classified into 2 categories:
 - (1) Supervised Learning
 - (2) Unsupervised Learning

Learning in Neural Network

Supervised Learning

- In supervised learning, the network is presented with inputs together with the target (teacher signal) outputs.
- Then, the neural network tries to produce an output as close as possible to the target signal by adjusting the values of internal weights.

Unsupervised Learning

- In unsupervised learning, there is no teacher (target signal) from outside and the network adjusts its weights in response to only the input patterns.

Hebbian Learning

- The operation of a neural network is determined by the values of the interconnection weights.
- There is no algorithm that determines how the weights should be assigned in order to solve a specific problems.
- Hence, the weights are determined by a learning process.

Hebbian Learning

- The oldest and most famous of all learning rules is Hebb's postulate of learning.

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic changes take place in one or both cells such that A's efficiency as one of the cell's firing B is increased”

- Application to artificial neurons:

If two interconnected neurons are both “on” at the same time, then the weight between them should be increased.

Hebb's Algorithm

Hebb's Algorithm

- Initialize all weights and bias to 0

- Adjust the weights as follows:

$$w_i(\text{new}) = w_i(\text{old}) + \Delta w_i$$

$$\text{where } \Delta w_i = x_i y$$

- Adjust the bias as follows:

$$b(\text{new}) = b(\text{old}) + \Delta b$$

$$\text{where } \Delta b = y$$

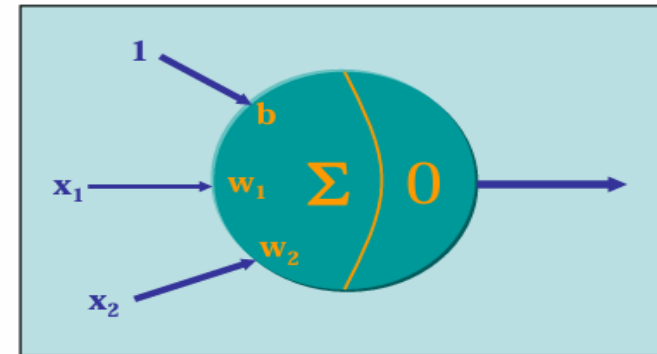
- The activation function used here is bipolar sigmoidal function, so the output range is $[-1, 1]$. That is:

$$y(x) = \begin{cases} -1, & g(h_i) < 0 \\ +1, & g(h_i) \geq 0 \end{cases}$$

Hebbian Learning: Example 1

- **PROBLEM:**
Construct a Hebb Net which performs like an AND function, that is, only when both features are “active” will the data be in the target class.
- **TRAINING SET** (with the bias input always at 1):

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1



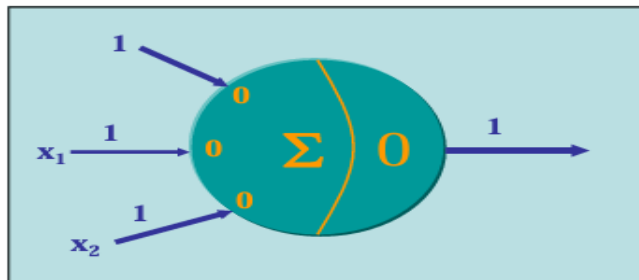
Hebbian Learning: Example 1

Training – First Input

- Initialize the weights to 0

Present the first input (1 1 1) with a target of 1

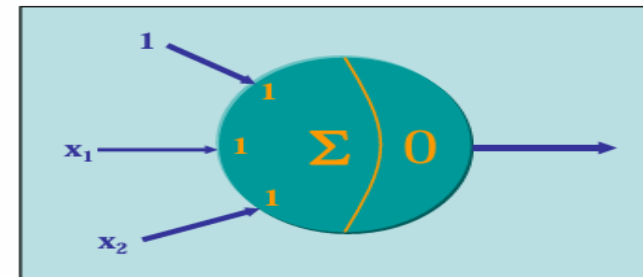
Update the weights:



$$\begin{aligned}w_1(\text{new}) &= w_1(\text{old}) + x_1 t \\ &= 0 + 1 = 1\end{aligned}$$

$$\begin{aligned}w_2(\text{new}) &= w_2(\text{old}) + x_2 t \\ &= 0 + 1 = 1\end{aligned}$$

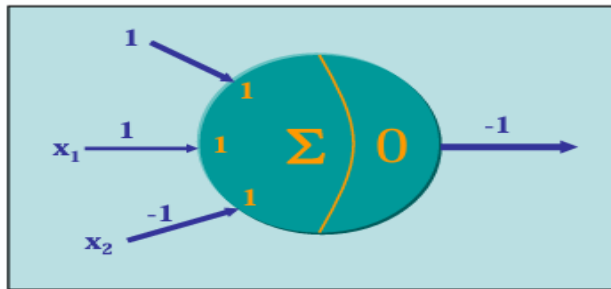
$$\begin{aligned}b(\text{new}) &= b(\text{old}) + t \\ &= 0 + 1 = 1\end{aligned}$$



Hebbian Learning: Example 1

Training – Second Input

- Present the second input: (1 -1 1) with a target of -1

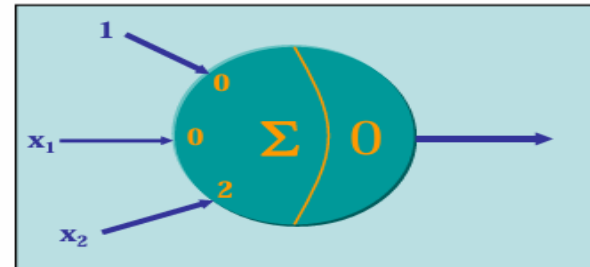


Update the weights:

$$\begin{aligned}w_1(\text{new}) &= w_1(\text{old}) + x_1 t \\ &= 1 + 1(-1) = 0\end{aligned}$$

$$\begin{aligned}w_2(\text{new}) &= w_2(\text{old}) + x_2 t \\ &= 1 + (-1)(-1) = 2\end{aligned}$$

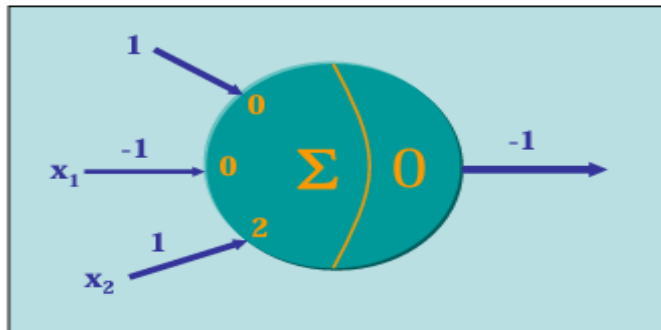
$$\begin{aligned}b(\text{new}) &= b(\text{old}) + t \\ &= 1 + (-1) = 0\end{aligned}$$



Hebbian Learning: Example 1

Training – Third Input

- Present the third input: $(-1 \ 1 \ 1)$ with a target of -1

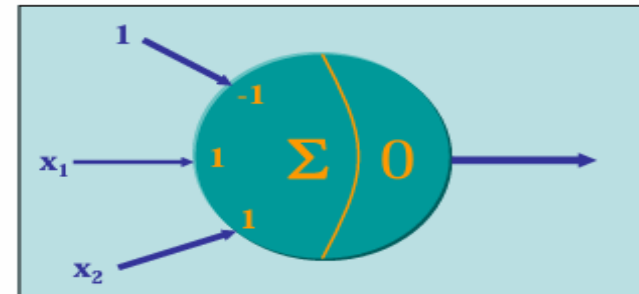


Update the weights:

$$\begin{aligned} w_1(\text{new}) &= w_1(\text{old}) + x_1 t \\ &= 0 + (-1)(-1) = 1 \end{aligned}$$

$$\begin{aligned} w_2(\text{new}) &= w_2(\text{old}) + x_2 t \\ &= 2 + 1(-1) = 1 \end{aligned}$$

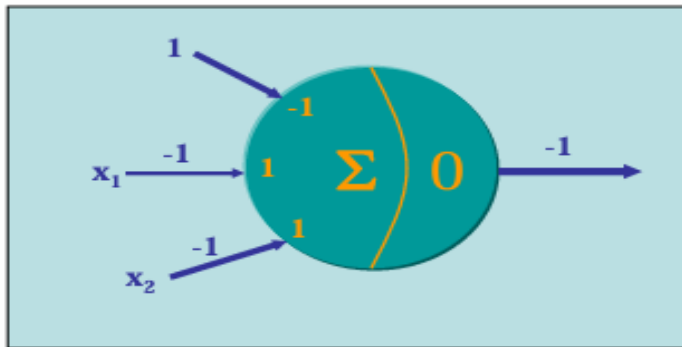
$$\begin{aligned} b(\text{new}) &= b(\text{old}) + t \\ &= 0 + (-1) = -1 \end{aligned}$$



Hebbian Learning: Example 1

Training – Fourth Input

- Present the fourth input: $(-1 \ -1 \ 1)$ with a target of -1

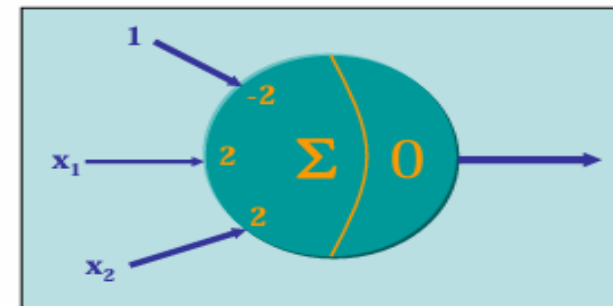


Update the weights:

$$\begin{aligned} w_1(\text{new}) &= w_1(\text{old}) + x_1 t \\ &= 1 + (-1)(-1) = 2 \end{aligned}$$

$$\begin{aligned} w_2(\text{new}) &= w_2(\text{old}) + x_2 t \\ &= 1 + (-1)(-1) = 1 \end{aligned}$$

$$\begin{aligned} b(\text{new}) &= b(\text{old}) + t \\ &= -1 + (-1) = -2 \end{aligned}$$

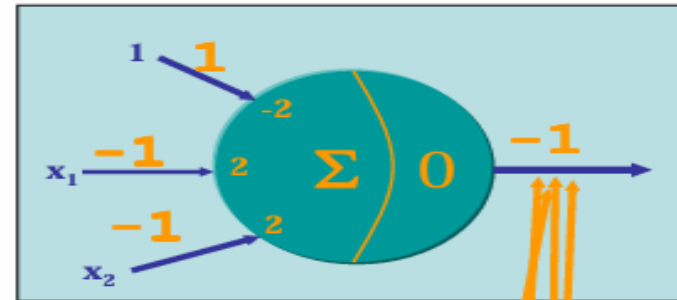


Hebbian Learning: Example 1

Final Neuron

- This neuron works:

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1

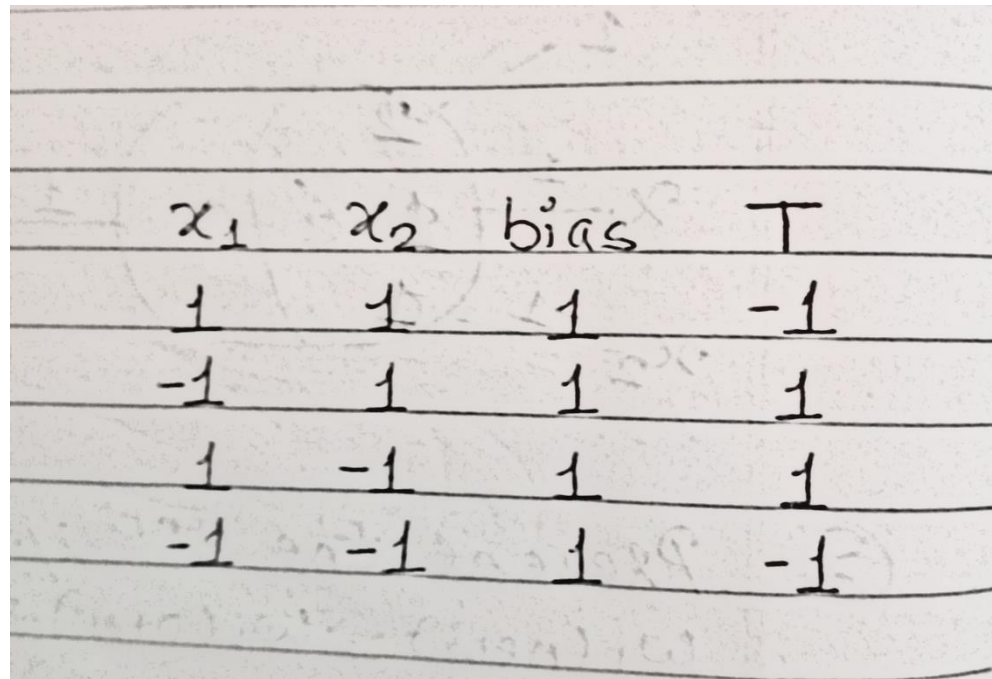


$$\begin{aligned}1 * 2 + 1 * 2 + 1 * (-2) &= 2 > 0 \\(-1) * 2 + 1 * 2 + 1 * (-2) &= -2 < 0 \\1 * 2 + (-1) * 2 + 1 * (-2) &= -2 < 0 \\(-1) * 2 + (-1) * 2 + 1 * (-2) &= -6 < 0\end{aligned}$$

Hebbian Learning: Example 2

- Show that a simple neural network cannot implement X-OR gate.

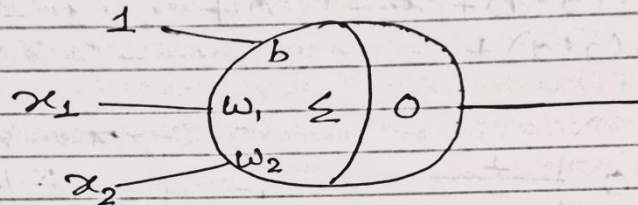
Solution,



x_1	x_2	bias	T
1	1	1	-1
-1	1	1	1
1	-1	1	1
-1	-1	1	-1

Hebbian Learning: Example 2

→ Soln



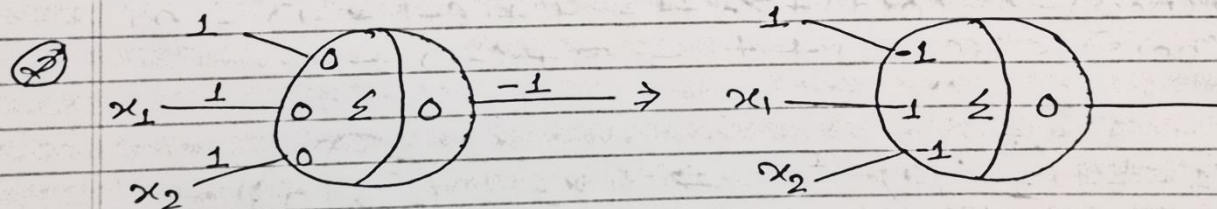
Initialize the weight to 0.

- ① Now present the 1st i/p (1, 1, 1) with a target of -1.
Now update the weight.

$$w_1(\text{new}) = w_1(\text{old}) + x_1 t = 0 + 1 \times -1 = -1$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 t = 0 + 1 \times -1 = -1$$

$$b(\text{new}) = b(\text{old}) + t = 0 + (-1) = -1$$



- ② Present the 2nd i/p (-1, 1, 1) with a target of 1.

$$w_1(\text{new}) = w_1(\text{old}) + x_1 t = -1 + (-1 \times 1) = -2$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 t = -1 + (1 \times 1) = 0$$

$$b(\text{new}) = b(\text{old}) + t = -1 + 1 = 0$$

Hebbian Learning: Example 2

Diagram 1: Initial state of a neuron with two inputs, x_1 and x_2 , and a bias input. The weights are $w_1 = -1$, $w_2 = 1$, and the bias is $b = 0$. The target is 1.

Diagram 2: After the first update, the weights are $w_1 = 0$, $w_2 = -2$, and the bias is $b = 0$.

③ Present the 3rd i/p (1, -1, 1) with target of 1.
 $w_1(\text{new}) = w_1(\text{old}) + x_1 t = -2 + 1 \times 1 = -1$
 $w_2(\text{new}) = w_2(\text{old}) + x_2 t = 0 + (-1 \times 1) = -1$
 $b(\text{new}) = b(\text{old}) + t = 0 + 1 = 1$

Diagram 3: After the second update, the weights are $w_1 = -1$, $w_2 = -1$, and the bias is $b = 1$.

④ Present the 4th i/p (-1, -1, 1) with target -1.
 $w_1(\text{new}) = w_1(\text{old}) + x_1 t = -1 + (-1 \times -1) = 0$
 $w_2(\text{new}) = w_2(\text{old}) + x_2 t = -1 + (-1 \times -1) = 0$
 $b(\text{new}) = b(\text{old}) + t = 1 - 1 = 0$

Diagram 4: After the third update, the weights are $w_1 = 0$, $w_2 = 0$, and the bias is $b = 0$.

Now, $1 \times 0 + 1 \times 0 + 1 \times 0 = 0$
 $-1 \times 0 + 1 \times 0 + 1 \times 0 = 0$
 $1 \times 0 + (-1 \times 0) + 1 \times 0 = 0$
 $-1 \times 0 + (-1 \times 0) + 1 \times 0 = 0$
 Here, neuron doesn't fire.

Is Hebbian Learning Supervised or Unsupervised?

- Hebbian Learning is an unsupervised learning algorithm.
- In Hebbian learning:
 - The target output is only used to initialize activations.
 - The learning rule does not compare predictions with the correct answer (no error-based correction).
 - The weight updates rely on simple co-activation, not minimizing a loss function.
 - This makes it different from true supervised learning, which explicitly corrects mistakes.

Key Difference from Supervised Learning

Hebbian Learning	Supervised Learning
Updates weights based on input-output co-activation.	Updates weights by minimizing an error function.
No explicit comparison to the target value.	Compares predicted output to actual target.
No backpropagation or error correction.	Uses error correction mechanisms (like gradient descent).
Works in self-organizing, correlation-based learning.	Works in labeled datasets with feedback.

Simple Computations in Neural network

- There are 2 types of components: Linear and Non-linear.
- Linear: Input function
 - calculate weighted sum of all inputs.

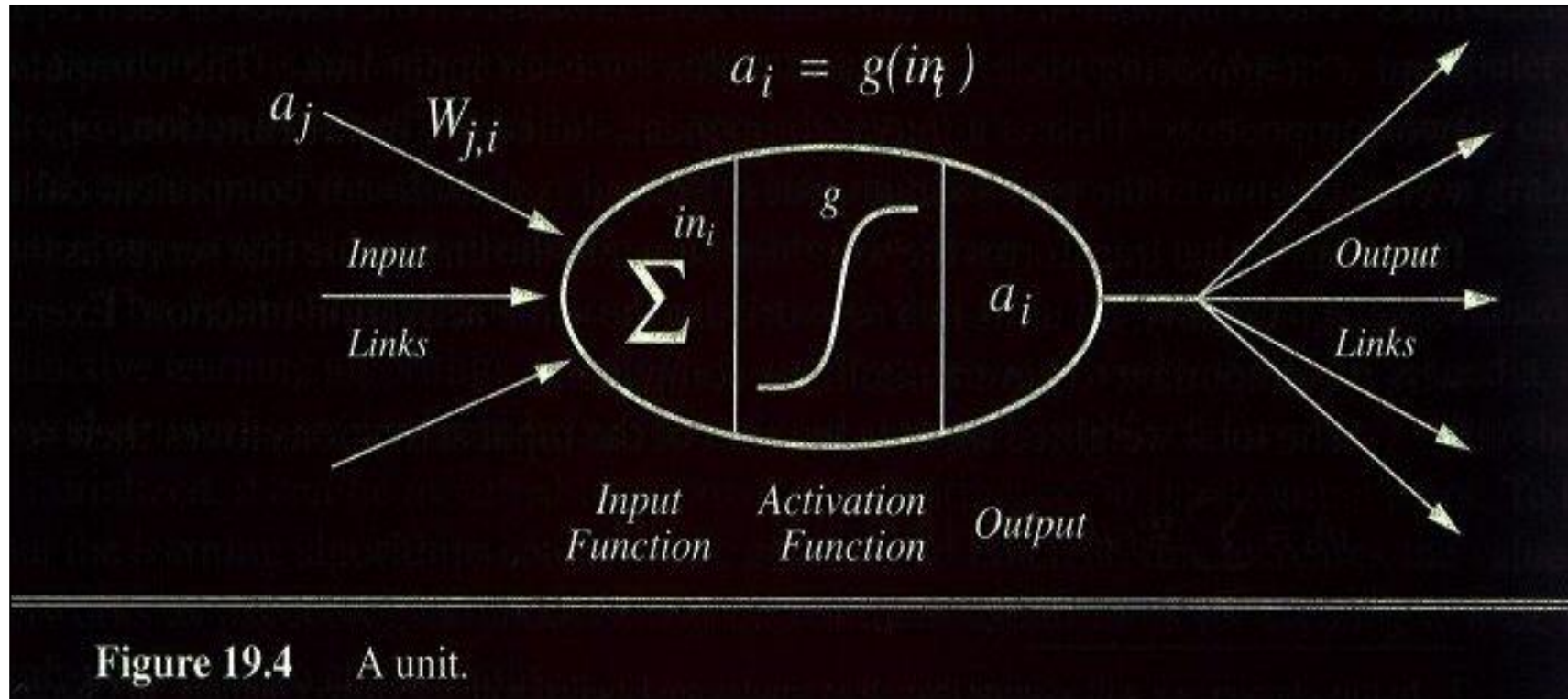
$$in_i = \sum_j W_{j,i} a_j = \mathbf{W}_i \cdot \mathbf{a}$$

- Non-linear: Activation function (g):
 - transform sum into activation level.

$$a_i \leftarrow g(in_i) = g \left(\sum_j W_{j,i} a_j \right)$$

A Computing Unit

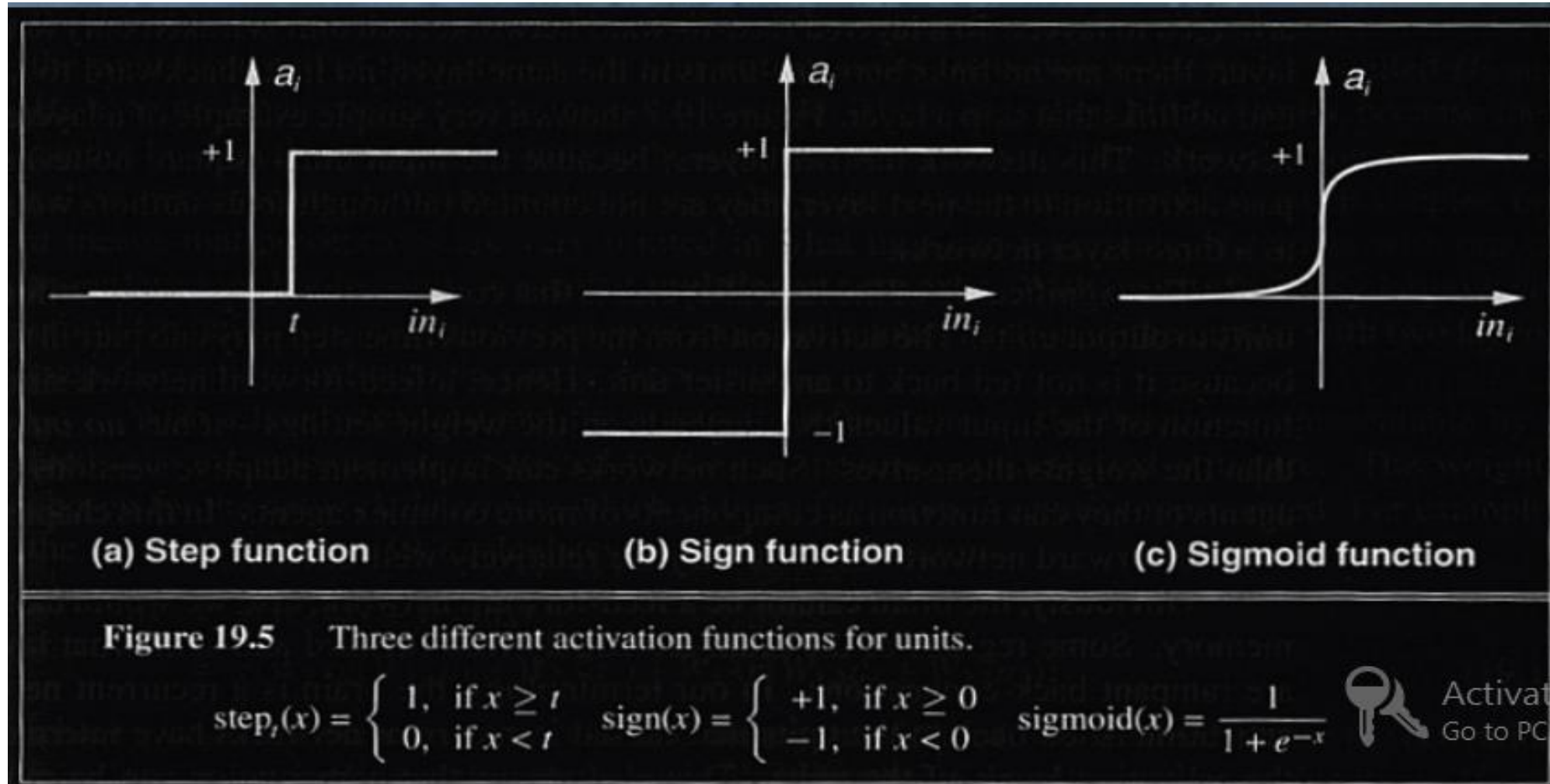
- Now in more detail but for a particular model only.



Activation Functions

- Use different functions to obtain different models.
- 3 most common choices :
 - 1) Step function
 - 2) Sign function
 - 3) Sigmoid function
- An output of 1 represents firing of a neuron down the axon.

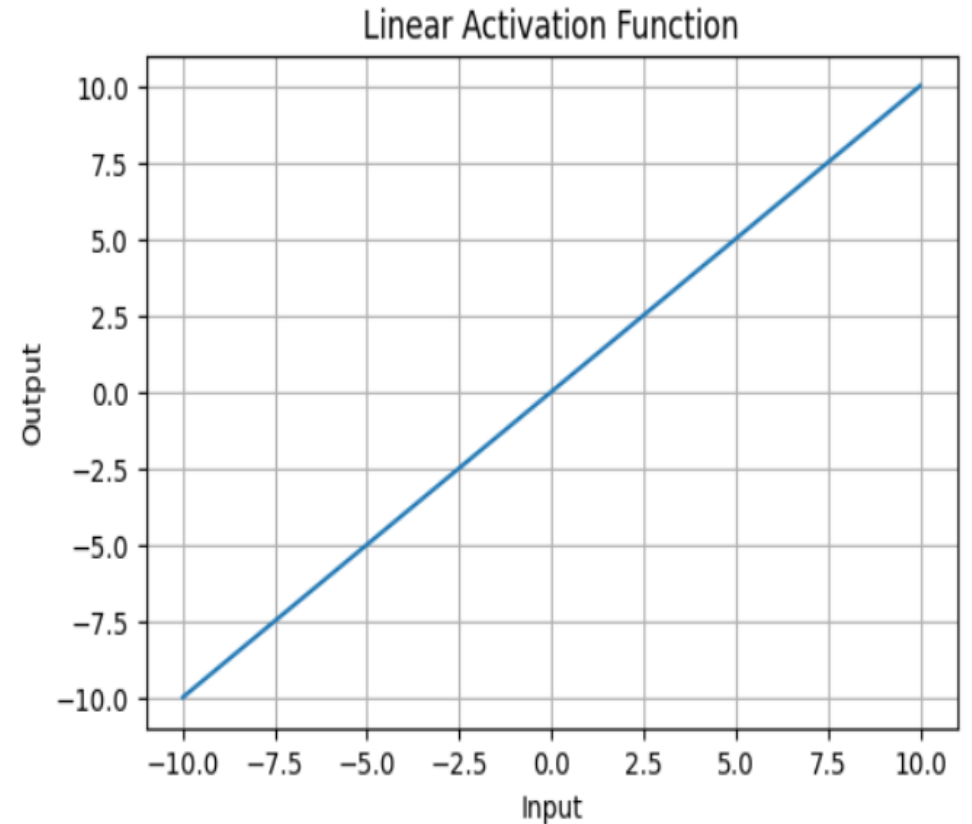
Activation Functions



Types of Activation Functions in Deep Learning

1. Linear Activation Function

- Linear Activation Function resembles straight line define by $y=x$. No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.
- The range of the output spans from $(-\infty$ to $+\infty)$.
- Using it in all layers makes the network behave like a linear model that limits the ability to learn complex patterns.
- Commonly used in the output layer for regression tasks.



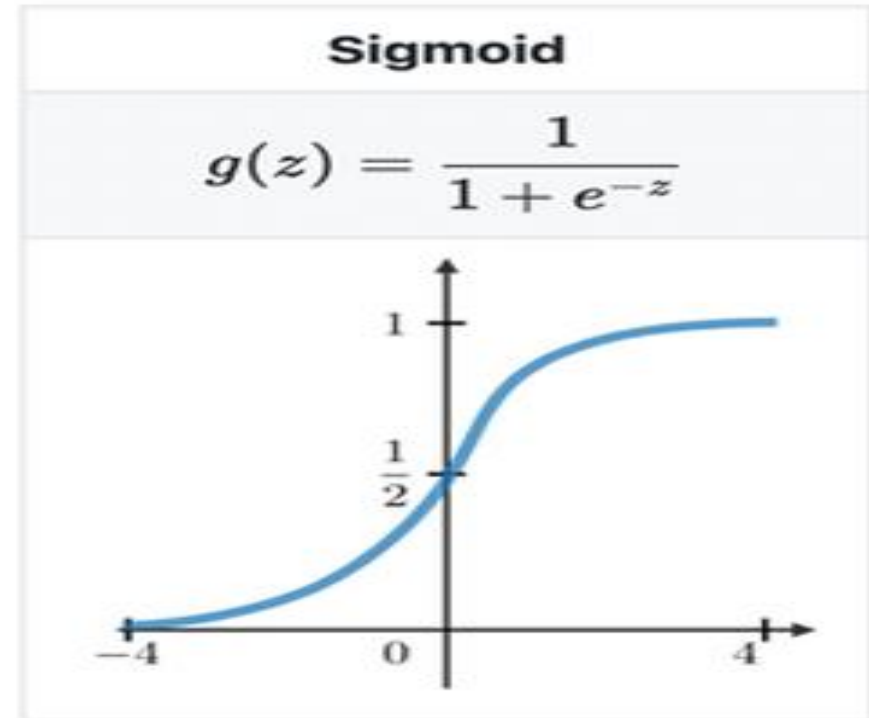
Linear Activation Function or Identity Function returns the input as the output

Types of Activation Functions in Deep Learning

2. Non-Linear Activation Functions

1. Sigmoid Function

- Sigmoid activation function is characterized by 'S' shape.
- It is mathematically defined as $A = \frac{1}{1+e^{-x}}$. This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.
- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.



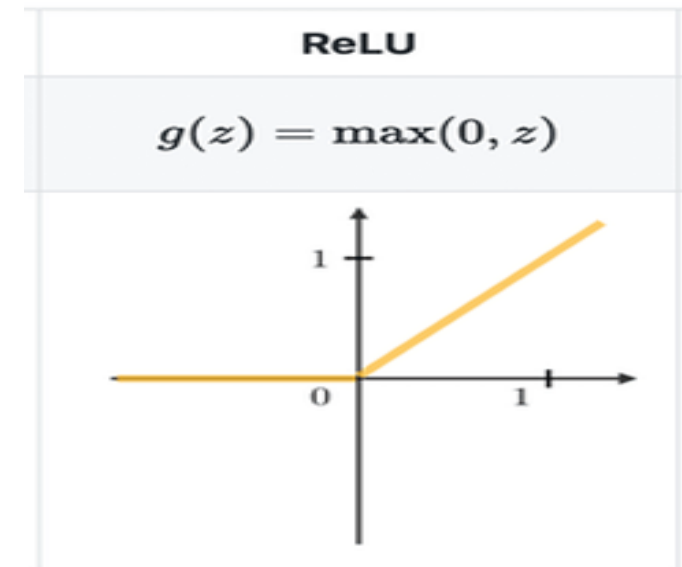
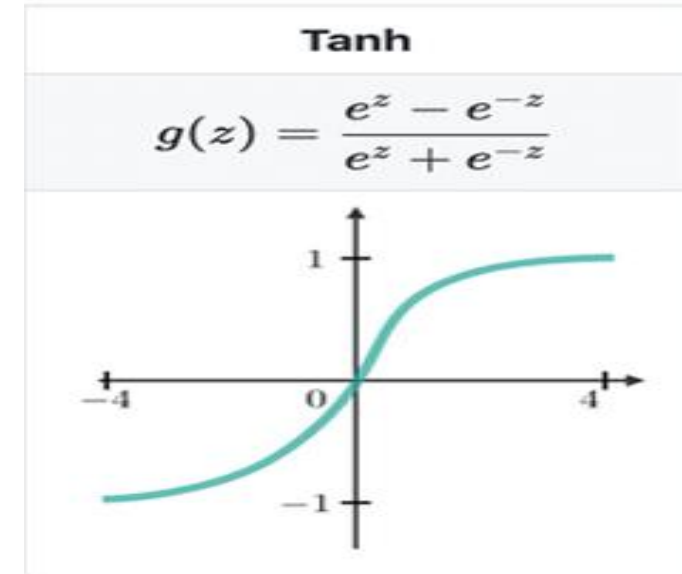
Types of Activation Functions in Deep Learning

2. Tanh Activation Function

- Tanh function (hyperbolic tangent function) is a shifted version of the sigmoid, allowing it to stretch across the y-axis.
- It is defined as: $f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$.
- Outputs values from -1 to +1.

3. ReLU(Rectified Linear Unit)Function

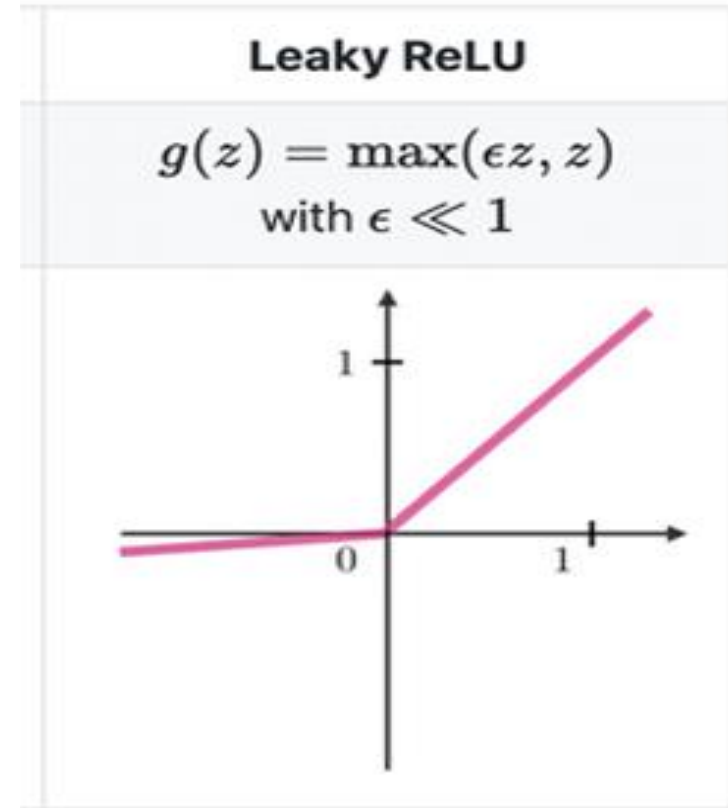
- ReLU activation is defined by $A(x) = \max(0, x)$, this means that if the input x is positive, ReLU returns x , if the input is negative, it returns 0.
- Value Range is $[0, \infty)$, meaning the function only outputs non-negative values.
- Introduces non-linearity, enabling learning of complex patterns
- Activates only positive neurons, making the network sparse and efficient



Types of Activation Functions in Deep Learning

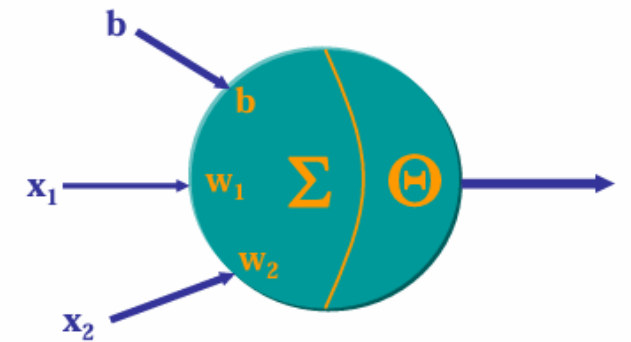
4. Leaky ReLU

- Leaky ReLU is similar to ReLU but allows a small negative slope (α , e.g., 0.01) instead of zero.
- Defined as:
$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$
- Solves the “dying ReLU” problem, where neurons get stuck with zero outputs.
- Range: $(-\infty, +\infty)$



5.2 Perceptron, Multilayer Perceptron and Backpropagation

- Perceptron
- A Perceptron is the simplest kind of feed forward neural network invented by Frank Rosenblatt in 1958.
- A perceptron can learn any linearly separable functions, given enough training i.e. it uses an iterative learning procedure which can be proven to converge to the correct weights for linearly separable data.
- It has a bias and a threshold function.
- The model consists of a linear combiner followed by an activation function.
- The weighted sum of the inputs is applied to the activation function, which produces an output equal to +1 if its input is positive and -1 if it is negative.



Activation function:

$$f(s) = \begin{cases} 1 & \text{if } s > \Theta \\ 0 & \text{if } -\Theta \leq s \leq \Theta \\ -1 & \text{if } s < -\Theta \end{cases}$$

5.2 Perceptron, Multilayer Perceptron and Backpropagation

- Weights are changed only when an error occurs.
- In perceptron error is calculated by taking the difference between calculated output and actual (target) output of a neuron.
- The output “y” is obtained on the basis of the net input calculated & activation function being applied over the net input.

$$\underline{y_{in}} = b + \sum_{i=1}^n x_i w_i$$

$$\underline{y} = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

- The weights are updated using the following:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i$$

t is either +1 or -1

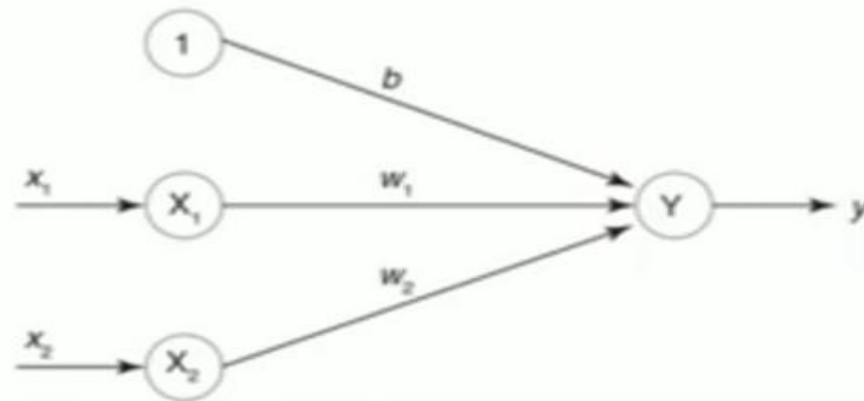
α is the learning rate $0 < \alpha \leq 1$

If an error does not occur, the weights are not changed

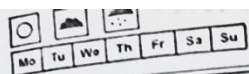
Perceptron Example: AND Function

- The perceptron network, which uses perceptron learning rule, is used to train the AND function.
- The input patterns are presented to the network one by one.
- When all the four input patterns are presented, then one epoch is said to be completed.
- The initial weights and threshold are set to zero.
- The learning rate is set equal to 1.

x_1	x_2	t
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1



Perceptron Example: AND Function



Date: / /

Here, we have initialized all the weights values to 0 & let $\alpha = 1$.

Activation funcⁿ used is:

$$y = f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} > 0 \\ 0, & \text{if } y_{in} = 0 \\ -1, & \text{if } y_{in} < 0 \end{cases}$$

Epoch 1: Initial weights $w_1 = w_2 = b = 0$

① Take 1st set of i/p (1, 1) with $t = 1$.
Calculate net i/p as:

$$y_{in} = 1 \times 0 + 1 \times 0 + 1 \times 0 = 0$$

$$y = f(y_{in}) = f(0) = 0 \quad \therefore \text{error} \because y \neq t$$

$$\Delta w_1 = \alpha \cdot t \cdot x_1 = 1 \times 1 \times 1 = 1$$

$$\Delta w_2 = \alpha \cdot t \cdot x_2 = 1 \times 1 \times 1 = 1$$

$$\Delta b = \alpha \cdot t = 1 \times 1 = 1$$

$$\therefore w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 0 + 1 = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 0 + 1 = 1$$

$$b(\text{new}) = b(\text{old}) + \Delta b = 0 + 1 = 1$$

② Take 2nd set of i/p (1, -1) with $t = -1$

$$y_{in} = 1 \times 1 + 1 \times 1 + -1 \times 1 = 1$$

$$y = f(1) = 1 \quad \therefore y \neq t$$

Now, update weights as:

$$\Delta w_1 = \alpha \cdot t \cdot x_1 = 1 \times -1 \times 1 = -1$$

$$\Delta w_2 = \alpha \cdot t \cdot x_2 = 1 \times -1 \times -1 = +1$$

$$\Delta b = \alpha \cdot t = 1 \times -1 = -1$$

$$\therefore w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 1 + (-1) = 0$$

$$\therefore w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 1 + 1 = 2$$

$$\therefore b(\text{new}) = b(\text{old}) + \Delta b = 1 - 1 = 0$$

③ Take 3rd set of i/p (-1, 1) with target o/p $t = -1$.

$$y_{in} = 1 \times 0 + -1 \times 0 + 1 \times 2 = 2$$

$$\therefore y = f(2) = 1 \quad \therefore y \neq t.$$

Again, we update the weights as:

$$\Delta w_1 = \alpha \cdot t \cdot x_1 = 1 \times -1 \times -1 = 1$$

$$\Delta w_2 = \alpha \cdot t \cdot x_2 = 1 \times -1 \times 1 = -1$$

$$\Delta b = \alpha \cdot t = 1 \times -1 = -1$$

$$w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 0 + 1 = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 2 - 1 = 1$$

$$b(\text{new}) = b(\text{old}) + \Delta b = 0 - 1 = -1$$

④ Take 4th set of i/p (-1, -1) with target o/p $t = -1$.

$$y_{in} = 1 \times -1 + (-1 \times 1) + (-1 \times 1) = -3$$

$$\therefore y = f(-3) = -1 \quad \therefore y = t$$

Perceptron Example: AND Function

Hence, no need for calculation of Δw .

$$w_1(\text{new}) = w_1(\text{old}) = 1$$

$$w_2(\text{new}) = w_2(\text{old}) = 1$$

$$b(\text{new}) = b(\text{old}) = -1$$

Epoch-2

Here, $w_1 = 1$, $w_2 = 1$ & $b = -1$

① Take 1st i/p set $(1, 1)$ with $t = 1$.

$$y_{in} = b + x_1 w_1 + x_2 w_2$$

$$y_{in} = 1 \times -1 + 1 \times 1 + 1 \times 1 = 1$$

$$\therefore y = f(1) = 1 \quad \therefore y = t$$

$$\therefore w_1(\text{new}) = w_1(\text{old}) = 1$$

$$\therefore w_2(\text{new}) = w_2(\text{old}) = 1$$

$$\therefore b(\text{new}) = b(\text{old}) = -1$$

② Take 2nd i/p set $(1, -1)$ with $t = -1$.

$$y_{in} = 1 \times -1 + 1 \times 1 + (-1 \times 1) = -1$$

$$\therefore y = f(-1) = -1 \quad \therefore y = t$$

$$\therefore w_1(\text{new}) = w_1(\text{old}) = 1$$

$$\therefore w_2(\text{new}) = w_2(\text{old}) = 1$$

$$\therefore b(\text{new}) = b(\text{old}) = -1$$

③ Take 3rd i/p set $(-1, 1)$ with $t = -1$.

$$y_{in} = 1 \times -1 + -1 \times 1 + 1 \times 1 = -1$$

$$\therefore y = -1 \quad \therefore y = t$$

$$\therefore w_1(\text{new}) = 1, w_2(\text{new}) = 1 \text{ \& } b(\text{new}) = -1$$

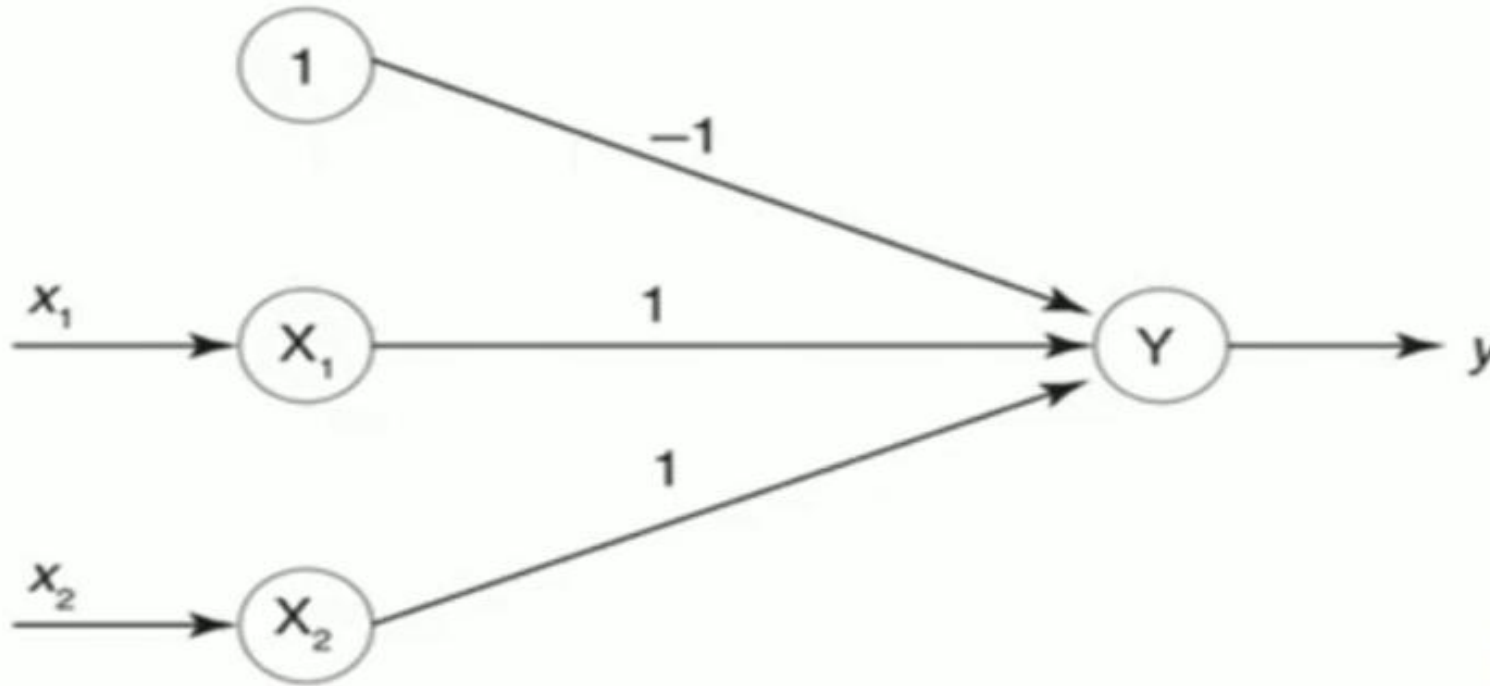
④ Take 4th i/p set $(-1, -1)$ with $t = -1$.

$$y_{in} = 1 \times -1 + (-1 \times 1) + (-1 \times 1) = -3$$

$$\therefore y = -1 \quad \therefore y = t$$

$$\therefore w_1(\text{new}) = 1, w_2(\text{new}) = 1 \quad \therefore b(\text{new}) = -1$$

Perceptron Example: AND Function



Limitations of Perceptron

- Can solve only linearly separable problems, hence logic gates such as XOR cannot be implemented.
- There is no hidden layer to learn intermediate features. Therefore, it cannot learn complex patterns.
- Cannot learn complex nonlinear relationships.
- The perceptron uses a hard threshold activation function, commonly implemented as the sign (sgn) function (or equivalently a step function). Such that output is discrete (either $-1/+1$ or $0/1$). The output changes abruptly at the threshold. The activation function is not differentiable at the threshold and has a derivative of zero almost everywhere else. Hence, gradient information is unavailable, gradient-descent-based learning such as backpropagation cannot be applied directly to the classical perceptron.

Adaline Network

- Adaline network is a variation on the Perceptron Network.
 - inputs are +1 or -1
 - outputs are +1 or -1
 - uses a bias input
- It is trained using the Delta Rule which is also known as the least mean squares (LMS) or Widrow-Hoff rule.
- The activation function, during training is the identity function. After training the activation is a threshold function.

Adaline Algorithm

Step 0: initialize the weights to small random values and select a learning rate, α .

Step 1: for each input vector s , with target output, t set the inputs to s .

Step 2: compute the neuron inputs

Neuron input

$$y_in = b + \sum x_i w_i$$

Step 3: use the delta rule to update the bias and weights:

Delta rule

$$b(\text{new}) = b(\text{old}) + \alpha(t - y_in)$$

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_in)x_i$$

Step 4: stop if the largest weight change across all the training samples is less than a specified tolerance, otherwise cycle through the training set again.

Learning Rate for Adaline network

- The performance of an ADALINE neuron depends heavily on the choice of the learning rate.
 - if it is too large the system will not converge
 - if it is too small the convergence will take to long
- Typically, α is selected by trial and error
 - typical range: $0.01 < \alpha < 10.0$
 - often start at 0.1

Running Adaline

- One unique feature of ADALINE is that its activation function is different for training and running
- When running ADALINE use the following:
 - initialize the weights to those found during training
 - compute the net input
 - apply the activation function

Neuron input

$$y_{\text{in}} = \mathbf{b} + \sum \mathbf{x}_i \mathbf{w}_i$$

Activation Function

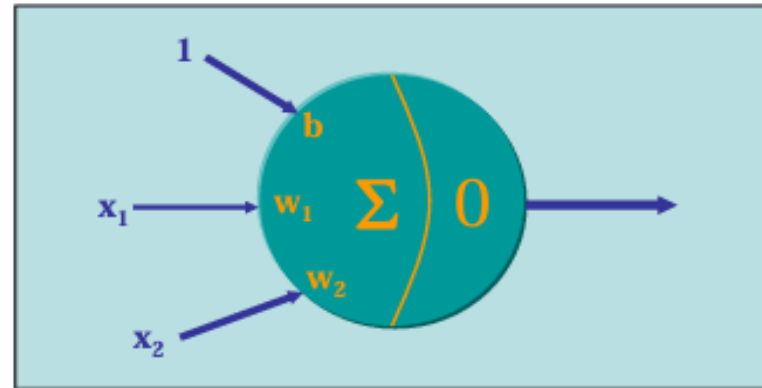
$$y = \begin{cases} 1 & \text{if } y_{\text{in}} \geq 0 \\ -1 & \text{if } y_{\text{in}} < 0 \end{cases}$$

Example – AND Function

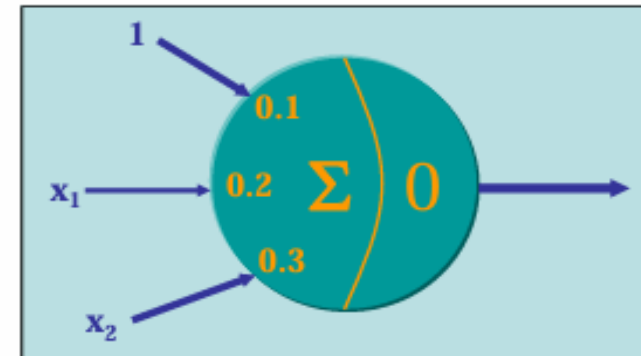
- Construct an AND function for an ADALINE neuron:

– let $\alpha = 0.1$

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1



Initial Conditions: Set the weights to small random values:

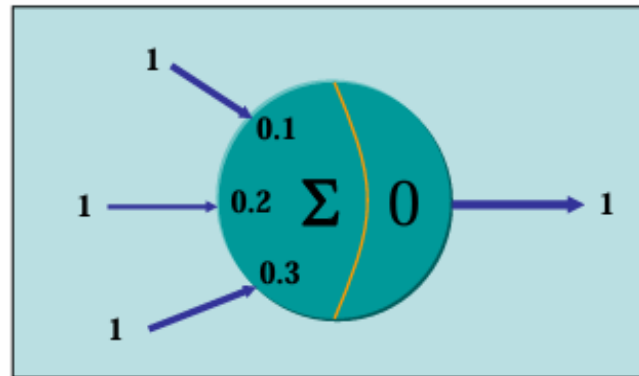


First Training Run

- Apply the input (1,1) with output 1

The net input is:

$$y_in = 0.1 + 0.2*1 + 0.3*1 = 0.6$$



The new weights are:

$$b = 0.1 + 0.1(1-0.6) = 0.14$$

$$w_1 = 0.2 + 0.1(1-0.6)1 = 0.24$$

$$w_2 = 0.3 + 0.1(1-0.6)1 = 0.34$$

**The largest
weight
change
is 0.04**

Neuron input

$$y_in = b + \sum x_i w_i$$

Delta rule

$$b(\text{new}) = b(\text{old}) + \alpha(t - y_in)$$

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_in)x_i$$

Second Training Run

- Apply the second training set (1 -1) with output -1

The net input is:

$$y_{in} = 0.14 + 0.24*1 + 0.34*(-1) = 0.04$$

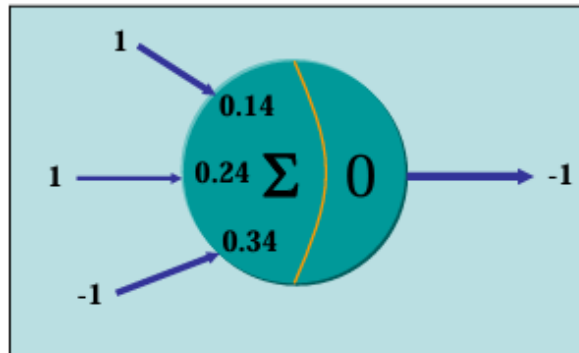
The new weights are:

$$b = 0.14 - 0.1(1+0.04) = 0.04$$

$$w_1 = 0.24 - 0.1(1+0.04)1 = 0.14$$

$$w_2 = 0.34 + 0.1(1+0.04)1 = 0.44$$

**The largest
weight
change
is 0.1**



Third Training Run

- Apply the third training set (-1 1) with output -1

The net input is:

$$y_{in} = 0.04 - 0.14 * 1 + 0.44 * 1 = 0.34$$

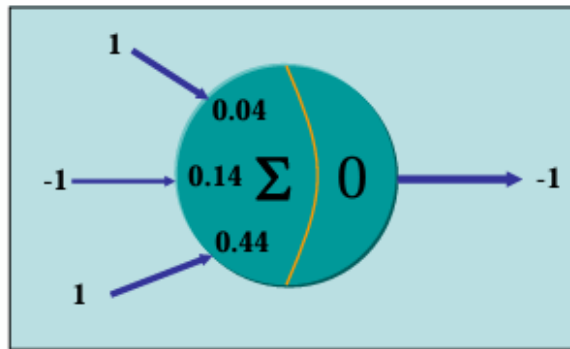
The new weights are:

$$b = 0.04 - 0.1(1+0.34) = -0.09$$

$$w_1 = 0.14 + 0.1(1+0.34)1 = 0.27$$

$$w_2 = 0.44 - 0.1(1+0.34)1 = 0.31$$

**The largest
weight
change
is 0.13**



Fourth Training Run

- Apply the fourth training set (-1 -1) with output -1

The net input is:

$$y_{in} = -0.09 - 0.27*1 - 0.31*1 = -0.67$$

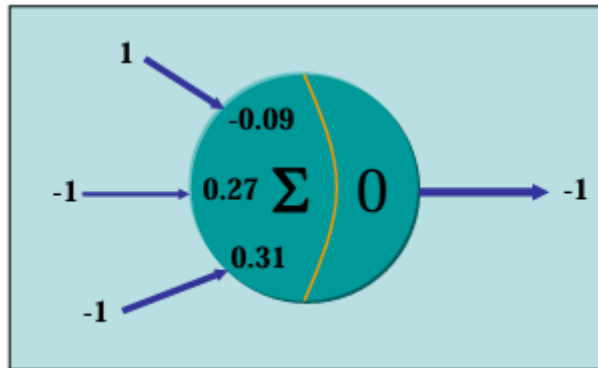
The new weights are:

$$b = -0.09 - 0.1(1+0.67) = -0.27$$

$$w_1 = 0.27 + 0.1(1+0.67)1 = 0.43$$

$$w_2 = 0.31 + 0.1(1+0.67)1 = 0.47$$

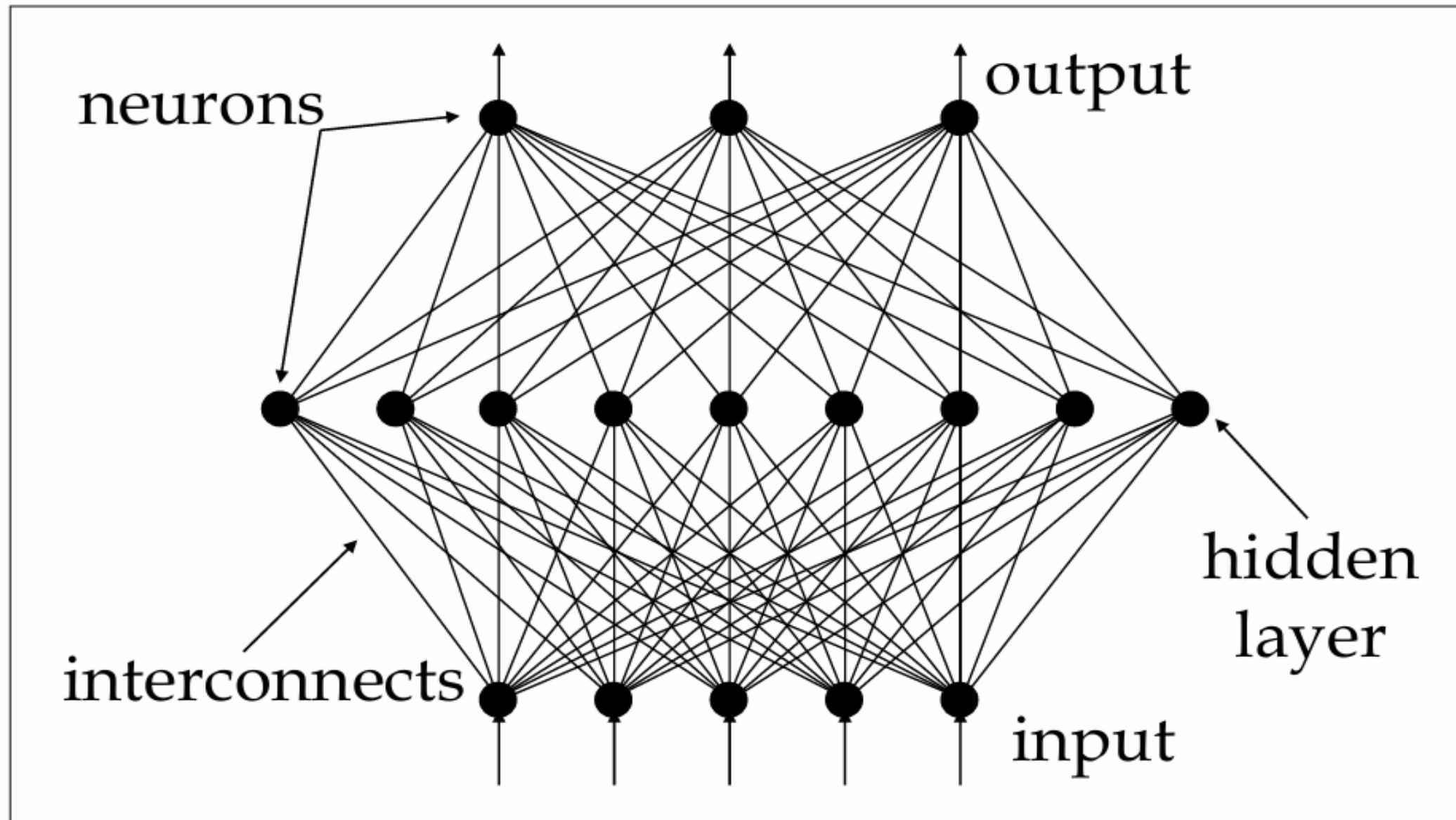
**The largest
weight
change
is 0.16**



Conclusion

- Continue to cycle through the four training inputs until the largest change in the weights over a complete cycle is less than some small number (say 0.01).
- In this case, the solution becomes:
 - $b = -0.5$
 - $w_1 = 0.5$
 - $w_2 = 0.5$

Multi-Layered Perceptron



Properties

- Patterns of activation are presented at the inputs and the resulting activation of the outputs is computed.
- The values of the weights determine the function computed.
- A network with one hidden layer is sufficient to represent every Boolean function.
- With real weights every real valued function can be approximated with a single hidden layer.

Backpropagation

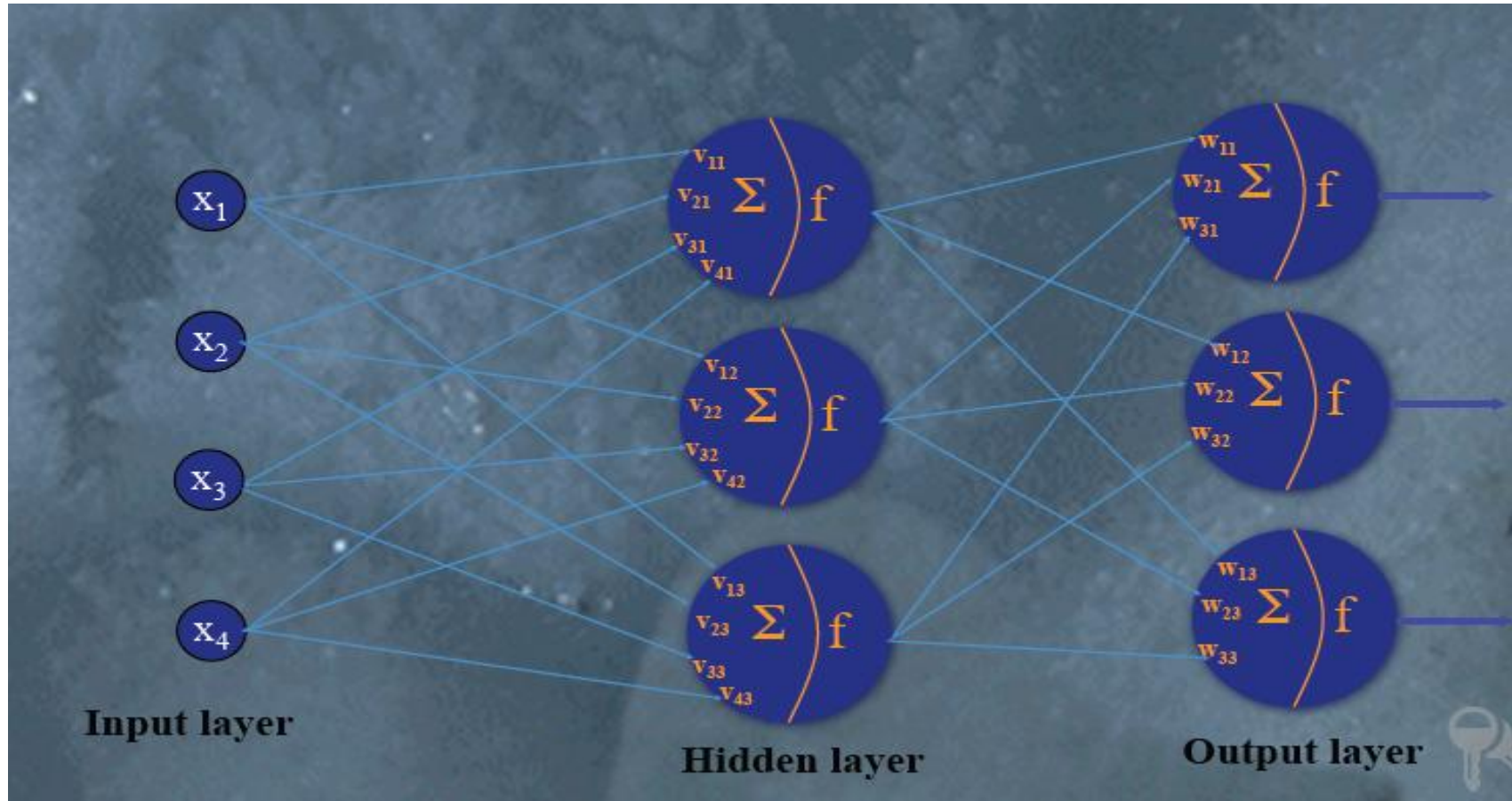
- Backpropagation is the most well know and widely used neural network system.
- It is a multi-layered, perceptron-like structure.
- Uses the backpropagation rule (or generalized delta rule) for training.
- Back propagation networks are necessarily multilayer perceptron (usually with one input, one hidden, and one output layer).
- As the algorithm's name implies, the errors (and therefore the learning) propagate backwards from the output nodes to the inner nodes. So technically speaking, back propagation is used to calculate the gradient of the error of the network with respect to the network's modifiable weights.

Characteristics

- A multi-layered perceptron has three distinctive characteristics.
 - The network contains one or more layers of hidden neurons.
 - The network exhibits a high degree of connectivity.
 - New value from the hidden layer is send to the output layer.
 - Output layer processes the information received from the hidden layer and produces an output.
 - Final output is then processed by an activation function.

Architecture

- Typical backpropagation system (bias not show).



Backpropagation

1. Number of Nodes and Layers

- Depends upon the training sets

2. Setting the Weight

- Initial weights are set at some random numbers and they are adjusted during the NN training.
- Changes of the weights should be attempted simultaneously.
- Weight Update
 - Updated after each iteration.

Backpropagation

3. Running and Training Neural Network

➤ Consists of Forward Pass and Backward Pass.

Forward Pass

➤ Outputs are calculated and compared with the desired outputs.

➤ Error from the desired and actual output is calculated.

Backward Pass

➤ Error is used to alter the weights in the network in order to reduce the size of the error.

Forward Pass and Backward Pass are repeated until the error is low enough.

Back propagation technique

- Randomly choose the initial weights
- While error is too large
 - For each training pattern (presented in random order):
 - Apply the inputs to the network
 - Calculate the output for every neuron from the input layer, through the hidden layer(s), to the output layer
 - Calculate the error at the outputs
 - Use the output error to compute error signals for pre-output layers
 - Use the error signals to compute weight adjustments
 - Apply the weight adjustments
- Periodically evaluate the network performance

Options

- There are a number of options in the design of a backpropagation system:
- Initial weights – best to set the initial weights (and all other free parameters) to random numbers inside a small range of values :
(say – 0.5 to 0.5)
- Number of cycles – tend to be quite large for backpropagation systems
- Number of neurons in the hidden layer – as few as possible
- Learning rate should not be very high or very low. The most popular learning rate is 0.35.
- Generally, the activation function used mostly is the sigmoid function.

Momentum

- Momentum is a technique used in the backpropagation algorithm to accelerate learning and prevent oscillations by incorporating past weight updates into the current update.
- **Why is Momentum Needed?**
 - 1.Smooth Updates: Reduces sudden changes in direction due to noisy gradients.
 - 2.Faster Convergence: Helps the model escape shallow local minima and move faster in valleys.
 - 3.Prevents Oscillations: Stabilizes learning, especially in scenarios where gradients fluctuate frequently.

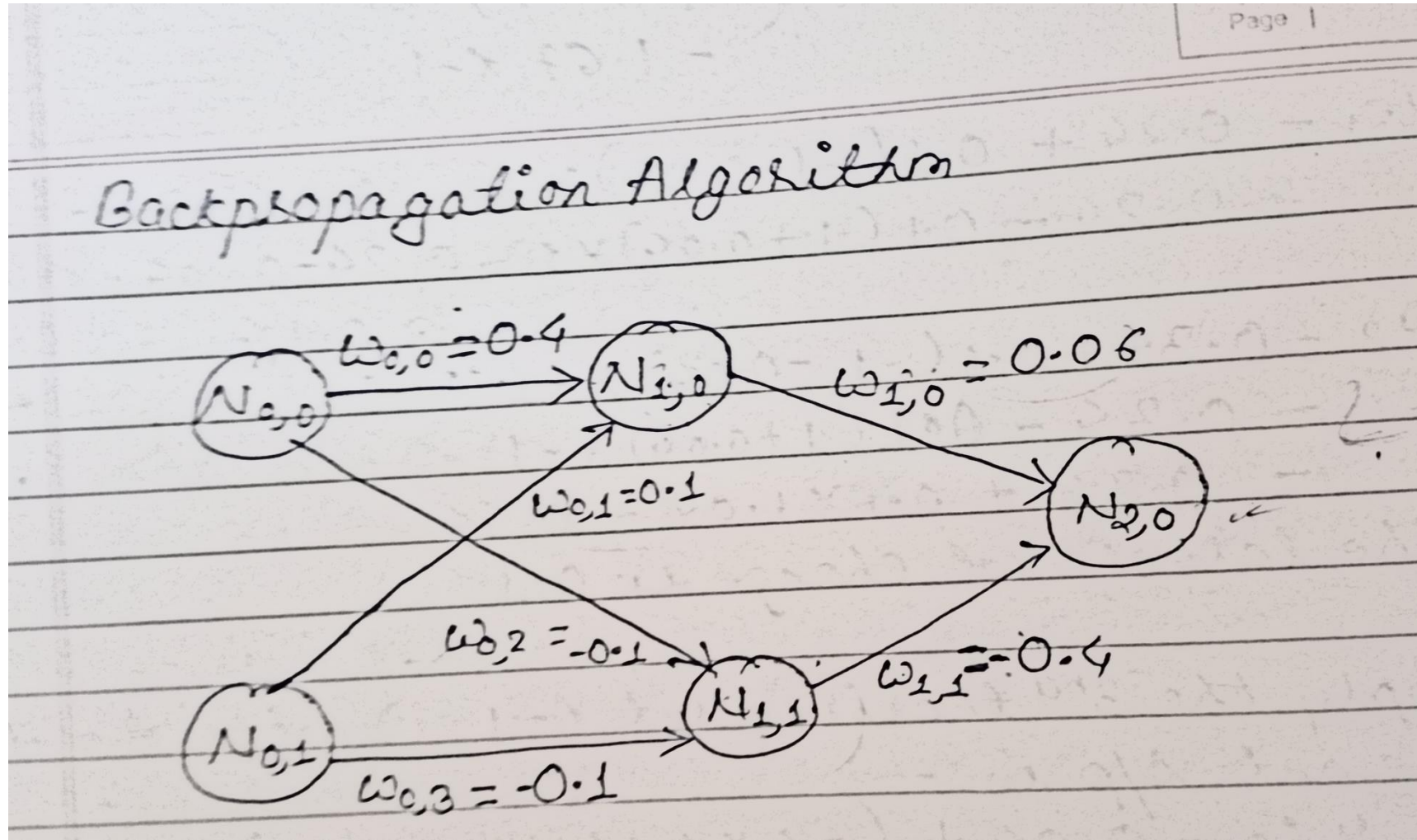
Momentum

- In the backpropagation algorithm, the momentum term typically ranges between **0 and 1**.
- **0 (No Momentum)**: The update depends only on the current gradient, with no influence from past updates.
- **0.5 - 0.9 (Typical Values)**: These values help smooth out updates, preventing oscillations and accelerating learning in the right direction.
- **>1 (Rarely Used)**: This can lead to instability, causing the updates to overshoot and diverge.
- A common default choice in practice is **0.9**, as it provides a good balance between stability and faster convergence.

Overall Steps Involved

1. Feed Forward Computation
2. Backpropagation to Output Layer
3. Backpropagation to Hidden Layer
4. Weight Updates

Example: AND gate Implementation



Example: AND gate Implementation

For AND gate implementation.

$N_{0,0}$	$N_{0,1}$	O/P ($N_{3,0}$)
1	1	1
1	0	0
0	1	0
0	0	0

We take, learning rate (α) = 0.45
momentum term (β) = 0.9

Example: AND gate Implementation

$$\text{Sigmoid function } f(x) = \frac{1.0}{1.0 + \exp^{-x}}$$

① Feed Forward Calculation

Step 1:- I/p (1,1) & $t = 1$

$$\begin{aligned} N_{1,0} &= f(w_{0,0} * N_{0,0} + w_{0,1} * N_{0,1}) \\ &= 0.4 * 1 + 0.1 * 1 \\ &= 0.5 \end{aligned}$$

$$f(0.5) = \frac{1}{1.0 + \exp^{-0.5}} = 0.622459$$

Example: AND gate Implementation

$$\begin{aligned}N_{1,1} &= f(\omega_{0,2} \times N_{0,0} + \omega_{0,3} \times N_{0,1}) \\&= -0.1 \times 1 + (-0.1 \times 1) \\&= -0.2\end{aligned}$$

$$f(-0.2) = \frac{1}{1 + e^{-(-0.2)}} = 0.450166$$

Step 2: Calculate ^{weight} o/p for o/p layer.

$$\begin{aligned}N_{2,0} &= f(N_{1,0} \times \omega_{1,0} + N_{1,1} \times \omega_{1,1}) \\&= 0.622459 \times 0.06 + 0.450166 \times (-0.4) \\&= 0.03974754 - 0.1800664 \\&= -0.1403188\end{aligned}$$

$$f(x_3) = \frac{1}{1.0 + e^{-(-0.1427188)}} = 0.464381$$

Example: AND gate Implementation

② Back Propagation to the o/p layer

Step 1: Calculation of error of $N_{2,0}$ node from the table, o/p should be 1.

Predicted o/p at $N_{2,0} = 0.464381$

$$\text{Initial Error } (N_{2,0} \text{ Error}) = N_{2,0} \times (1 - N_{2,0}) \times (1 - N_{2,0})$$

$$= 0.464381 \times (1 - 0.464381)^2$$
$$= 0.133225$$

Once this error is known, it will be used for backward propagation.

Step 2: Error is propagated from o/p layer to the hidden layer. The weight $w_{1,0}$ & $w_{1,1}$ will be adjusted using learning rate & momentum.

Example: AND gate Implementation

$$\begin{aligned}\Delta w_{1,0} &= \alpha \times N_{2,0} \text{error} \times N_{1,0} \\ \text{rate of change} &= 0.45 \times 0.133225 \times 0.622459 \\ \text{of wt} &= 0.037317 \\ w_{1,0}(\text{new}) &= w_{1,0}(\text{old}) + \Delta w_{1,0} + [\beta \times \Delta(t-1)] \\ &= 0.06 + 0.037317 + 0.9 \times (1-1) \\ &= 0.097317 \\ \Delta w_{1,1} &= \alpha \times N_{2,0} \text{Error} \times N_{1,1} \\ &= 0.45 \times 0.133225 \times 0.450166 \\ &= 0.026988 \\ w_{1,1}(\text{new}) &= w_{1,1}(\text{old}) + \Delta w_{1,1} + [\beta \times \Delta(t-1)] \\ &= (-0.4) + 0.026988 + 0 \\ &= -0.373012\end{aligned}$$

Example: AND gate Implementation

③ Back Propagation to Hidden Layer
→ Error has been propagated from hidden layer down to i/p layer.

$$\begin{aligned}N_{1,0}(\text{error}) &= N_{2,0}(\text{error}) \times W_{1,0}(\text{new}) \\&= 0.133225 \times 0.097317 \\&= 0.012965\end{aligned}$$

$$\begin{aligned}N_{1,1}(\text{error}) &= N_{2,0}(\text{error}) \times W_{1,1}(\text{new}) \\&= 0.133225 \times (-0.0373012) \\&= -0.049706\end{aligned}$$

Now, change in weight:

$$\begin{aligned}\Delta W_{0,0} &= \alpha \times N_{1,0}(\text{error}) \times N_{0,0} \\&= 0.45 \times 0.012965 \times 1 \\&= 0.005834\end{aligned}$$

Example: AND gate Implementation

Date |
Page |

$$\begin{aligned}\Delta w_{0,1} &= \alpha * N_{1,0}(\text{error}) * N_{0,1} \\ &= 0.45 * 0.012965 * 1 \\ &= 0.005834\end{aligned}$$
$$\begin{aligned}\Delta w_{0,2} &= \alpha * N_{1,1}(\text{error}) * N_{0,0} \\ &= 0.45 * (-0.049706) * 1 \\ &= -0.022368\end{aligned}$$
$$\begin{aligned}\Delta w_{0,3} &= \alpha * N_{1,1}(\text{error}) * N_{0,1} \\ &= 0.45 * (-0.049706) * 1 \\ &= -0.022368\end{aligned}$$

Example: AND gate Implementation

Now, calculate the new weights betⁿ hidden layer & i/p layer.

$$\begin{aligned}w_{0,0}(\text{new}) &= w_{0,0}(\text{old}) + \Delta w_{0,0} + (\beta \times \Delta(t-1)) \\&= 0.4 + 0.005834 + 0.9 \times 0 \\&= 0.405834\end{aligned}$$

$$\begin{aligned}w_{0,1}(\text{new}) &= w_{0,1}(\text{old}) + \Delta w_{0,1} + (\beta \times \Delta(t-1)) \\&= 0.1 + 0.005834 + 0 \\&= 0.105834\end{aligned}$$

$$\begin{aligned}w_{0,2}(\text{new}) &= w_{0,2}(\text{old}) + \Delta w_{0,2} + (\beta \times \Delta(t-1)) \\&= -0.1 + (-0.022368) + 0 \\&= -0.122368\end{aligned}$$

$$\begin{aligned}w_{0,3}(\text{new}) &= w_{0,3}(\text{old}) + \Delta w_{0,3} + (\beta \times \Delta(t-1)) \\&= -0.1 + (-0.022368) + 0 \\&= -0.122368\end{aligned}$$

Example: AND gate Implementation

④ Weight Updates
→ Now, the important task is to see if error has decreased or not with new weights.
Again, using forward pass.

$$N_{1,0} = f(x_0) = f(w_{0,0} \times N_{0,0} + w_{0,1} \times N_{0,1})$$
$$= f(0.406 \times 1 + 0.1 \times 1)$$
$$= f(0.506)$$
$$= 0.623868314$$
$$N_{1,1} = f(x_2) = f(w_{0,2} \times N_{0,0} + w_{0,3} \times N_{0,1})$$
$$= f(-0.122 \times 1 + (-0.122 \times 1))$$
$$= f(-0.244)$$
$$= 0.43430085$$
$$N_{2,0} = f(x_3) = f(N_{1,0} \times w_{1,0} + N_{1,1} \times w_{1,2})$$
$$= f(0.623868314 \times 0.097317 + 0.43430085 \times (-0.373012))$$
$$= f(0.22274381)$$
$$= 0.474186972$$

Next step is to calculate error of $N_{2,0}$, now from $o/p = 1$.

Predicted Value ($N_{2,0}$) = 0.474186972

Error Calculation:-

$$N_{2,0}(\text{Error}) = N_{2,0} \times (1 - N_{2,0}) \times (1 - N_{2,0})$$
$$= N_{2,0} \times (1 - N_{2,0})^2$$
$$= 0.474186972 \times (1 - 0.474186972)^2$$
$$= 0.1311029015$$

Initial Error = 0.133225 Improved!!

Why Learning Converges: Universal Approximation Theorem

- The Universal Approximation Theorem states that “A feed-forward neural network with only one hidden layer containing a sufficient number of neurons can approximate any continuous function on a bounded domain to any desired level of accuracy.”
- Formally, the theorem can be expressed as:
- Let $f(x)$ be any continuous function. Then there exists a neural network $g(x)$ having one hidden layer, finite number of neurons & nonlinear activation function such that $|f(x) - g(x)| < \epsilon$, for any small $\epsilon > 0$.
- This means that we can make the prediction error as small as we want.
- Neural networks approximate functions by adjusting the weights and biases of their neurons. When a neural network is trained, it iteratively adjusts these parameters to minimize the error between its predictions and the actual outputs.
- The idea behind the Universal Approximation Theorem is that hidden layers can capture increasingly complex patterns in the data.
- When enough neurons are used, the network can learn subtle nuances of the target function

Why Learning Converges: Universal Approximation Theorem

- Mathematical Foundations of Function Approximation
- A neural network's function $\hat{f}(x)$ can be described mathematically as a composition of linear transformations and activation functions.
- For a network with a single hidden layer, the output is given by:

$$\hat{f}(x) = \sum_{i=1}^M c_i \cdot \sigma(w_i^T x + b_i)$$

Where:

- M is the number of neurons in the hidden layer.
- c_i are the weights associated with the output layer.
- w_i and b_i are the weights and biases of the hidden neurons.
- σ is the activation function (commonly non-linear).
- The idea is that, by adjusting the weights c_i , w_i and b_i , the neural network can approximate any continuous function $f(x)$ over a given domain.

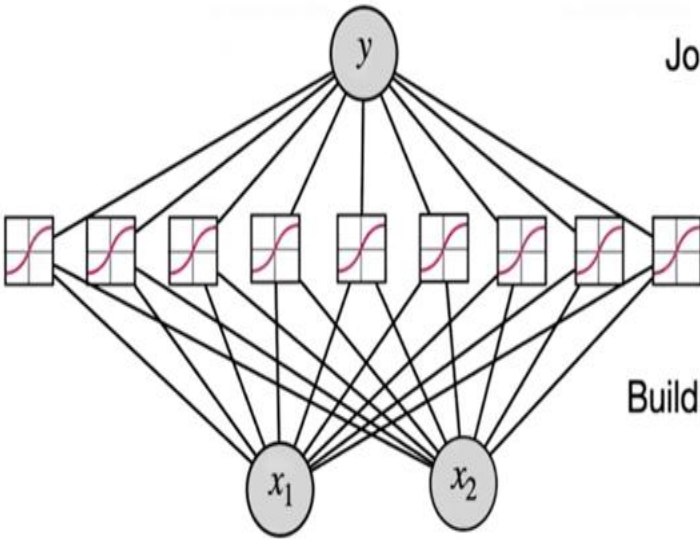
Why Learning Converges: Universal Approximation Theorem

- Role of Activation Functions

- A crucial aspect of the Universal Approximation Theorem is the requirement for non-linearity in the neural network, introduced via the activation function σ .
- Without non-linearity, the network would reduce to a simple linear model and be unable to approximate complex functions.
- Common Activation Functions
 1. Sigmoid Function
 2. ReLU (Rectified Linear Unit)
 3. Tanh (Hyperbolic Tangent)

Why Learning Converges: Universal Approximation Theorem

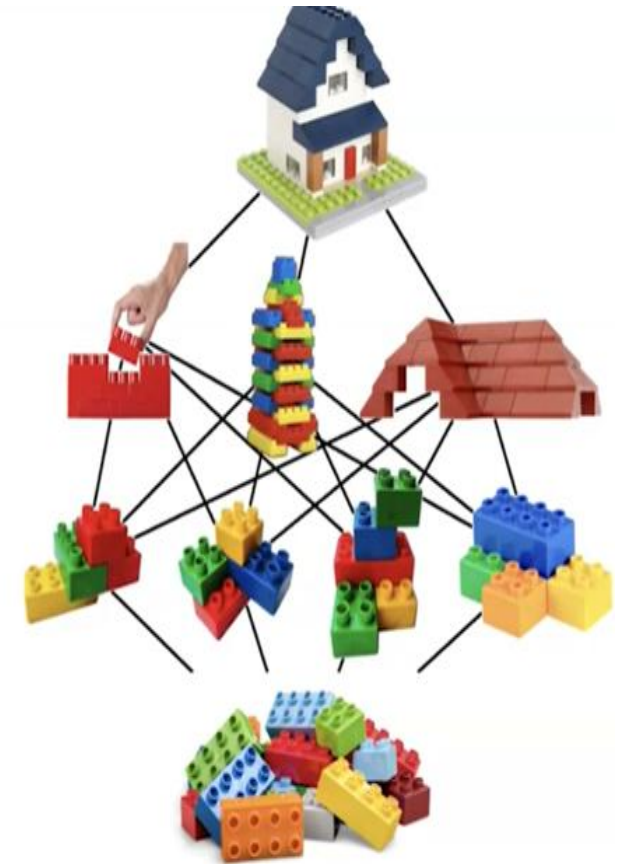
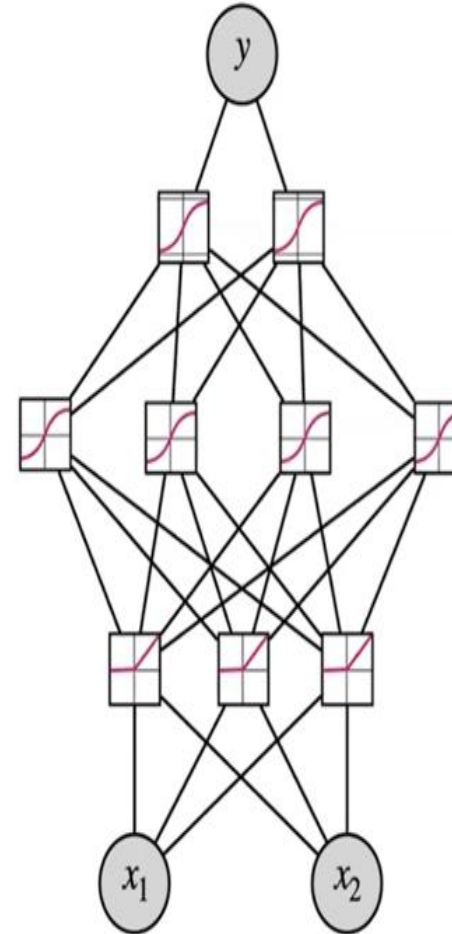
Why two layers?



Join them



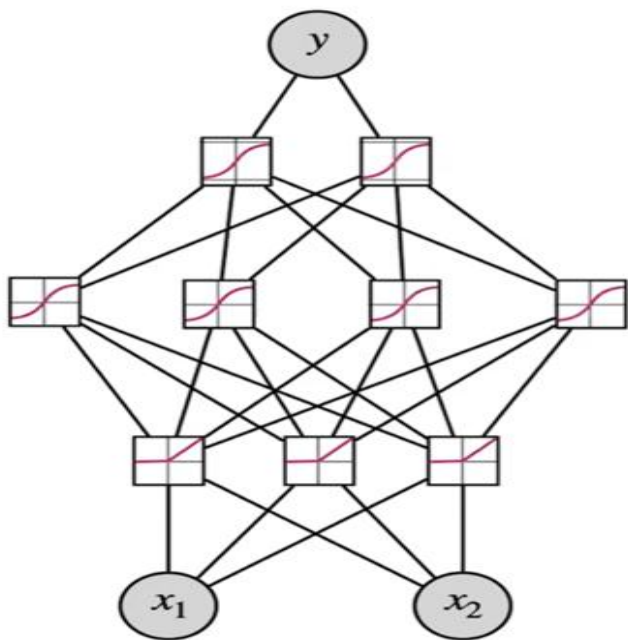
Building blocks



Why Learning Converges: Universal Approximation Theorem

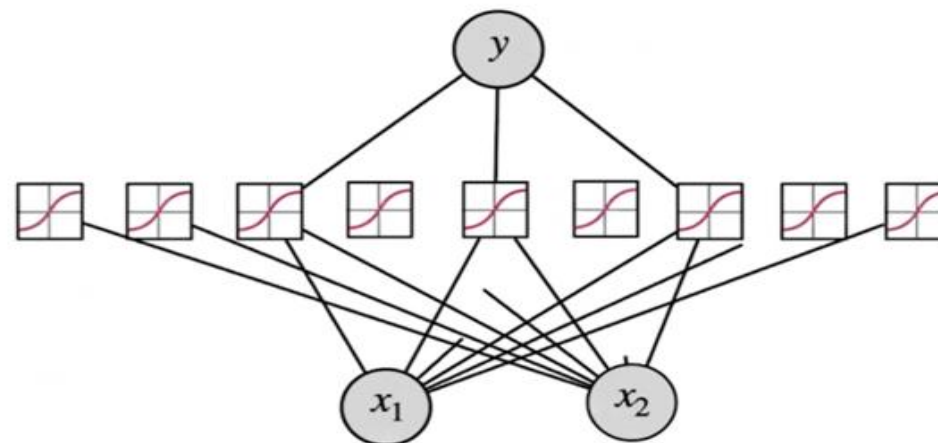


MLP (Multi-Layer Perceptron)



Universal Approximation Theorem

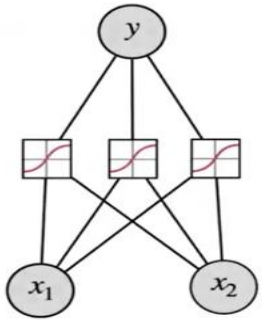
Two layers is all
that's needed!



Problem: Could be too wide

Why Learning Converses: Universal Approximation Theorem

Why two layers?



Function

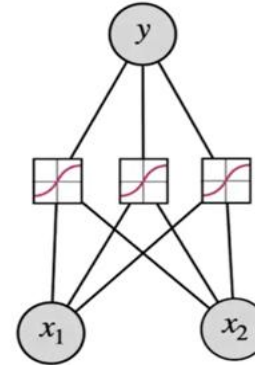


Approximation



Building blocks

Why two layers?



Function

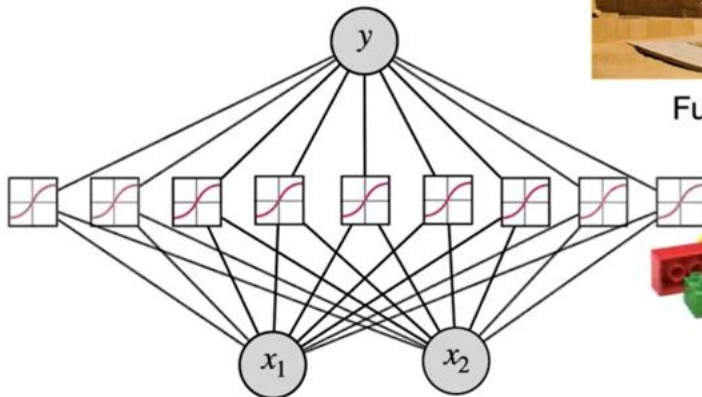


Approximation



Building blocks

Why two layers?



Function

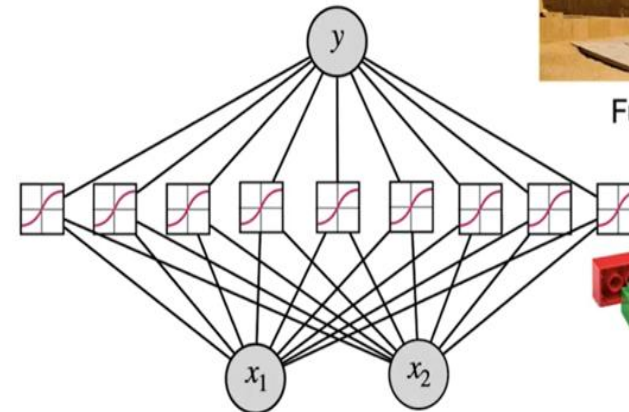


Approximation



Building blocks

Why two layers?



Function



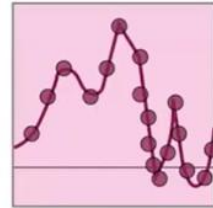
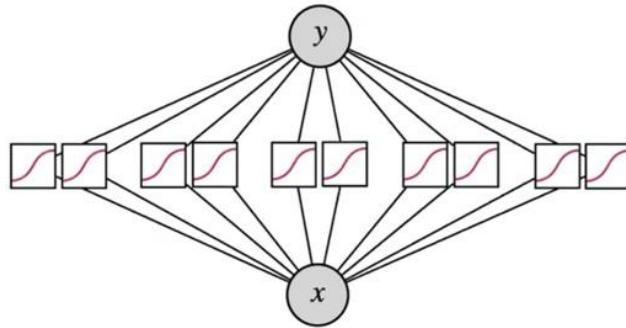
Approximation



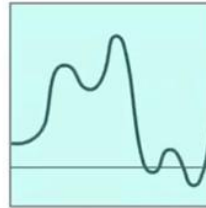
Building blocks

Why Learning Converses: Universal Approximation Theorem

All you need is 2 layers

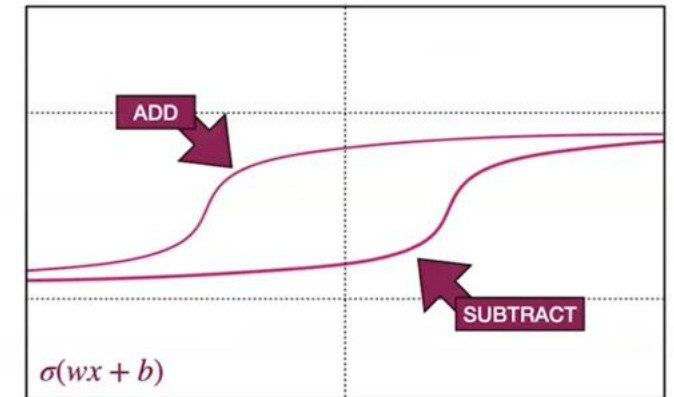
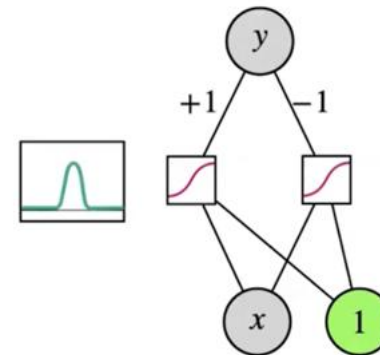
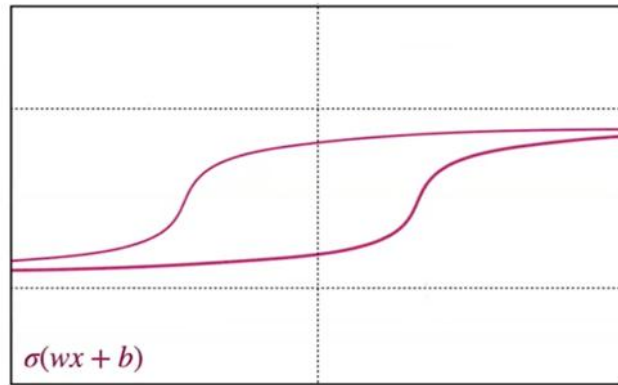
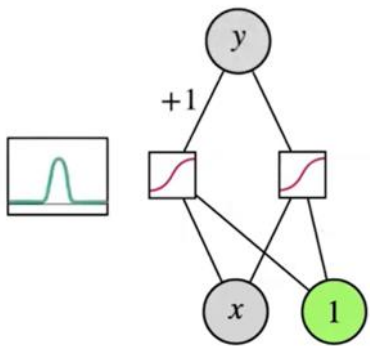
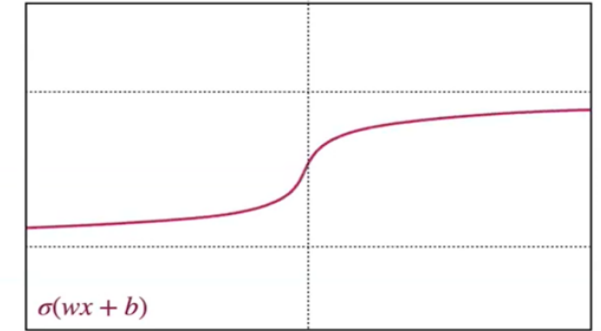
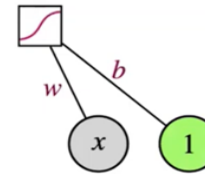


Dataset
Function

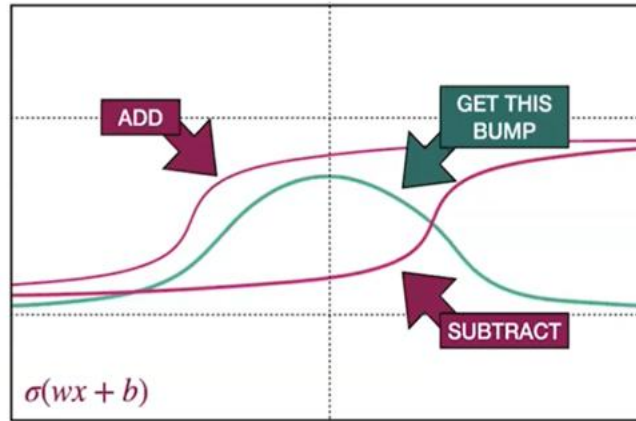
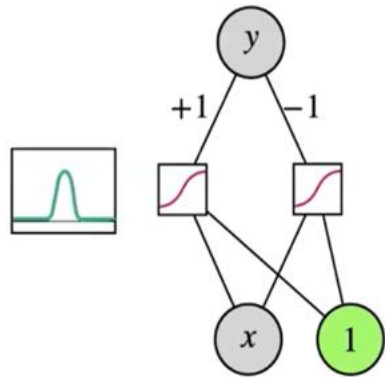


Model

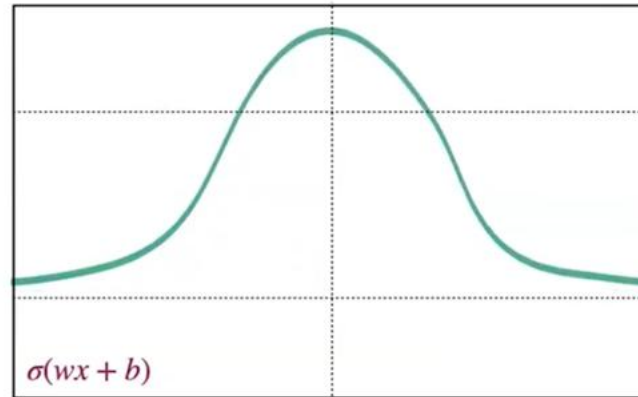
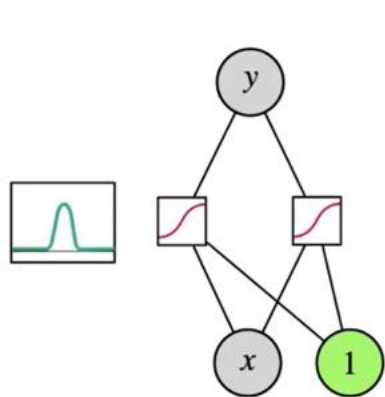
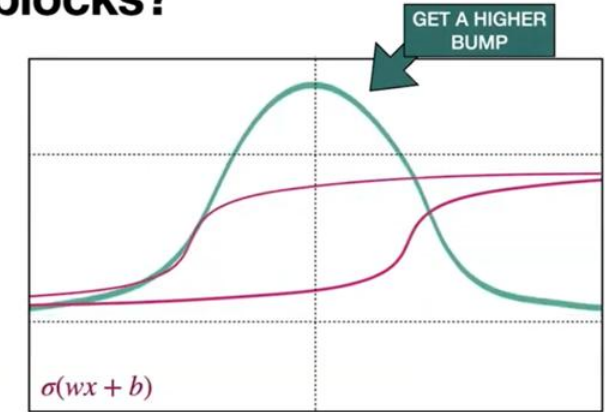
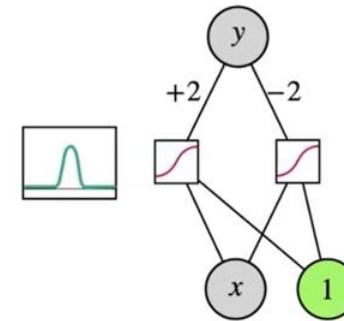
How to build the blocks?



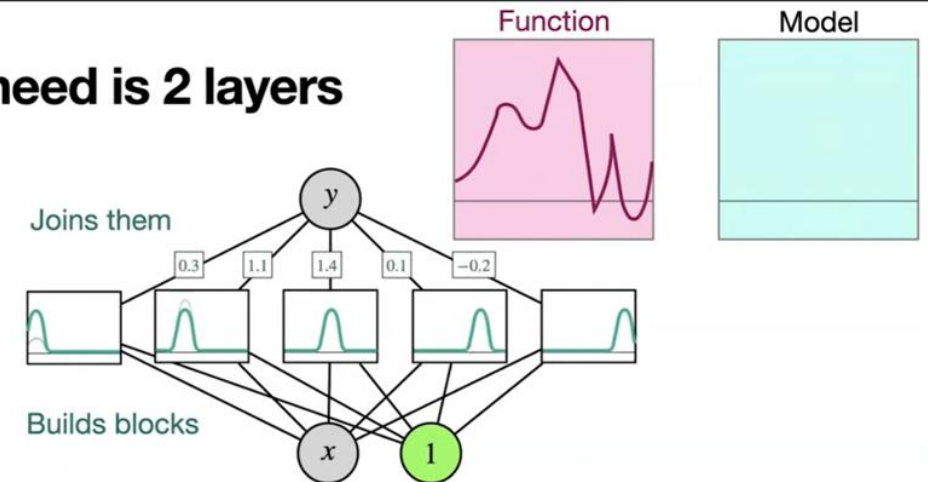
Why Learning Converges: Universal Approximation Theorem



How to build the blocks?

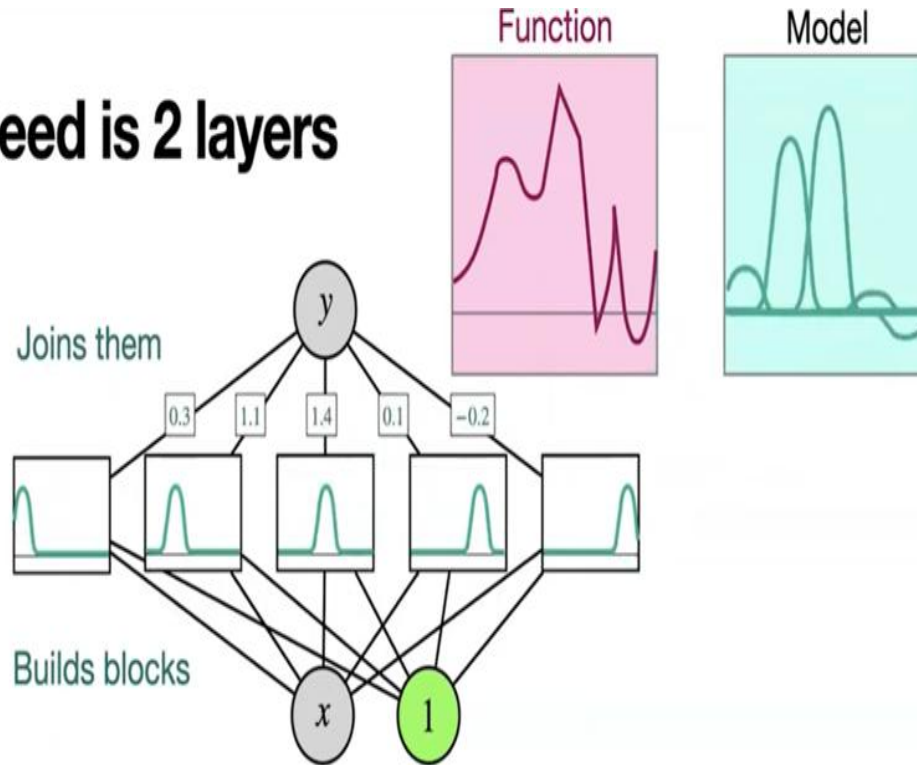


All you need is 2 layers

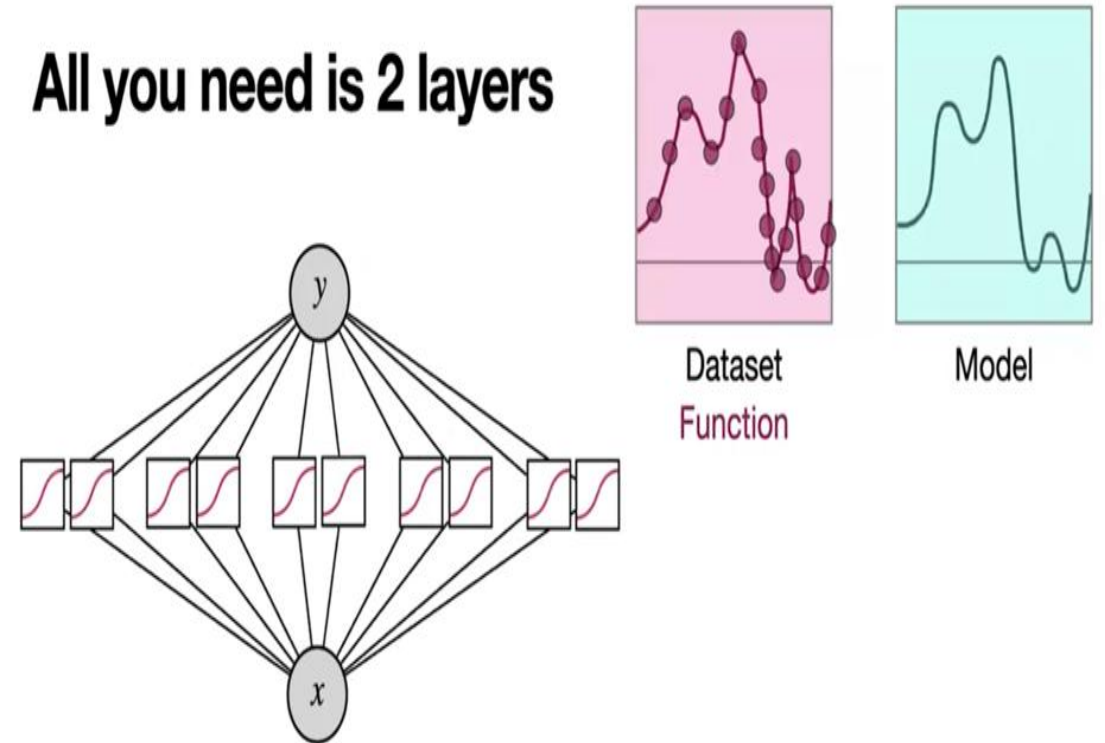


Why Learning Converges: Universal Approximation Theorem

All you need is 2 layers

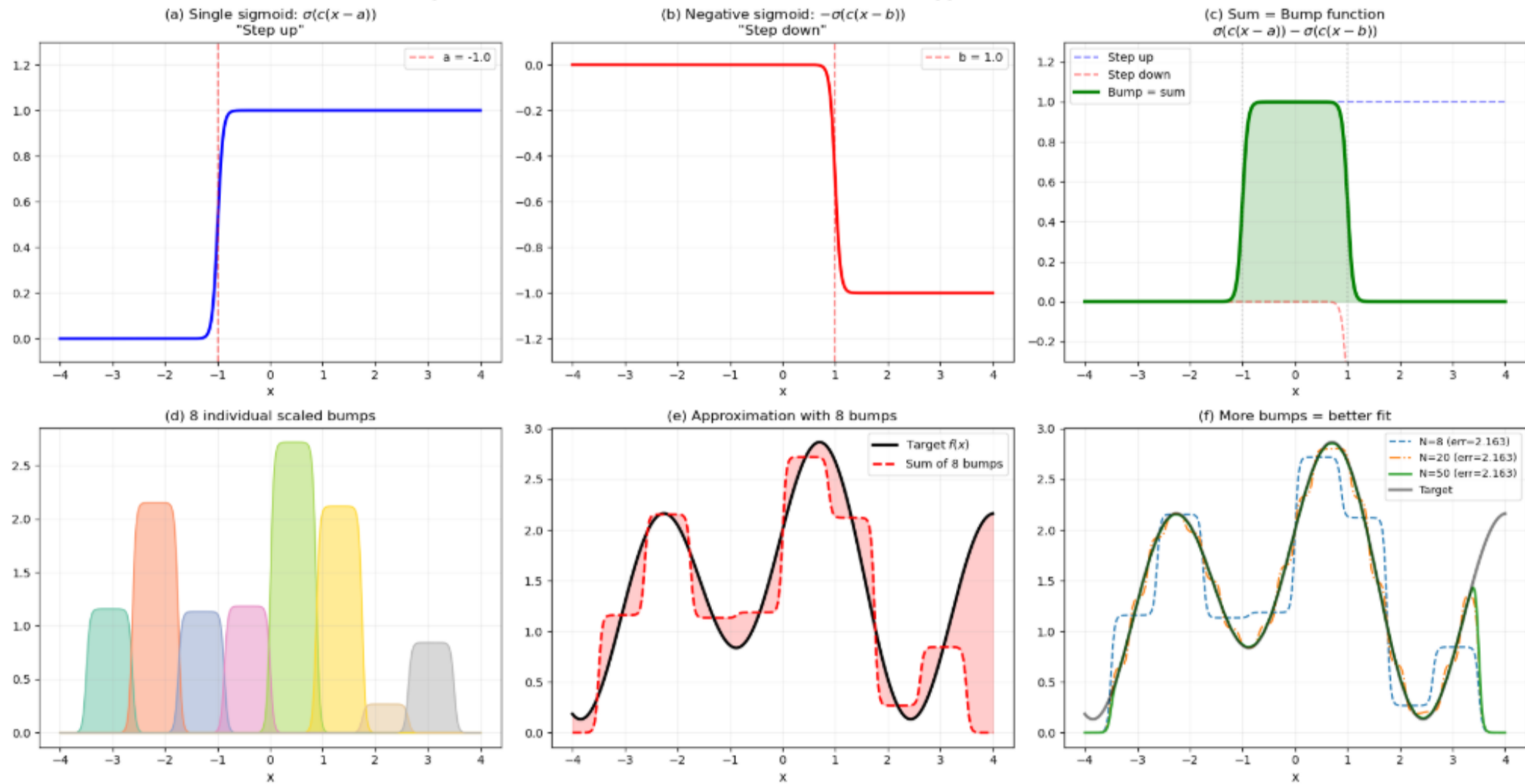


All you need is 2 layers

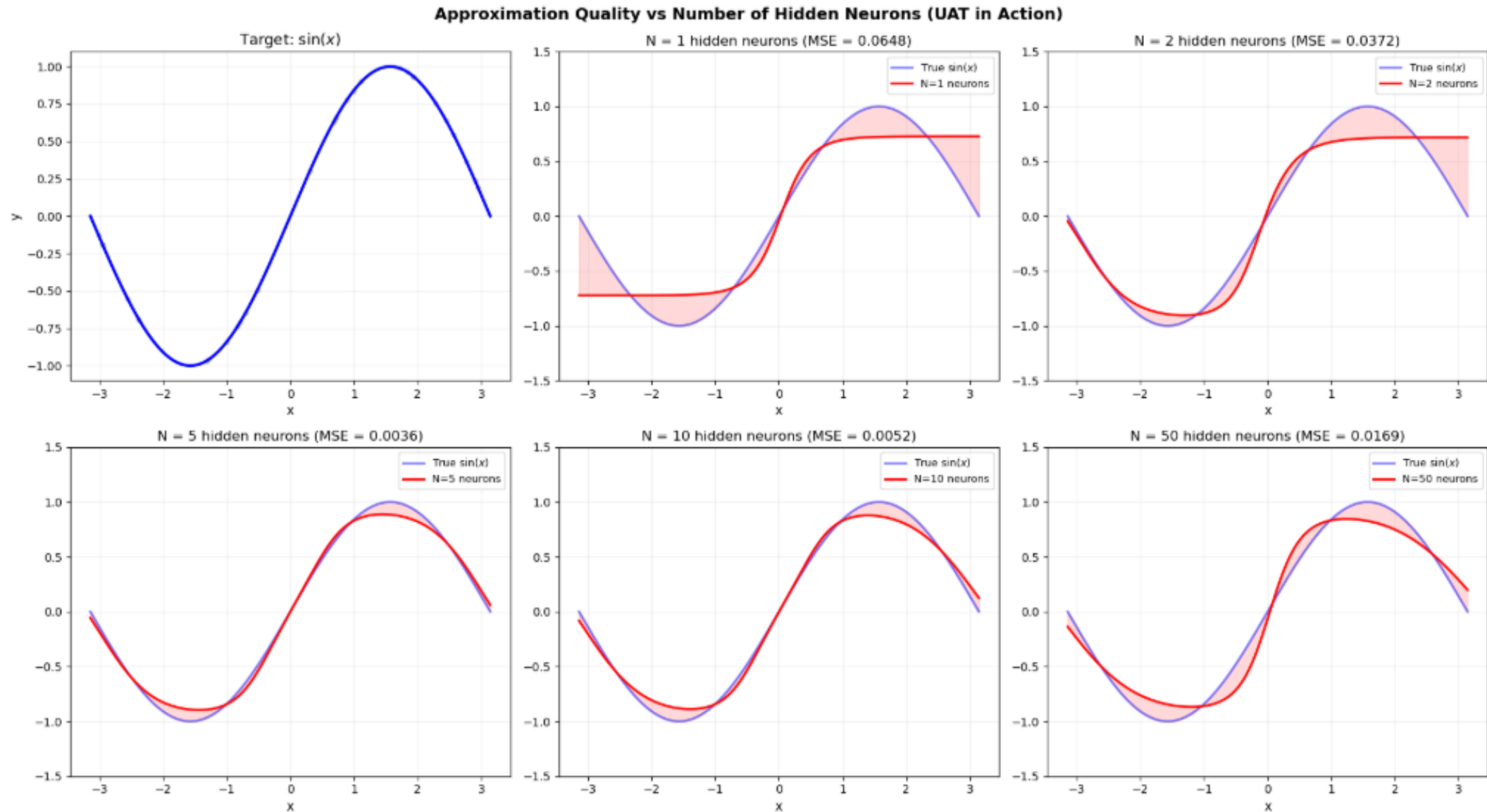


Why Learning Converges: Universal Approximation Theorem

Bump Function Construction: The Heart of the Universal Approximation Theorem



Why Learning Converges: Universal Approximation Theorem



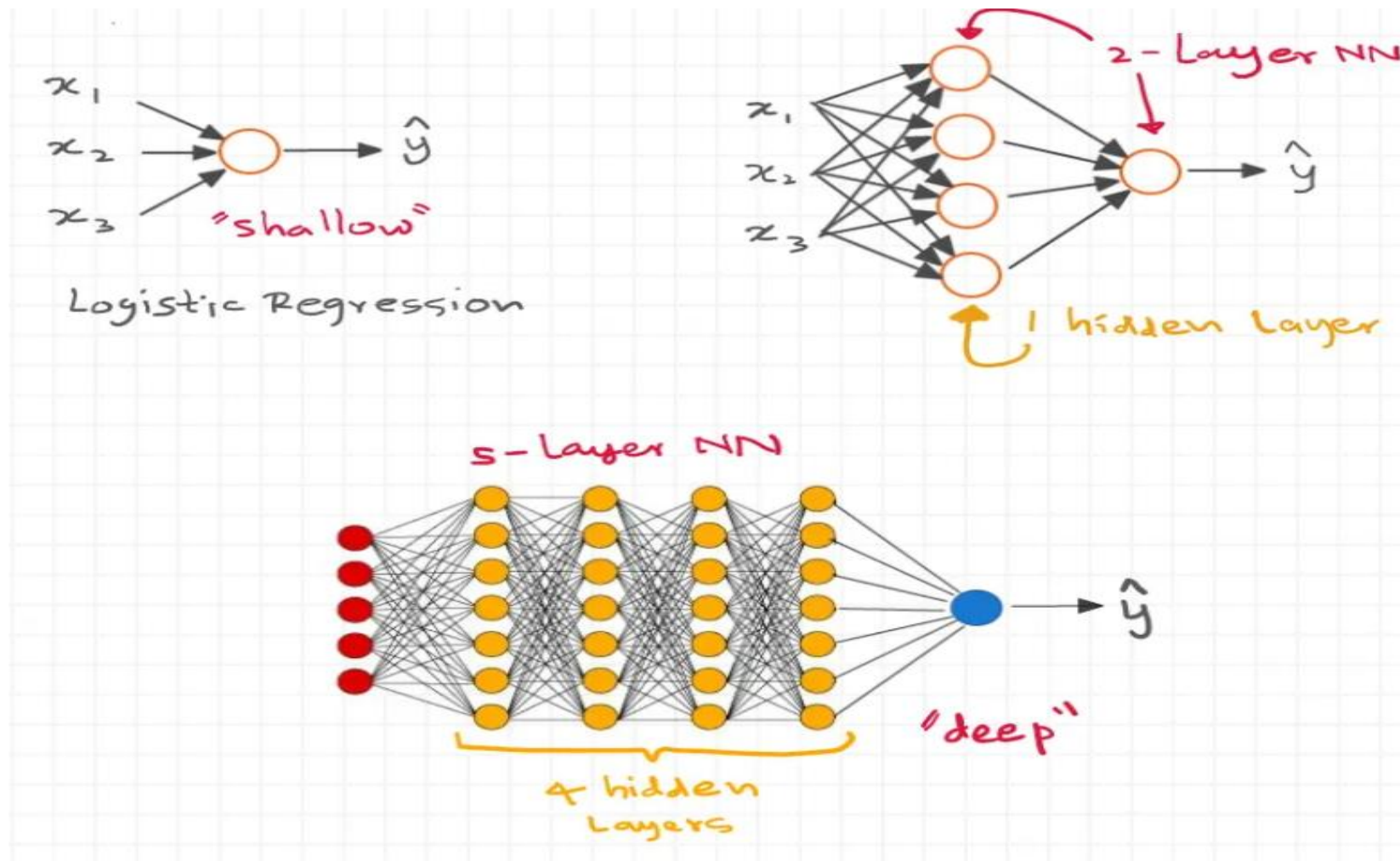
Why Learning Converges: Universal Approximation Theorem

- Practical Limitations
- While the Universal Approximation Theorem is mathematically elegant, it comes with certain practical limitations:
 1. Network Size and Efficiency
 - The theorem guarantees that a neural network can approximate any continuous function, but it doesn't specify the size of the network required.
 - In some cases, the number of neurons required to achieve a high degree of accuracy may be impractically large.
 2. Overfitting
 - The network may fit the training data perfectly, but it can also overfit, meaning it performs poorly on unseen data. Regularization techniques like dropout or early stopping are needed to mitigate this risk.
 3. Generalization
 - The theorem applies to the function approximation on the given training set, but it doesn't guarantee how well the network will generalize to new, unseen data. Techniques such as cross-validation help ensure good generalization.
 4. Training Difficulties
 - While the theorem assures that such an approximation exists, it doesn't provide insights on how to efficiently train the network. Gradient-based optimization methods can get stuck in local minima or saddle points, making it challenging to find the optimal solution.

5.3 Introduction to Deep Learning

- Deep Learning is a subset of Machine Learning that uses Artificial Neural Networks (ANNs) with many hidden layers to automatically learn complex patterns from data.
- Instead of manually designing features, Deep Learning automatically discovers useful features from raw data.
- Deep Learning is a machine learning technique that trains deep neural networks consisting of multiple hidden layers to solve complex tasks.
- The name Deep refers to the presence of many hidden layers.
- Example: In Traditional Neural Network we have Input → Hidden → Output while in Deep Neural Network we have Input → Hidden 1 → Hidden 2 → Hidden 3 → Hidden 4 → Output.
- The greater number of hidden layers enables the network to learn increasingly complex representations.

5.3 Introduction to Deep Learning



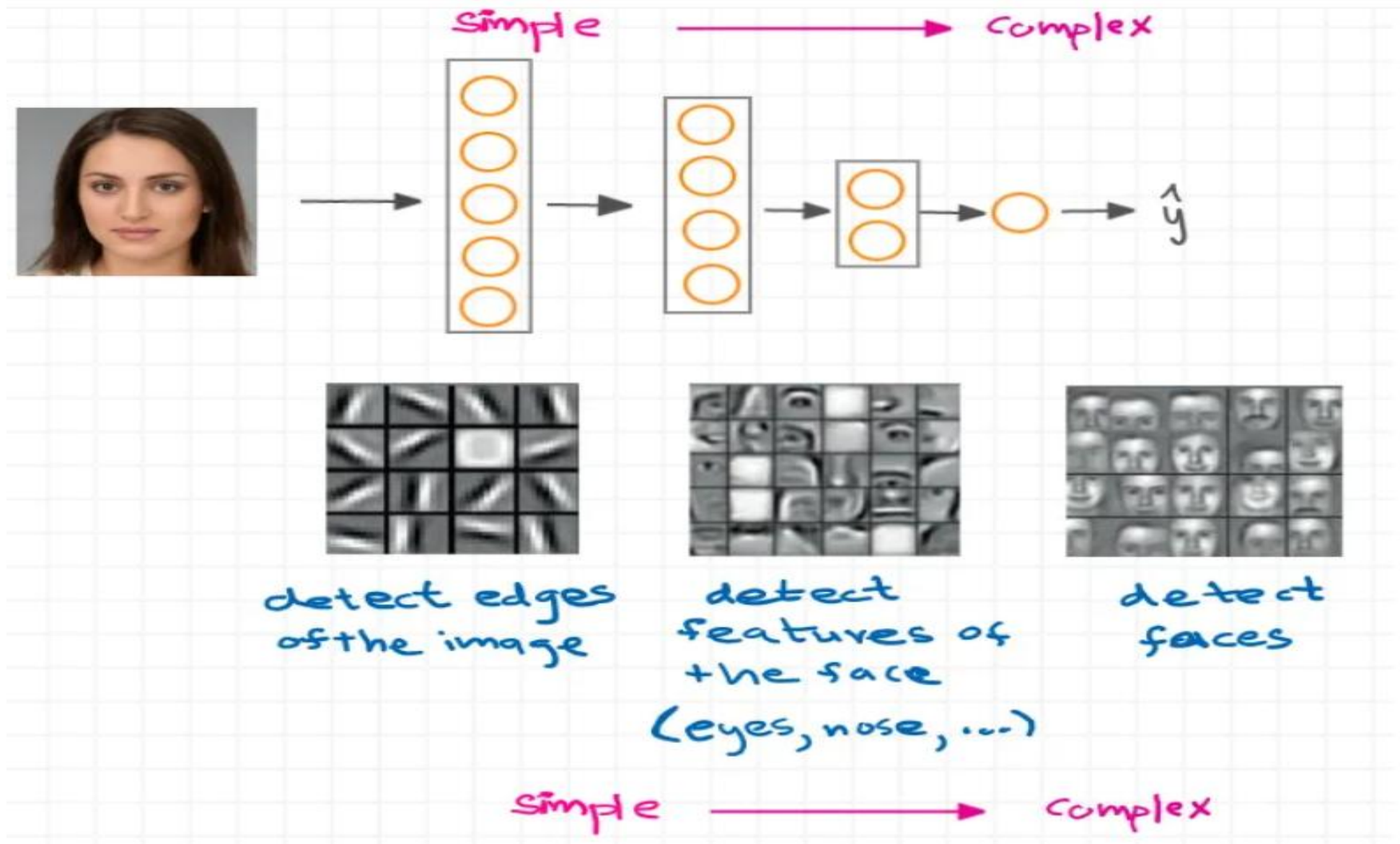
5.3 Introduction to Deep Learning

- The Traditional Machine Learning Pipeline: Feature Engineering
 - In classical Machine Learning, the process is linear:
 - Data: We have raw input (e.g., pixel values of an image, sensor readings).
 - Feature Extraction: This is the crucial, human-driven step. Human must use domain expertise to extract meaningful features from the raw data. For example, to detect a face, we might manually write code to detect edges, corners, and specific geometric ratios. This is often called feature engineering.
 - Classification/Regression: We feed these hand-crafted features into a shallow model (like SVM, Random Forest, or Logistic Regression) to map them to our output.
 - The performance of a classical Machine Learning model is heavily dependent on the quality of the human-engineered features.

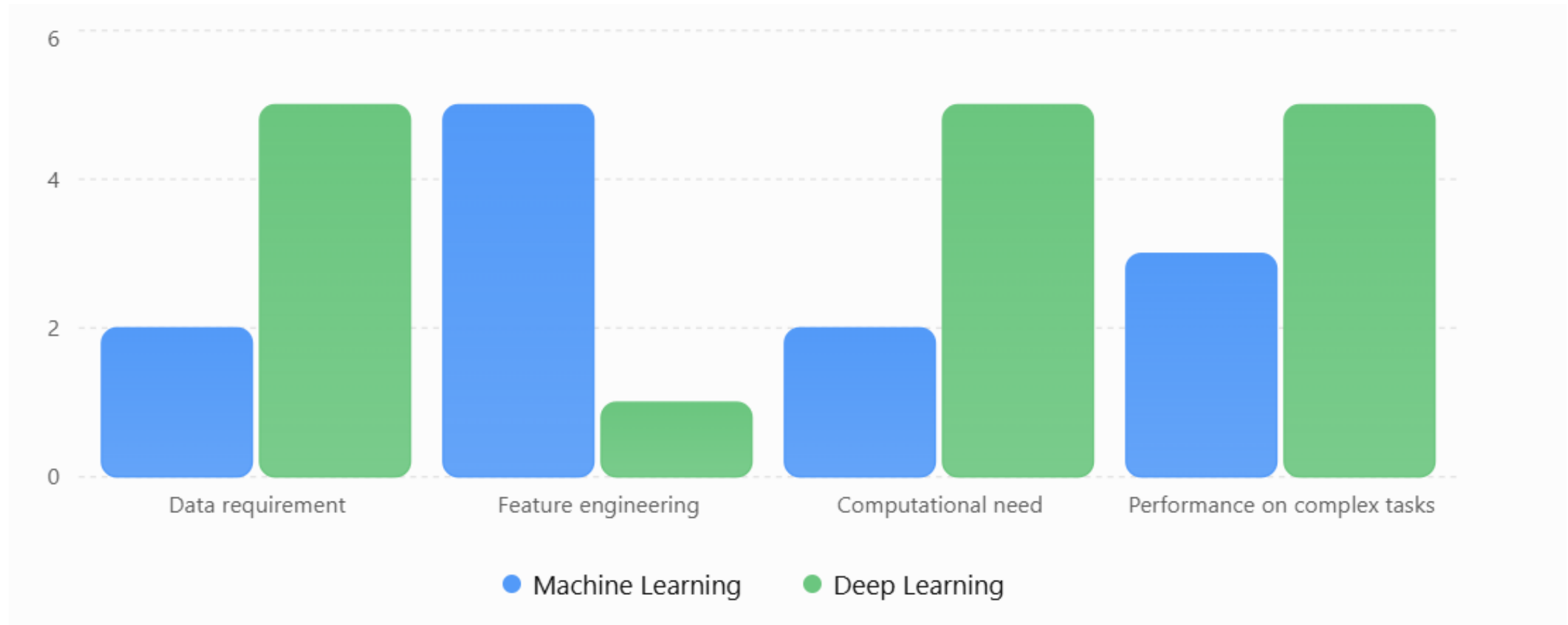
5.3 Introduction to Deep Learning

- The Deep Learning Pipeline: Feature Representation Learning
 - Deep Learning inverts this paradigm.
 - Data: Raw, minimally processed data (e.g., raw pixels).
 - Feature Extraction + Classification: This is handled jointly by the Deep Neural Network (DNN). The initial layers learn low-level features (edges, colors). Middle layers combine these into mid-level features (shapes, textures). Final layers learn high-level features (object parts, semantic concepts).
 - Output: The final layer produces the prediction.
 - The key distinction is automatic feature discovery. In Deep Learning, the algorithm learns the features from the data during training.
 - It is an end-to-end learning system.

5.3 Introduction to Deep Learning



Machine Learning Vs Deep Learning



Machine Learning Vs Deep Learning

Machine Learning	Deep Learning
Subset of AI	Subset of Machine Learning
Works well on small and medium datasets	Requires huge datasets
Manual feature engineering	Automatic feature learning
Requires less computational power (CPU)	Requires huge computational power (GPU)
Faster training	Longer training time
Easier to interpret	Difficult to interpret
Suitable for structured data	Excels with images, audio, video and text
Examples: Decision Tree, SVM, Naïve Bayes	Examples: CNN, RNN, LSTM, Transformer

Deep Neural Network Vs Traditional Neural Network

- Traditional Neural Network (often called a Shallow or Single Hidden Layer Network) consists of: An Input Layer, One Hidden Layer & An Output Layer.
- **The Distinction lies in "Depth":** A Deep Neural Network (DNN) is fundamentally a neural network with more than one hidden layer. It is a stack of multiple non-linear transformations.
- But why does depth matter?

1. Representational Power (The "Universal Approximation Theorem" vs. "Depth Efficiency")

- The Universal Approximation Theorem states that a network with a single hidden layer can approximate any continuous function, provided it has enough neurons. However, it might require an astronomically large number of neurons in that single layer. A deep network can represent complex functions exponentially more efficiently. It uses hierarchical composition.

2. Learning Hierarchies

- Traditional nets treat features globally. Deep nets build a hierarchy. They move from abstract to concrete. They learn the concept of "objectness" by first understanding the concept of "edge-ness."

3. The "Vanishing Gradient" Problem

- To train networks, we use Backpropagation, which relies on the chain rule. When you have many layers, you multiply many small gradients together, causing the gradient to become infinitesimally small (vanish). It becomes impossible to update the weights in the early layers. While Deep Learning uses ReLU activation functions, Batch Normalization, Residual Connections (ResNets), and improved Weight Initialization to train Deep Neural Network.

Building Blocks of Deep Neural Networks

- Deep Neural Network is composed of four fundamental building blocks:
- Layers: The core structure (Input, Hidden, Output).
- Weights and Biases: The learnable parameters.
- Activation Functions: Non-linear functions (Sigmoid, Tanh, ReLU) that allow the network to learn non-linear decision boundaries.
- Loss Function: The objective to optimize (e.g., Mean Squared Error, Cross-Entropy).
- All learning happens via two synchronized processes passing information back and forth through these layers.
 - Forward Propagation (The Prediction Phase)
 - Backward Propagation (The Learning Phase)

Building Blocks of Deep Neural Networks

- Part A: Forward Propagation (The Prediction Phase)
- Forward Propagation is the "Feed-Forward" pass. We take an input and push it through the network to get an output prediction.
- Let's mathematically define what happens in a single layer l .
- Linear Transformation (Weighted Sum): We take the input vector from the previous layer, $a^{[l-1]}$. We multiply it by our Weight matrix, $W^{[l]}$, and add our bias vector, $b^{[l]}$.
$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$
Here, $z^{[l]}$ is the pre-activation vector.
- Non-Linear Transformation (Activation): We pass $z^{[l]}$ through a non-linear activation function, $g^{[l]}$ (e.g., ReLU). $a^{[l]} = g^{[l]}(z^{[l]})$. This $a^{[l]}$ becomes the input for the next layer.
- We process data in batches. We utilize Linear Algebra libraries to compute this for all samples simultaneously. This is an Input $\rightarrow$ Weighted Sum $\rightarrow$ Activation $\rightarrow$ Output flow. At the end of the forward pass, we get our prediction $\hat{y}$.
- We then compare $\hat{y}$ to the true label y using a Loss Function $\mathcal{L}$. This gives us the "error" or "cost."

Building Blocks of Deep Neural Networks

- Part B: Backward Propagation (The Learning Phase)
- If Forward Propagation is asking the network a question, Backward Propagation is telling the network how wrong its answer was and *how* to adjust.
- Backpropagation is a clever application of the Chain Rule of Calculus. We calculate the Gradient of the Loss function with respect to every weight and bias in the network. Our objective is to find the values of weights W that minimize the Loss $\mathcal{L}$. We do this by calculating $\frac{\partial \mathcal{L}}{\partial W}$.
- The Output Gradient (The starting point): We start at the end and calculate the derivative of the Loss with respect to the final layer's output $\hat{y}$.
- Going Backwards (Layer by Layer): We propagate the error backward. For the final layer, we calculate how the loss changes with respect to the pre-activation $z^{[L]}$, then we calculate the gradients for $W^{[L]}$ and $b^{[L]}$.
- The Chain Rule in Action: To find the gradient for the weights of an earlier layer $W^{[l]}$, we multiply the gradients of all the layers *after* it.

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{\partial \mathcal{L}}{\partial a^{[L]}} \cdot \frac{\partial a^{[L]}}{\partial z^{[L]}} \cdot \frac{\partial z^{[L]}}{\partial a^{[L-1]}} \cdots \frac{\partial a^{[l]}}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

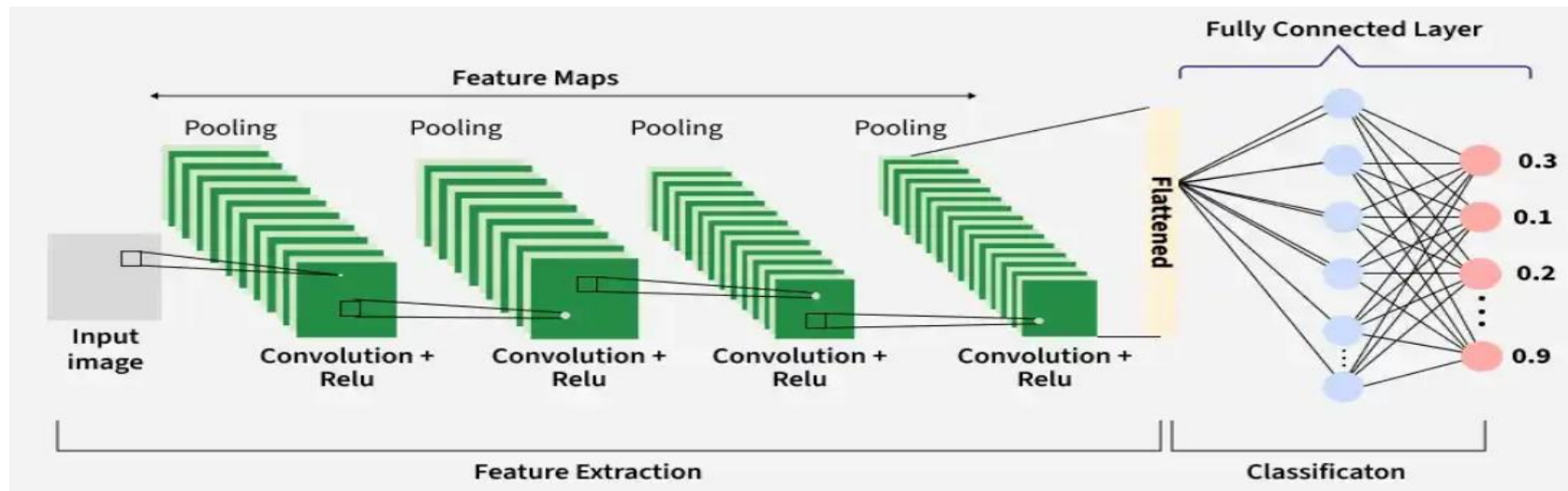
- Gradient Descent (The Update): Once we have the gradients, we perform a small, calculated step in the opposite direction of the gradient to reduce the loss. This is the optimization step.

$$W = W - \alpha \frac{\partial \mathcal{L}}{\partial W}$$

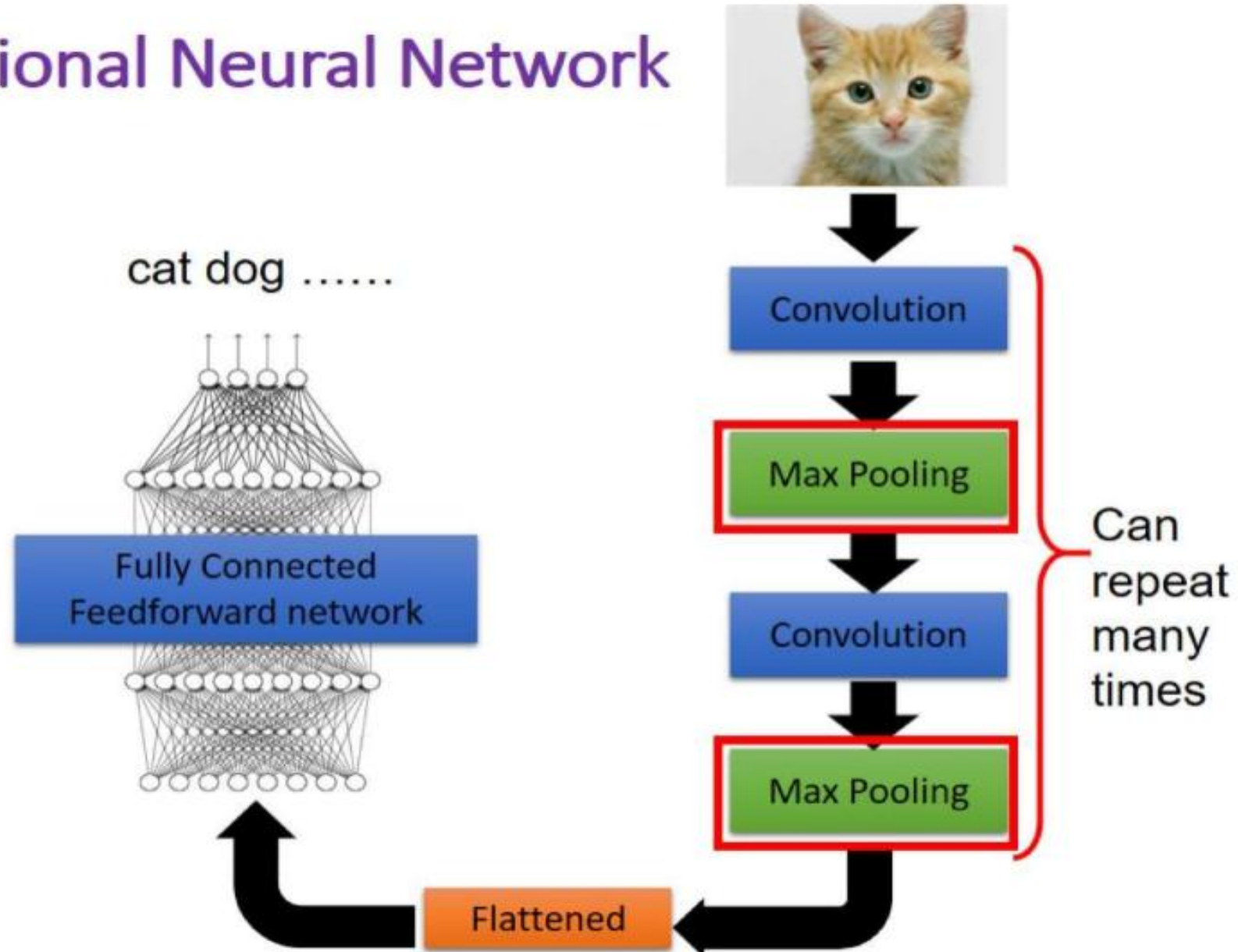
- Where α is our Learning Rate.

Introduction to Convolutional Neural Network (CNN)

- A Convolutional Neural Network (CNN) is a deep learning architecture that automatically learns hierarchical features from images using convolution operations.
- CNNs are extensively used in computer vision; have been applied also in acoustic modelling for automatic speech recognition.
- It capture spatial patterns using convolution operations that uses a small matrix called a filter (kernel) which slides across the image and computes feature values.
- It detect objects regardless of their position in the image & reduces computation by focusing on local regions instead of the entire image much like how humans focus on local details before recognizing an object.



Convolutional Neural Network



Introduction to Convolutional Neural Network (CNN)

A CNN typically consists of the following layers:

1. Input Layer

- The input layer receives the raw image data and passes it to the network for processing. In CNNs, input is typically a 3D volume (width $\times$ height $\times$ depth). Stores pixel values of the image (e.g., $32 \times 32 \times 3$ for RGB images). Preserves the spatial structure of the image for further feature extraction.

2. Convolutional Layer

- Applies filters to the input to create feature maps. Filters slide over the input data to detect specific features (e.g., edges in an image). Each filter specializes in detecting certain patterns or characteristics.

3. Activation Layer

- Introduces non-linearities to allow the network to learn complex patterns. Common activation functions include ReLU (Rectified Linear Unit), Sigmoid, and Tanh.

4. Pooling Layer

- Reduces the spatial dimensions (width and height) of the feature maps while retaining key information. Common pooling methods include Max Pooling (takes the maximum value) and Average Pooling (takes the average value). Helps in reducing computational complexity and controlling overfitting.

Introduction to Convolutional Neural Network (CNN)

5. Flattening

- Flattening converts the multi-dimensional feature maps into a one-dimensional vector after convolution and pooling. This vector is then passed to the fully connected layer for classification or regression.

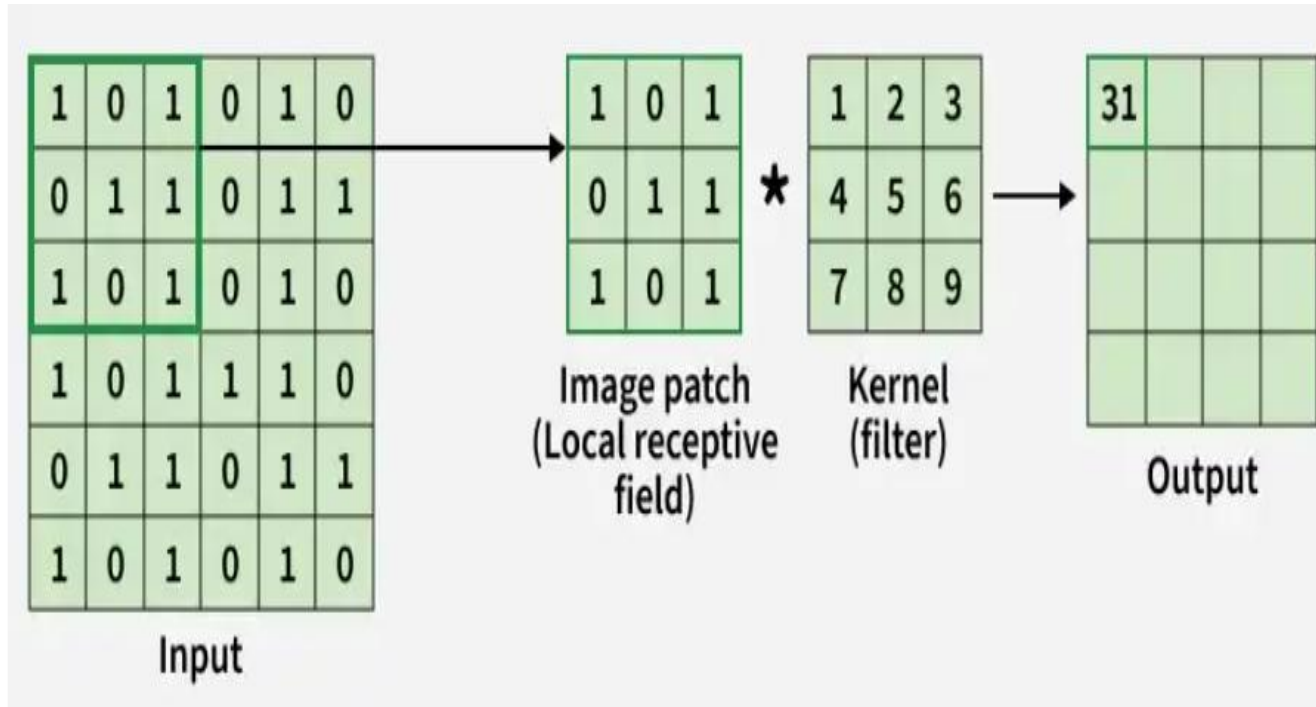
6. Fully Connected (FC) Layer

- Connects all neurons from the previous layer to the output neurons. Used for final classification or regression tasks.

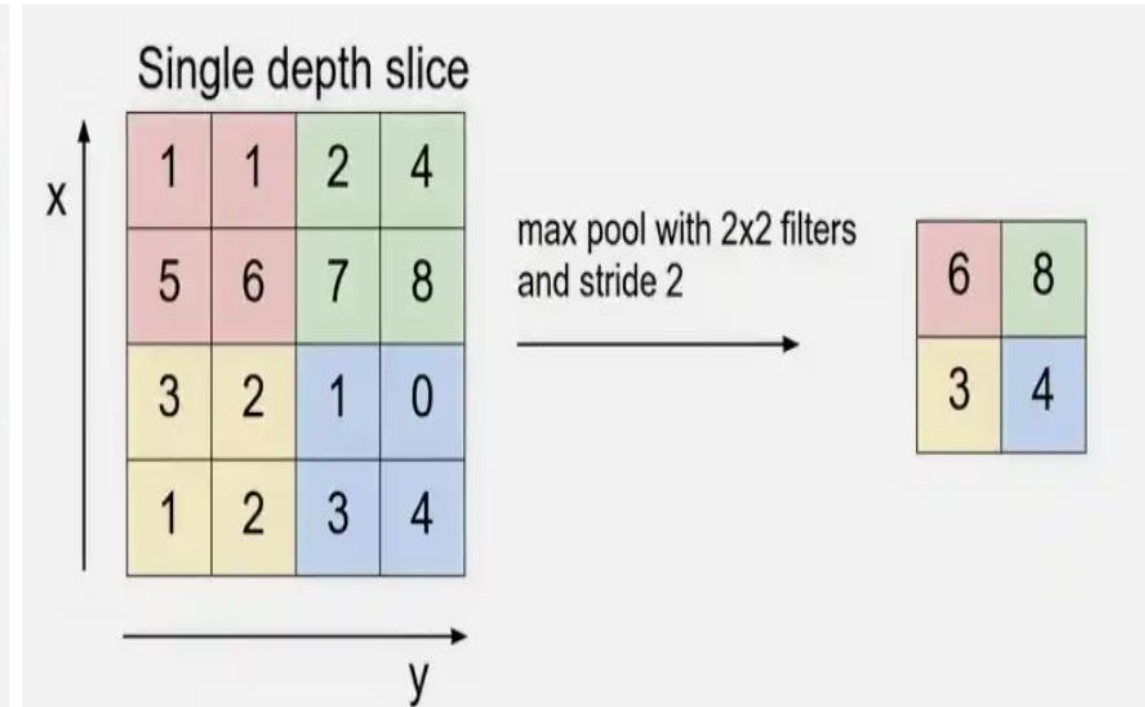
7. Output Layer

- Provides predictions (e.g., class probabilities) based on the features extracted.

Introduction to Convolutional Neural Network (CNN)



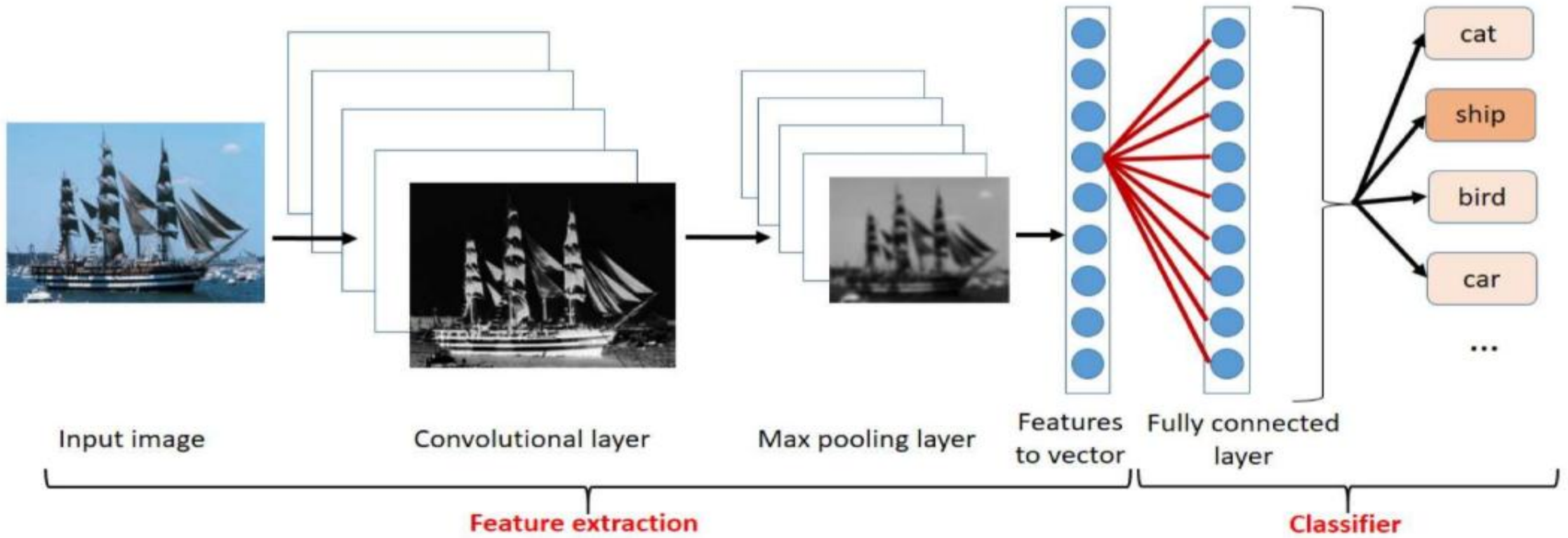
Convolution



Max Pooling

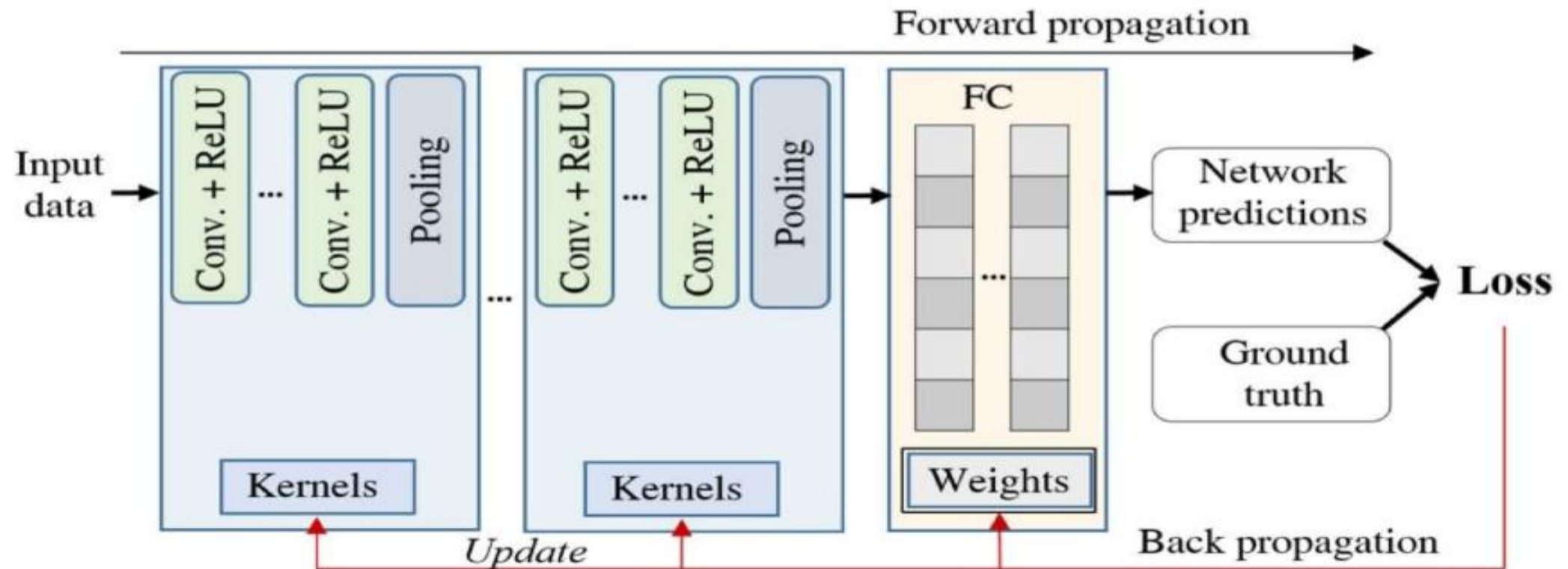
Introduction to Convolutional Neural Network (CNN)

Image classification



Introduction to Convolutional Neural Network (CNN)

Forward and Backward Pass in CNN



Application of Convolutional Neural Network (CNN)

1. Image Classification

- Identify the category of an image. Examples: Cat vs Dog, Healthy vs Diseased Plant & Handwritten Digit Recognition.

2. Face Recognition

- Recognize or verify a person's identity for smartphone face unlock, airport security & attendance systems.

3. Object Detection

- Locate and identify multiple objects in an image. Examples: Detect cars, pedestrians, and traffic signs for autonomous vehicles.

4. Medical Image Analysis

- Assist doctors by analyzing: X-rays, MRI scans, CT scans & Ultrasound images for tumor detection, pneumonia detection & fracture identification.

Application of Convolutional Neural Network (CNN)

5. Autonomous Vehicles

- CNNs help self-driving cars to detect lanes, recognize traffic signs, identify pedestrians and cyclists & avoid obstacles.

6. Handwritten Character Recognition

- For postal code recognition, bank cheque processing, etc.

7. Agriculture

- To detect disease in plant, fruit quality inspection & crop monitoring using drones.

8. Manufacturing and Quality Inspection

- CNNs detect product defects, surface cracks, missing components & packaging errors.

5.4. Concepts on Recurrent and Generative Neural Networks

- Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by retaining information from previous steps. They are especially effective for tasks where context and order matter.
- An RNN is a neural network in which the output from the previous time step is fed back into the network, enabling it to remember past information.
- The word recurrent means repeating. The same neural network is used repeatedly for each element in the sequence. For example, consider the sentence: "I love artificial intelligence." The RNN processes: Word 1 → Word 2 → Word 3 → Word 4 using the same weights at every time step.
- Designed for sequential and temporal data, it maintains memory of past inputs.
- Widely used in NLP, forecasting and speech tasks.

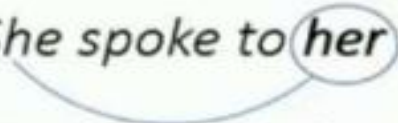
5.4. Concepts on Recurrent and Generative Neural Networks

- RNNs are used for time series analysis.
- When you need to make a prediction based on previous data, not only the current data. They make use of sequential information.

Examples:

- **Stock Price Prediction:** You can't just predict the price of the current month by just looking at one input (ex. The previous day). You need to observe the data and trend previously, to make a new prediction.
- **Text Generation:** You can't predict the next word based on the previous word. You need to look at the whole sentence to understand it's context.

*"She spoke to **her** husband"*



Another way to think about RNNs is that they have a "memory" which captures information about what has been calculated so far.

5.4. Concepts on Recurrent and Generative Neural Networks

- Components of RNN

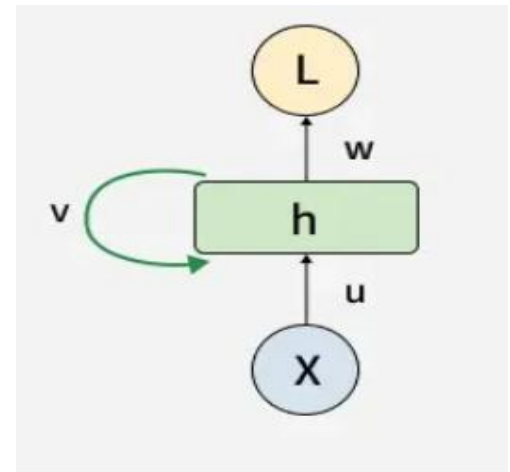
- An RNN consists of the following components:

1. Input Layer

- Receives sequential data. Examples: words in a sentence, daily stock prices, temperature readings & audio samples. Each input is processed one time step at a time.

2. Recurrent Neuron

- A recurrent neuron is the basic computational unit of an RNN. Unlike a neuron in a feed-forward network, it receives “Current input” & “Previous hidden state”.
- It combines these to produce: New hidden state & Output.
- Example if we are writing a story, then we remember the previous sentence while writing the next one. Similarly, the recurrent neuron remembers previous information using its hidden state.



Recurrent Neurons

5.4. Concepts on Recurrent and Generative Neural Networks

3. Hidden State

- The hidden state stores information learned from previous inputs. It acts as the network's memory. Example: "I went to the market because I needed vegetables" is a given sentence then while reading the word vegetables, the RNN still remembers earlier words like market and needed.
- Mathematical Representation:
 - At time step t : $h_t = f(W_x x_t + W_h h_{t-1} + b)$ where:
 - x_t = current input
 - h_{t-1} = previous hidden state
 - W_x = input weights
 - W_h = recurrent weights
 - b = bias
 - f = activation function (usually tanh)
 - The output is: $y_t = g(W_h h_t)$ where g is the output activation function.

5.4. Concepts on Recurrent and Generative Neural Networks

4. Recurrent Layer

- The recurrent layer consists of many recurrent neurons connected over time.
- These layer share same weights, reused at every time step, maintains hidden states & learns temporal relationships.
- This process is known as weight sharing, reducing the number of parameters.

5. Output Layer

- Gives final output of the system

5.4. Concepts on Recurrent and Generative Neural Networks

- RNN Unfolding or Unrolling
- RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.
- Although an RNN contains a loop, during training it is unrolled through time.
- For a sequence of four words:

$x_1 \rightarrow h_1 \rightarrow y_1$



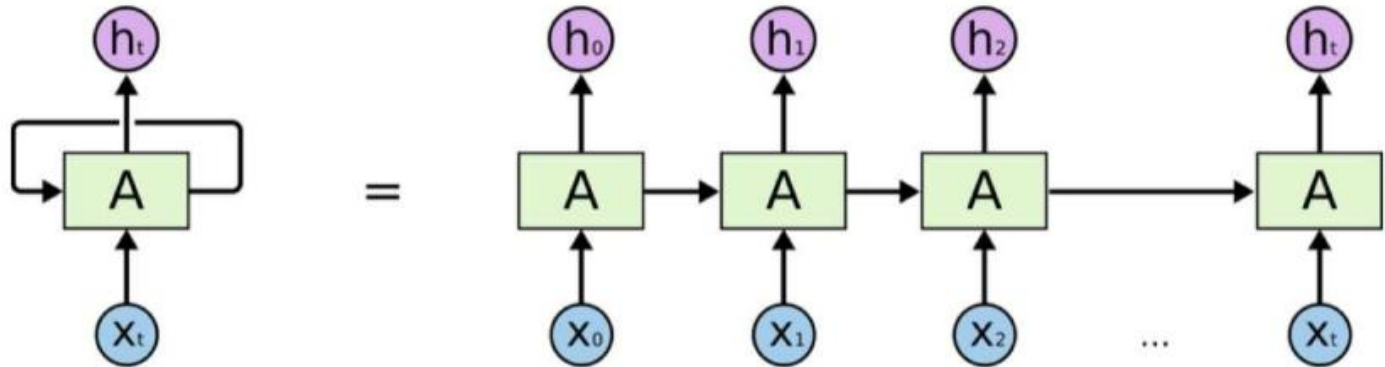
$x_2 \rightarrow h_2 \rightarrow y_2$



$x_3 \rightarrow h_3 \rightarrow y_3$



$x_4 \rightarrow h_4 \rightarrow y_4$

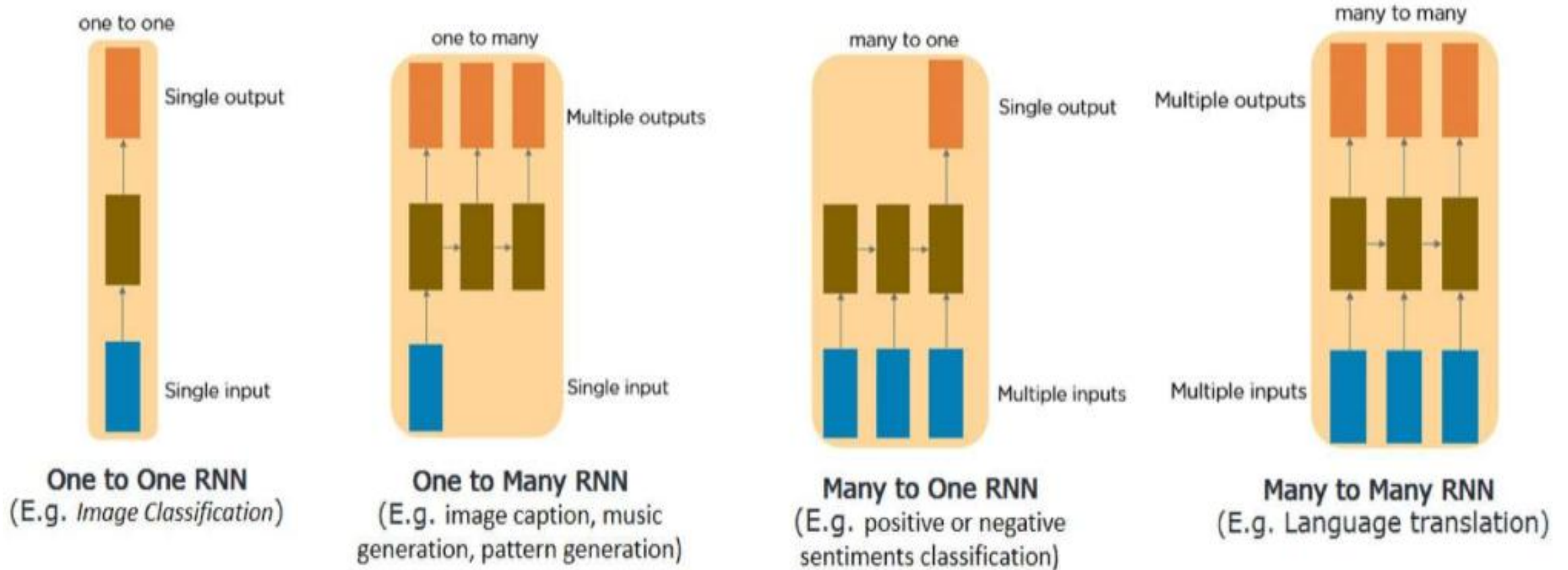


- The same network appears multiple times, but all copies share the same weights.

RNN Unfolding or Unrolling

- Suppose the input sequence is: "AI is amazing"
- Time Step 1:
 - Input: AI
 - Hidden state: h_1
 - Output: y_1
- Time Step 2:
 - Input: is
 - Previous hidden state: h_1
 - New hidden state: h_2
 - Output: y_2
- Time Step 3:
 - Input: amazing
 - Previous hidden state: h_2
 - New hidden state: h_3
 - Output: y_3
- Each hidden state carries information forward.

Types of Recurrent Neural Network



Applications of RNN

Applications

- 01 Translation
- 02 Text Summarization
- 03 Image Captioning
- 04 Conversational Agents
- 05 Language Modelling
- 06 Sentiment Analysis
- 07 Language Generation

Generative Neural Networks

- A generative model learns the probability distribution of the training data and generates new data samples that resemble the original data.
- Instead of answering: "What class is this?"
- It answers: "What could a new example from this class look like?"

History of Generative Model

The development of generative models has progressed through several stages:

1. Statistical Generative Models (1950s–1980s)

- Examples: Gaussian Models, Naïve Bayes, Hidden Markov Models
- Characteristics:
 - Strong mathematical assumptions.
 - Limited ability to model highly complex data.

2. Probabilistic Graphical Models (1980s–2000s)

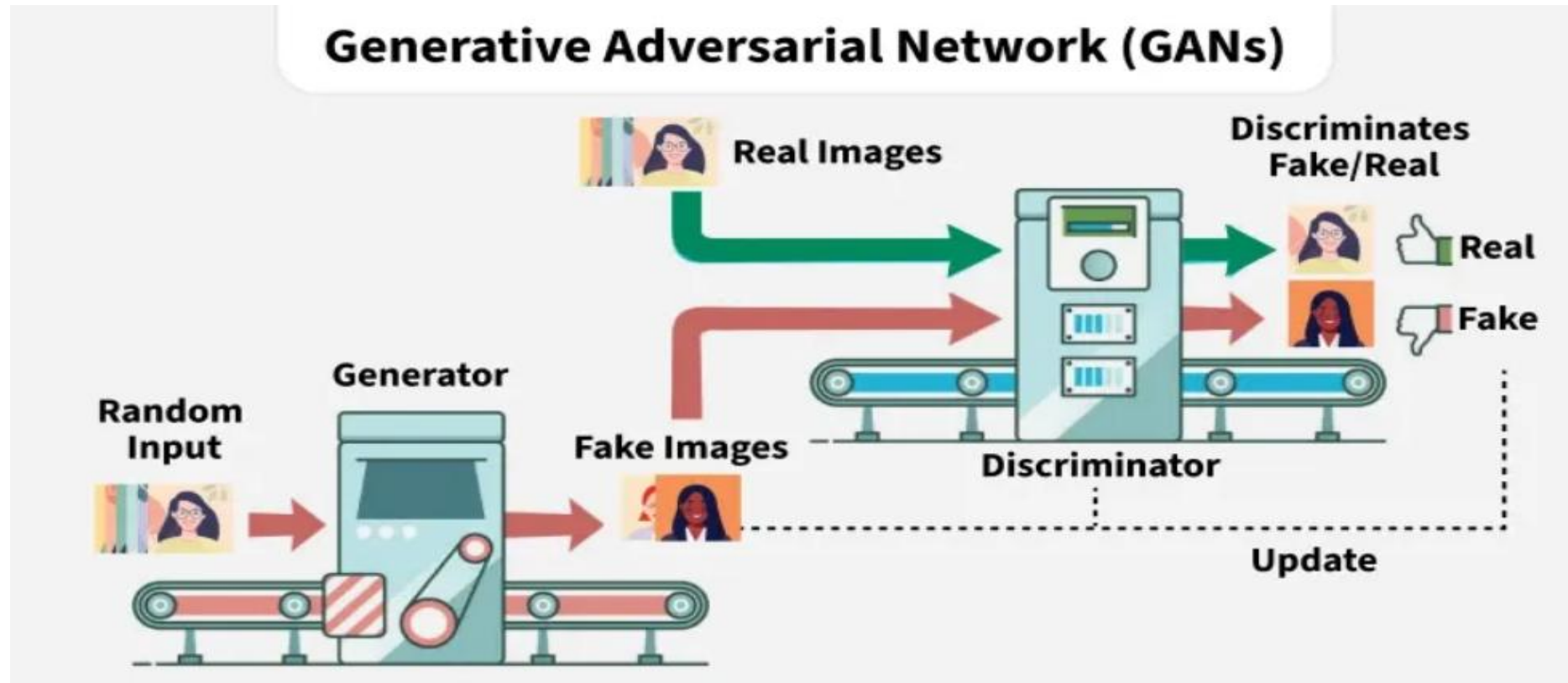
- Examples: Bayesian Networks , Markov Models , Hidden Markov Models
- Applications:
 - Speech recognition.
 - Natural language processing.
 - Bioinformatics.

3. Deep Generative Models (2010–Present)

- Examples: Autoencoders , Variational Autoencoders (VAE) , Generative Adversarial Networks (GAN) , Diffusion Models
- These models can generate realistic images , videos, music, speech, text

Generative Adversarial Networks (GANs)

- GANs are models that generate new, realistic data by learning from existing data.
- Introduced by Ian Goodfellow in 2014, they enable machines to create content like images, videos and music.
- A GAN consists of two neural networks (Generator & Discriminator) competing against each other. This competition leads to increasingly realistic generated data.



Generative Adversarial Networks (GANs)

- Architecture of GAN
- GAN consists of two neural networks the Generator and the Discriminator

1. Generator Model

- The generator is a deep neural network that takes random noise as input to generate realistic data samples like images or text. It learns the underlying data patterns by adjusting its internal parameters during training through backpropagation. Its objective is to produce samples that the discriminator classifies as real.
- Generator Loss Function: The generator tries to minimize this loss: $J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$

where:

- J_G measure how well the generator is fooling the discriminator.
- $G(z_i)$ is the generated sample from random noise z_i
- $D(G(z_i))$ is the discriminator's estimated probability that the generated sample is real.

The generator aims to maximize $D(G(z_i))$ meaning it wants the discriminator to classify its fake data as real (probability close to 1).

Generative Adversarial Networks (GANs)

2. Discriminator Model

- The discriminator is a binary classifier that distinguishes real data from generated samples. Through training, it refines its parameters to improve detection of fake data and when working with images, it uses convolutional layers to extract features and enhance classification accuracy.

- Discriminator Loss Function: The discriminator tries to minimize this loss:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- J_D measures how well the discriminator classifies real and fake samples.
- x_i is a real data sample.
- $G(z_i)$ is a fake sample from the generator.
- $D(x_i)$ is the discriminator's probability that x_i is real.
- $D(G(z_i))$ is the discriminator's probability that the fake sample is real.

The discriminator wants to correctly classify real data as real (maximize $\log D(x_i)$) and fake data as fake (maximize $\log(1 - D(G(z_i)))$)

Working of GAN

1. Generator's First Move

- The generator starts with a random noise vector like random numbers. It uses this noise as a starting point to create a fake data sample such as a generated image. The generator's internal layers transform this noise into something that looks like real data.

2. Discriminator's Turn

- The discriminator receives two types of data: Real samples from the actual training dataset & Fake samples created by the generator.
- D's job is to analyze each input and find whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 shows the data is likely real and 0 suggests it's fake.

3. Adversarial Learning

- If the discriminator correctly classifies real and fake data it gets better at its job. If the generator fools the discriminator by creating realistic fake data, it receives a positive update and the discriminator is penalized for making a wrong decision.

Working of GAN

4. Generator's Improvement

- Each time the discriminator mistakes fake data for real, the generator learns from this success. Through many iterations, the generator improves and creates more convincing fake samples.

5. Discriminator's Adaptation

- The discriminator also learns continuously by updating itself to better spot fake data. This constant back-and-forth makes both networks stronger over time.

6. Training Progression

- As training continues, the generator becomes highly proficient at producing realistic data. Ideally, the generator improves to the point where the discriminator finds it difficult to distinguish real from generated data, although in practice GAN training can be unstable and may not always reach this balance. At this point, the generator can produce high-quality synthetic data that can be used for different applications.

Applications of GANs

1. Image Generation

- Generate realistic human faces, animals, or scenery.

2. Image Super-Resolution

- Convert low-resolution images into high-resolution images.

3. Image-to-Image Translation

- Examples: Sketch → Photograph, Black-and-white → Color image, Day → Night

4. Medical Imaging

- Generate synthetic medical images to support research and training.

Applications of GANs

5. Data Augmentation

- Create additional training samples when real data is limited.

6. Deepfake Generation

- Generate realistic synthetic faces and videos (raising important ethical concerns).

7. Art and Design

- Generate paintings, logos, game assets, and digital artwork.

8. Fashion and Product Design

- Create new clothing styles, accessories, and product prototypes.

References

- Russell, S., Norvig, P. (2021). Artificial intelligence: A modern approach. Pearson.
- Rich, E., Knight, K., Nair, S. B. (2009). Artificial intelligence. McGraw-Hill Education
- Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep learning. MIT Press.